UNIVERSITY OF PATRAS

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

DIVISION OF ELECTRONICS AND EMBEDDED SYSTEMS

JPEG COMPRESSION ON LOW POWER RISC-V

D I P L O M A T H E S I S

Dimosthenis, Karamparpas

SUPERVISOR: GEORGE, THEODORIDIS

PATRAS – OCTOBER,2026

University of Patras, Department of Electrical and Computer Engineering.

Dimosthenis, Karamparpas

© 2026 – All rights reserved

The whole work is an original work, produced by Dimosthenis, Karamparpas, and does not violate the rights of third parties in any way. If the work contains material which has not been produced by him/her, this is clearly visible and is explicitly mentioned in the text of the work as a product of a third party, noting in a similarly clear way his/her identification data, while at the same time confirming that in case of using original graphics representations, images, graphs, etc., has obtained the unrestricted permission of the copyright holder for the inclusion and subsequent publication of this material.

CERTIFICATION

It is certified that the Diploma Thesis titled

DIPLOMA THESIS TITLE

of the Department of Electrical and Computer Engineering student

Dimosthenis, Karamparpas

Registration Number: 1089862

was presented publicly at the Department of Electrical and Computer Engineering at

06/10/2026

and was examined by the following examining committee:

George Theodoridis, Associate Professor, Division of Electronics and Computers (supervisor)

Odysseas Koufopavlou, Professor, Division of Electronics and Computers (committee member)

Michael Birbas, Associate Professor, Division of Electronics and Computers (committee member)

The Supervisor	The Director of the Division
George Theodoridis	George Theodoridis
Associate Professor	Associate Professor

ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΑΤΡΩΝ

ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΤΕΧΝΟΛΟΓΙΑΣ ΥΠΟΛΟΓΙΣΤΩΝ

ΤΟΜΕΑΣ ΗΛΕΚΤΡΟΝΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ

ΣΥΜΠΙΕΣΗ JPEG

ΣΕ ΧΑΜΗΛΗΣ ΕΝΕΡΓΕΙΑΣ RISC-V

Δ Ι Π Λ Ω Μ Α Τ Ι Κ Η Ε Ρ Γ Α Σ Ι Α

ΔΗΜΟΣΘΕΝΗΣ ΚΑΡΑΜΠΑΡΠΑΣ

ΕΠΙΒΛΕΠΩΝ: ΓΕΩΡΓΙΟΣ ΘΕΟΔΩΡΙΔΗΣ

ΠΑΤΡΑ – ΟΚΤΩΒΡΙΟΣ 2026

Πανεπιστήμιο Πατρών, Τμήμα Ηλεκτρολόγων Μηχανικών και Τεχνολογίας Υπολογιστών.

Δημοσθένης Καραμπάρπας

© 2026 – Με την επιφύλαξη παντός δικαιώματος

Το σύνολο της εργασίας αποτελεί πρωτότυπο έργο, παραχθέν από τον Δημοσθένη Καραμπάρπα, και δεν παραβιάζει δικαιώματα τρίτων καθ' οιονδήποτε τρόπο. Αν η εργασία περιέχει υλικό, το οποίο δεν έχει παραχθεί από τον/την ίδιο/α, αυτό είναι ευδιάκριτο και αναφέρεται ρητώς εντός του κειμένου της εργασίας ως προϊόν εργασίας τρίτου, σημειώνοντας με παρομοίως σαφή τρόπο τα στοιχεία ταυτοποίησής του, ενώ παράλληλα βεβαιώνει πως στην περίπτωση χρήσης αυτούσιων γραφικών αναπαραστάσεων, εικόνων, γραφημάτων κ.λπ., έχει λάβει τη χωρίς περιορισμούς άδεια του κατόχου των πνευματικών δικαιωμάτων για την συμπερίληψη και επακόλουθη δημοσίευση του υλικού αυτού.

ΠΙΣΤΟΠΟΙΗΣΗ

Πιστοποιείται ότι η Διπλωματική Εργασία με τίτλο

ΤΙΤΛΟΣ ΔΙΠΛΩΜΑΤΙΚΗΣ ΕΡΓΑΣΙΑΣ

του φοιτητή του Τμήματος Ηλεκτρολόγων Μηχανικών και Τεχνολογίας Υπολογιστών

ΔΗΜΟΣΘΕΝΗΣ ΚΑΡΑΜΠΑΡΠΑΣ ΤΟΥ ΑΝΤΩΝΙΟΥ

Αριθμός Μητρώου: 1089862

Παρουσιάστηκε δημόσια στο Τμήμα Ηλεκτρολόγων Μηχανικών και Τεχνολογίας Υπολογιστών στις

06/11/2026

και εξετάστηκε από την ακόλουθη εξεταστική επιτροπή:

Γεώργιος Θεοδωρίδης, Αναπληρωτής Καθηγητής, Τμήμα Ηλεκτρολόγων Μηχανικών και Τεχνολογίας Υπολογιστών (επιβλέπων)

Κουφοπαύλου Οδυσσέας, Καθηγητής, Τμήμα Ηλεκτρολόγων Μηχανικών και Τεχνολογίας Υπολογιστών (μέλος επιτροπής)

Μιχάλης Μπίρμπας, Αναπληρωτής Καθηγητής, Τμήμα Ηλεκτρολόγων Μηχανικών και Τεχνολογίας Υπολογιστών (μέλος επιτροπής)

Ο/Η Επιβλέπων/ουσα	Ο/Η Διευθυντής/τρια του Τομέα
Γεώργιος Θεοδωρίδης	Γεώργιος Θεοδωρίδης
Αναπληρωτής Καθηγητής	Αναπληρωτής Καθηγητής

Abstract

This thesis explores the optimization and parallelization of the JPEG compression algorithm on the Snitch cluster, a highly specialized, heterogeneous RISC-V multi-core architecture. Designed for extreme energy efficiency and compute density, the Snitch architecture relies heavily on Tightly Coupled Data Memory (TCDM), Stream Semantic Registers (SSR), and asynchronous Data Movers (DM) rather than traditional hardware caches.

The research is divided into two primary phases: single-core micro-architectural optimization and cluster-level parallelization. Initially, the core JPEG pipeline was evaluated across Double Precision (FP64) and Packed SIMD (FP16) implementations. Despite the theoretical throughput advantages of half-precision vectorization, the FP64 implementation was selected as the optimal single-core baseline. By leveraging hardware-software co-design—such as zero-cost matrix transpositions via 4D SSR strides, Indirect SSRs (ISSR) for zig-zag routing, and hardware queues (copift)—the FP64 data path completely decoupled the Floating-Point Unit (FPU) from integer ALU bottlenecks, reducing execution time to 33,129 cycles per Minimum Coded Unit (MCU).

Subsequently, the pipeline was transitioned to the 8-core cluster utilizing a Tile-by-Tile Producer-Consumer model. Memory bandwidth and spatial contention were addressed by offloading all DMA transfers to the dedicated DM core and implementing Dual-Level Bank Staggering to eliminate TCDM lockstep bank collisions without wasting capacity on memory padding. Furthermore, instruction scheduling (software pipelining) was applied to the scalar Huffman encoder to mask RAW_LOAD_USE pipeline latencies.

Ultimately, the fully optimized multi-core implementation achieved a total compute time of 32,508 cycles to process 8 MCUs simultaneously. This yields a super-linear architectural speedup of 8.15x, proving that combining asynchronous memory orchestration with aggressive instruction-level prefetching allows the compute cores to reach peak mathematical throughput entirely unhindered by memory topology limits.

ΕΚΤΕΤΑΜΕΝΗ ΕΛΛΗΝΙΚΗ ΠΕΡΙΛΗΨΗ

ΤΙΤΛΟΣ ΔΙΠΛΩΜΑΤΙΚΗΣ ΕΡΓΑΣΙΑΣ

ΟΝΟΜΑΤΕΠΩΝΥΜΟ ΦΟΙΤΗΤΗ:

ΚΑΡΑΜΠΑΡΠΑΣ ΔΗΜΟΣΘΕΝΗΣ

ΟΝΟΜΑΤΕΠΩΝΥΜΟ ΕΠΙΒΛΕΠΟΝΤΟΣ:

ΘΕΟΔΩΡΙΔΗΣ ΓΕΩΡΓΙΟΣ

1. Εισαγωγή

Οι συσκευές του Διαδικτύου των Πραγμάτων (Internet of Things – IoT), δηλαδή μικρές διασυνδεδεμένες συσκευές όπως αισθητήρες και κάμερες, καταγράφουν μεγάλους όγκους εικόνας. Τους όγκους αυτούς πρέπει να τους επεξεργαστούν ή να τους μεταδώσουν με πολύ περιορισμένη ενέργεια και ταχύτητα σύνδεσης. Γι' αυτό η συμπίεση εικόνας είναι κρίσιμη εργασία για συστήματα χαμηλής κατανάλωσης, με πιο διαδεδομένο πρότυπο το JPEG (Joint Photographic Experts Group, από το όνομα της επιτροπής που το ανέπτυξε).

Οι συνηθισμένοι επεξεργαστές δαπανούν μεγάλο μέρος της ενέργειάς τους σε βοηθητικούς μηχανισμούς και όχι στους ίδιους τους υπολογισμούς:

- **Κρυφές μνήμες (caches).** Μικρές γρήγορες μνήμες που κρατούν αυτόματα αντίγραφα δεδομένων.
- **Πρόβλεψη διακλαδώσεων.** Η εκ των προτέρων εκτίμηση της πορείας που θα ακολουθήσει το πρόγραμμα.

Η εργασία μελετά τη βελτιστοποίηση του JPEG στη συστοιχία Snitch, ένα ερευνητικό σύστημα πολλαπλών επεξεργαστικών πυρήνων. Η Snitch βασίζεται στην ανοιχτή αρχιτεκτονική επεξεργαστών RISC-V (Reduced Instruction Set Computer, πέμπτη γενιά). Δεν διαθέτει κρυφές μνήμες δεδομένων. Αντί γι' αυτές χρησιμοποιεί:

- **Τοπική μνήμη.** Μια μικρή, πολύ γρήγορη μνήμη, την οποία ελέγχει ρητά το πρόγραμμα.
- **Μηχανισμοί ροής δεδομένων.** Ειδικά κυκλώματα που μεταφέρουν δεδομένα αυτόματα προς και από τις μονάδες υπολογισμού.
- **Πυρήνας μεταφοράς δεδομένων.** Ένας ξεχωριστός πυρήνας που αναλαμβάνει αποκλειστικά τη μεταφορά δεδομένων.

Η σχεδίαση αυτή είναι πολύ αποδοτική ενεργειακά, αλλά μεταφέρει στο λογισμικό την ευθύνη για την οργάνωση των δεδομένων και τον συγχρονισμό των πυρήνων. Η εργασία εξετάζει πώς ο JPEG

μπορεί να αναδιαρθρωθεί ώστε να αξιοποιεί πλήρως αυτό το υλικό. Ακολουθεί δύο άξονες: τη βελτιστοποίηση σε έναν πυρήνα και την επέκταση σε οκτώ παράλληλους πυρήνες.

2. Ο Αλγόριθμος JPEG

Το JPEG απορρίπτει πληροφορία που το ανθρώπινο μάτι δεν αντιλαμβάνεται. Εκμεταλλεύεται το γεγονός ότι το μάτι είναι πιο ευαίσθητο στη φωτεινότητα από ό,τι στο χρώμα, και στις ομαλές μεταβάσεις από ό,τι στις λεπτές λεπτομέρειες. Τα στάδιά του είναι:

- **Αλλαγή χρωματικής αναπαράστασης.** Η εικόνα μετατρέπεται από κόκκινο, πράσινο και μπλε σε μία συνιστώσα φωτεινότητας και δύο συνιστώσες χρώματος. Η ανάλυση του χρώματος υποδιπλασιάζεται οριζόντια και κάθετα, κάτι που μειώνει τα δεδομένα κατά 50% χωρίς ορατή διαφορά. Η εικόνα χωρίζεται σε μπλοκ 8×8 εικονοστοιχείων (pixels), που ομαδοποιούνται σε μονάδες επεξεργασίας 16×16.

- **Μετασχηματισμός συχνότητας.** Ο Διακριτός Μετασχηματισμός Συνημιτόνου μετατρέπει κάθε μπλοκ σε 64 συντελεστές. Κάθε συντελεστής περιγράφει πόσο συμμετέχει ένα μοτίβο στο μπλοκ, από τη μέση φωτεινότητα έως τις λεπτότερες λεπτομέρειες. Ο μετασχηματισμός υπολογίζεται σε δύο περάσματα, πρώτα κατά γραμμές και μετά κατά στήλες. Στις φυσικές εικόνες η πληροφορία συγκεντρώνεται σε λίγους συντελεστές.

- **Κβαντοποίηση.** Είναι το μόνο στάδιο όπου χάνεται πληροφορία. Κάθε συντελεστής διαιρείται με μια σταθερά και στρογγυλοποιείται. Οι συντελεστές λεπτής λεπτομέρειας διαιρούνται με μεγάλους αριθμούς και οι περισσότεροι μηδενίζονται.

- **Τελική κωδικοποίηση.** Οι συντελεστές διατάσσονται με διαγώνια σάρωση («ζιγκ-ζαγκ») ώστε τα μηδενικά να συγκεντρωθούν στο τέλος. Για τη μέση φωτεινότητα αποθηκεύεται μόνο η διαφορά της από το προηγούμενο μπλοκ, και οι σειρές μηδενικών αντικαθίστανται από το πλήθος τους. Τέλος, η κωδικοποίηση Huffman δίνει σύντομους κωδικούς στα συχνά σύμβολα και μακρύτερους στα σπάνια.

3. Η Αρχιτεκτονική Snitch

Η Snitch αναπτύχθηκε από το Ομοσπονδιακό Πολυτεχνείο της Ζυρίχης και το Πανεπιστήμιο της Μπολόνια. Βασίζεται στον διαχωρισμό ελέγχου και

υπολογισμού, και κάθε πυρήνας αποτελείται από δύο μέρη:

- **Τμήμα ελέγχου.** Ένα πολύ απλό τμήμα που χειρίζεται μόνο τη ροή του προγράμματος.
- **Μονάδα Κινητής Υποδιαστολής.** Εκτελεί πράξεις με δεκαδικούς αριθμούς. Μπορεί να κάνει πολλαπλασιασμό και πρόσθεση σε μία εντολή, καθώς και να επεξεργάζεται ταυτόχρονα πολλούς μικρότερους αριθμούς μέσα σε έναν καταχωρητή (διανυσματική επεξεργασία).

Η συστοιχία έχει οκτώ υπολογιστικούς πυρήνες και έναν ένατο πυρήνα που ελέγχει τη μηχανή Άμεσης Πρόσβασης στη Μνήμη. Η μηχανή αυτή αντιγράφει μεγάλα τμήματα δεδομένων χωρίς να απασχολεί τους υπόλοιπους πυρήνες.

Όλοι οι πυρήνες μοιράζονται μια τοπική μνήμη 128 KB, χωρισμένη σε 32 ανεξάρτητες τράπεζες μνήμης. Οι διαδοχικές διευθύνσεις μοιράζονται κυκλικά στις τράπεζες. Κάθε τράπεζα εξυπηρετεί ένα αίτημα ανά κύκλο ρολογιού, οπότε όταν δύο αιτήματα στοχεύουν ταυτόχρονα την ίδια τράπεζα, το ένα περιμένει. Αυτό ονομάζεται σύγκρουση τραπεζών.

Τρεις μηχανισμοί υλικού είναι κεντρικοί για την εργασία:

- **Καταχωρητές ροής.** Συνδέονται απευθείας με τη μνήμη: κάθε ανάγνωση φέρνει αυτόματα το επόμενο δεδομένο και κάθε εγγραφή το αποθηκεύει. Η σειρά των διευθύνσεων προγραμματίζεται εκ των προτέρων, έως και σε τέσσερις διαστάσεις. Έτσι δεν χρειάζονται εντολές φόρτωσης και αποθήκευσης.
- **Έμμεσοι καταχωρητές ροής.** Διαβάζουν δεδομένα με σειρά που ορίζει ένας πίνακας θέσεων.
- **Επανάληψη στο υλικό.** Μια ειδική εντολή επαναλαμβάνει μια ομάδα υπολογιστικών εντολών χωρίς το κόστος συνηθισμένου βρόχου.

4. Υλοποίηση σε Έναν Πυρήνα

Η αρχική υλοποίηση σε γλώσσα C χρειαζόταν 121.961,75 κύκλους ρολογιού ανά μονάδα 16×16. Κάθε στάδιο υλοποιήθηκε στη συνέχεια σε δύο εκδοχές:

- **Διπλή ακρίβεια (64 bit).** Κάθε καταχωρητής περιέχει έναν αριθμό.
- **Μισή ακρίβεια (16 bit).** Κάθε καταχωρητής περιέχει έως τέσσερις αριθμούς, που επεξεργάζονται ταυτόχρονα.

- **Αλλαγή χρωματικής αναπαράστασης.** Ένας έτοιμος πίνακας 256 τιμών διαβάζεται από έμμεσο καταχωρητή ροής, οπότε η μετατροπή των εικονοστοιχείων σε δεκαδικούς αριθμούς γίνεται χωρίς κόστος. Η μισή ακρίβεια (2.103 κύκλοι) ήταν ταχύτερη από τη διπλή (4.039,5).

- **Μείωση χρώματος.** Στη διπλή ακρίβεια οι τρεις καταχωρητές ροής καλύπτουν ακριβώς το στάδιο, και η διαίρεση με 4 αντικαθίσταται από πολλαπλασιασμό με 0,25. Στη μισή ακρίβεια η αποθήκευση έπρεπε να γίνει χειροκίνητα, γιατί οι καταχωρητές ροής γράφουν μόνο ολόκληρες λέξεις των 64 bit. Παρ' όλα αυτά, η μισή ακρίβεια χρειάστηκε 725 κύκλους έναντι 1.456,25.

- **Μετασχηματισμός συνημιτόνου.** Στη διπλή ακρίβεια αρκεί να αλλάξει η σειρά ανάγνωσης των καταχωρητών ροής για να διαβαστεί ο πίνακας κατά στήλες, χωρίς αναδιάταξή του στη μνήμη. Το στάδιο χρειάστηκε 9.669,25 κύκλους. Στη μισή ακρίβεια ο βρόχος ξεπέρασε το όριο των 32 εντολών της επανάληψης στο υλικό και χρειάστηκε χειροκίνητη αποθήκευση. Ο χρόνος ήταν 6.814,25 κύκλοι.

- **Κβαντοποίηση.** Η στρογγυλοποίηση γίνεται προσθέτοντας και αφαιρώντας έναν πολύ μεγάλο σταθερό αριθμό, κάτι που αναγκάζει το υλικό να απορρίψει το δεκαδικό μέρος με κόστος μιας απλής πρόσθεσης. Επειδή κάθε στοιχείο είναι ανεξάρτητο, η μισή ακρίβεια αξιοποιήθηκε πλήρως: 478 κύκλοι έναντι 2.114,5, δηλαδή 4,42 φορές ταχύτερα.

- **Σάρωση ζιγκ-ζαγκ.** Στη διπλή ακρίβεια ο έμμεσος καταχωρητής ροής διαβάζει τα στοιχεία με τη σωστή σειρά. Μια ουρά υλικού επιτρέπει στη μονάδα υπολογισμού και στο τμήμα ελέγχου να δουλεύουν ταυτόχρονα, και το στάδιο ολοκληρώνεται σε 1.035 κύκλους. Στη μισή ακρίβεια κάθε στοιχείο έπρεπε να εξαχθεί και να μετατραπεί χωριστά. Χρειάστηκαν 10.355,25 κύκλοι, περισσότεροι ακόμη και από την αρχική υλοποίηση (9.387,25).

- **Κωδικοποίηση Huffman.** Αποτελείται από πράξεις σε επίπεδο bits και πολλές αποφάσεις ροής, που δεν επιταχύνονται. Παρέμεινε περίπου στους 14.800–14.900 κύκλους σε όλες τις εκδοχές.

Συνολικά η διπλή ακρίβεια χρειάστηκε 33.129 κύκλους και η μισή 35.362,25. Η μισή ακρίβεια υπερτερούσε σε τέσσερα από τα έξι στάδια, αλλά η καθυστέρηση στη σάρωση ζιγκ-ζαγκ ακύρωσε τα κέρδη της. Επιλέχθηκε η διπλή ακρίβεια ως βάση, επειδή είναι 3,68 φορές ταχύτερη από την αρχική υλοποίηση και δομικά απλούστερη.

5. Εκτέλεση σε Οκτώ Πυρήνες

Η επέκταση στους οκτώ πυρήνες βασίστηκε σε ένα μοντέλο παραγωγού-καταναλωτή σε τρία βήματα:

- **Φόρτωση.** Ο πυρήνας μεταφοράς αντιγράφει ένα τμήμα εικόνας 16×16 στον ιδιωτικό χώρο κάθε πυρήνα.
- **Εκτέλεση.** Οι οκτώ πυρήνες ξεκινούν ταυτόχρονα από ένα κοινό σημείο αναμονής και εκτελούν ολόκληρο τον αλγόριθμο.
- **Επιστροφή αποτελεσμάτων.** Ο πυρήνας μεταφοράς αντιγράφει τα συμπιεσμένα δεδομένα πίσω στην κύρια μνήμη.

Έτσι οι υπολογιστικοί πυρήνες δεν περιμένουν ποτέ την αργή κύρια μνήμη. Η τοπική μνήμη οργανώθηκε σε τρία μέρη:

- **Κοινός κατάλογος.** Περιέχει τις διευθύνσεις και τα μεγέθη των δεδομένων.
- **Ιδιωτικοί χώροι εργασίας.** Κάθε πυρήνας έχει έναν δικό του χώρο 14.080 bytes.
- **Επαναχρησιμοποίηση χώρου.** Τα διαδοχικά στάδια μοιράζονται δύο χώρους αποθήκευσης, ώστε όλα τα ενδιάμεσα δεδομένα να χωρούν στην τοπική μνήμη.

Επειδή οι πυρήνες εκτελούν τις ίδιες εντολές σχεδόν ταυτόχρονα, πανομοιότυπα τοποθετημένα δεδομένα θα οδηγούσαν όλα τα αιτήματα στην ίδια τράπεζα. Αντί να προστεθεί κενός χώρος, που θα σπαταλούσε μνήμη, εφαρμόστηκε μετατόπιση σε δύο επίπεδα:

- **Μεταξύ πυρήνων.** Ο χώρος κάθε πυρήνα μετατοπίζεται κατά τέσσερις τράπεζες σε σχέση με τον προηγούμενο, ώστε οι οκτώ πυρήνες να καλύπτουν ομοιόμορφα και τις 32 τράπεζες.
- **Εντός κάθε πυρήνα.** Οι πίνακες σταθερών και οι χώροι εργασίας τοποθετούνται σε διαφορετικές τράπεζες, ώστε οι τρεις καταχωρητές ροής να μη συγκρούονται.

Παρέμεινε ένα όριο της αρχιτεκτονικής. Στην ανάγνωση κατά στήλες του μετασχηματισμού συνημιτόνου, κάθε στοιχείο απέχει οκτώ θέσεις από το προηγούμενο. Αυτό περιορίζει την

ανάγνωση σε μόνο τέσσερις τράπεζες και προκαλεί αναπόφευκτες συγκρούσεις.

6. Αποτελέσματα

Οι μετρήσεις έγιναν σε πυρήνα εργασίας, ώστε να μην αλλοιώνονται από τις εργασίες αρχικοποίησης του πρώτου πυρήνα. Οι συγκρούσεις τραπεζών ήταν λίγες, περίπου 78 κύκλοι. Το κύριο πρόβλημα βρισκόταν στην κωδικοποίηση Huffman: ο πυρήνας ζητούσε ένα δεδομένο και το χρειαζόταν αμέσως στην επόμενη εντολή, με αποτέλεσμα 2.529 κύκλους αναμονής. Με αναδιάταξη των εντολών, ώστε τα δεδομένα να ζητούνται νωρίτερα, η αναμονή μειώθηκε στους 1.033 κύκλους, δηλαδή περίπου κατά 59%. Η κωδικοποίηση Huffman έπεσε από 14.814,5 σε 14.229 κύκλους.

Τελικά κάθε πυρήνας επεξεργάζεται μία μονάδα 16×16 σε 32.507,75 κύκλους, παράλληλα με τους υπόλοιπους. Η συστοιχία ολοκληρώνει έτσι οκτώ μονάδες σε 32.508 κύκλους, έναντι 265.032 για έναν πυρήνα, δηλαδή επιτάχυνση 8,15 φορές. Η επιτάχυνση ξεπερνά ελαφρώς τον αριθμό των πυρήνων, επειδή η βελτίωση του Huffman εφαρμόστηκε μετά τη μέτρηση του ενός πυρήνα.

7. Συμπεράσματα και Μελλοντική Εργασία

Τα βασικά συμπεράσματα της εργασίας είναι τρία:

- **Σταθερή ροή αντί για μεμονωμένη ταχύτητα.** Η αδιάκοπη ροή εργασίας μετράει περισσότερο από την ταχύτητα ενός σταδίου, όπως έδειξε η αποτυχία της μισής ακρίβειας.
- **Ρητός έλεγχος της μεταφοράς δεδομένων.** Η αξιοποίηση των ειδικών μηχανισμών μεταφοράς αποφέρει τα μεγαλύτερα κέρδη.
- **Διαχωρισμός μεταφοράς και υπολογισμού.** Με ξεχωριστό πυρήνα μεταφοράς και προσεκτική τοποθέτηση των δεδομένων, οι υπολογιστικοί πυρήνες λειτουργούν σχεδόν στα όρια του υλικού.

Ως μελλοντικές κατευθύνσεις προτείνονται:

- **Επιταχυντής για την κωδικοποίηση Huffman.** Ένα ειδικό κύκλωμα για το πιο αργό πλέον στάδιο.
- **Μετρήσεις ενέργειας.** Μέτρηση της πραγματικής κατανάλωσης σε κατασκευασμένο κύκλωμα.
- **Νεότερα πρότυπα συμπίεσης.** Εφαρμογή των τεχνικών σε σύγχρονα πρότυπα βίντεο, όπως τα HEVC και AV1.

Table of Contents

1 Introduction 1
1.1 Motivation: The Demands of Low-Power Computing 1
1.2 Problem Statement 2
1.3 Thesis Objectives and Contributions 2
1.4 Thesis Organization 3
2 Compression Algorithm JPEG 5
2.1 Introduction to Image Compression 5
2.2 Color Space Transformation 8
2.2.1 The RGB Color Space: Nature and Inherent Compression Limitations 8
2.2.2 Psychophysiology of the Human Visual System (HVS) 10
2.2.3 The YCbCr Color Space and the ITU-R BT.601 Standard 11
2.2.4 Inter-channel Decorrelation and Energy Compaction 12
2.2.5 Chroma Subsampling: Direct Utilization of YCbCr 13
2.3 2D Discrete Cosine Transform (2D-DCT) 15
2.3.1 Image Block Partitioning and DC Level Shift 15
2.3.2 Theoretical Foundation: Selection of the DCT over Alternative Transforms 16
2.3.3 Mathematical Definition of the 2D-DCT and the Separability Property 18
2.3.4 The 64 Basis Functions and Their Physical Interpretation 20
2.3.5 The Energy Compaction Phenomenon 21
2.3.6 Computational Considerations, Fast DCT Algorithms, and Blocking Artifacts 23
2.4 Quantization 24
2.4.1 The Psychovisual Foundation of Quantization 24
2.4.2 Luminance (Luma) and Chrominance (Chroma) Quantization Tables 25
2.4.3 Quality Factor (Qf) and Scaling 27
2.4.4 Sparsity Generation and Entropy Coding Preparation 28
2.5 Entropy Encoding 29
2.5.1 Theoretical Background: Statistical Entropy and Lossless Compression 29
2.5.2 Zig-Zag Scanning and Data Serialization 30
2.5.3 Differential Pulse-Code Modulation (DPCM) for the DC Coefficient 31
2.5.4 Run-Length Encoding (RLE) for AC Coefficients 32
2.5.5 Huffman Categories 33
2.5.6 Huffman Coding and Final Bitstream Formation 34

2.5.7 Bit-Writer Buffer and Byte Stuffing 34
2.6 Summary of the Compression Pipeline 35
2.6.1 The Three Phases of the Pipeline 35
2.6.2 Architectural Mapping on RISC-V Snitch 37
3 The RISC-V Architecture and the Snitch System 38
3.1 The RISC-V Architecture 38
3.1.1 The RISC-V Instruction Set Architecture (ISA) 38
3.1.2 Register File and Standard Extensions 39
3.1.3 Custom Extensions and Domain-Specific Acceleration 40
3.1.4 Energy and Thermal Constraints in Edge Computing 41
3.1.5 The Architectural Advantage of Multi-core Clusters 41
3.1.6 Exploiting Data Parallelism in Image Algorithms 42
3.2 The Snitch Cluster Architecture 43
3.2.1 Compute Cores and the Floating-Point Unit (FPU) 45
3.2.2 Data Management Core (DM Core) and Direct Memory Access (DMA) 47
3.2.3 Local Memory Hierarchy 51
3.3 Hardware Accelerators in Snitch 56
3.3.1 Stream Semantic Registers (SSR) 56
3.3.2 Floating-Point Repetition (FREP) 59
4 Single-Core Implementation 62
4.1 Initial Design 62
4.1.1 Color Space Conversion: Look-Up-Table Type Conversion and the Luminance Dependency 62
4.1.2 Chroma Subsampling: 2×2 Averaging through Multiplication Instead of Division 63
4.1.3 2D-DCT: Level Shift and Separable Row–Column Multiply-Accumulate Passes 63
4.1.4 Quantization: Reciprocal Multiplication, Rounding and Integer Conversion 64
4.1.5 Zig-Zag Reordering and DC Differential Encoding through a Precomputed Index Map 64
4.1.6 Entropy Coding: A Serial, Integer-Only Stage Kept in Scalar C 65
4.2 Color Space Conversion (YCbCr) and Data Precision Exploration (Double vs. Packed FP16) 65
4.2.1 Common Architectural Optimizations (ISSR & 4D Tiling) 67
4.2.2 Double Precision Implementation (FP64) 68
4.2.3 Vectorized Implementation (Packed SIMD FP16) 69

4.2.4 Comparative Analysis: Packing Overhead and Register Complexity 70
4.2.5 Cycle Count Analysis and Performance Evaluation 71
4.3 Chroma Subsampling (4:2:0) 71
4.3.1 Hardware Constraints: The 3 Data Mover Limit 72
4.3.2 Double Precision Implementation (FP64) 72
4.3.3 Vectorized Implementation (Packed SIMD FP16) 74
4.3.4 Comparative Analysis: SSR Write Constraints and ALU Involvement 75
4.3.5 Cycle Count Analysis and Performance Evaluation 76
4.4 Two-Dimensional Discrete Cosine Transform (2D-DCT) 77
4.4.1 Double Precision Implementation (FP64) 77
4.4.2 Vectorized Implementation (Packed SIMD FP16) 78
4.4.3 Comparative Analysis: Sub-Indexing and Instruction Buffer Overflows 80
4.4.4 Cycle Count Analysis and Performance Evaluation 81
4.5 Quantization and Zero-Latency Rounding 81
4.5.1 Micro-Architectural Optimization: "Magic Number" Fast Rounding 82
4.5.2 Double Precision Implementation (FP64) 82
4.5.3 Vectorized Implementation (Packed SIMD FP16) 83
4.5.4 Comparative Analysis: SIMD Success in Element-Wise Operations 84
4.5.5 Cycle Count Analysis and Performance Evaluation 84
4.6 Zig-Zag Reordering and Differential Encoding 85
4.6.1 Double Precision Implementation (FP64) 85
4.6.2 Scalar Fallback Implementation (FP16) 86
4.6.3 Comparative Analysis: The Cost of Synchronization 87
4.6.4 Cycle Count Analysis and Performance Evaluation 88
4.7 Performance Evaluation and Baseline Selection 89
4.7.1 The Theoretical SIMD Advantage (Compute-Bound Stages) 90
4.7.2 The Architectural Bottleneck: Data Serialization 91
4.7.3 The Serial Limit: Entropy Coding 91
4.7.4 Conclusion and Baseline Selection 91
4.8 Revisiting the Unpack: A Hybrid FP16/FP64 Pipeline 92
4.8.1 Root Cause: NaN-Boxing of Packed Registers 92
4.8.2 Evaluating Unpacking Alternatives 92
4.8.3 Hybrid Pipeline Design 93
4.8.4 Results and Verification 94
4.8.5 Implications for the Baseline Selection 95
5 Multi-Core Architecture & Memory Optimization 97
5.1 Multi-Core Parallelization (Tile-by-Tile, DM Core/DMA) 97

5.1.1 The Role of the Data Mover (DM) Core 97
5.1.2 Tile-by-Tile Synchronization and Execution 98
5.1.3 Execution Flow Diagram 98
5.2 TCDM Memory Layout and Workspace Partitioning 100
5.2.1 The TCDM Root Control Directory 100
5.2.2 Strict Core Isolation and Partitioning 100
5.2.3 Intra-Core Memory Aliasing and Buffer Reusability 101
5.3 Resolving TCDM Bank Contention 103
5.3.1 Lockstep Execution and the "Bank 13" Collision Scenario 104
5.3.2 Prime-Stride Alignment vs. Algorithmic Staggering 104
5.3.3 Level 1: Inter-Core Bank Staggering 105
5.3.4 Level 2: Intra-Core Workspace Alignment 106
5.3.5 The "Jump of 8" (Power-of-Two Stride) Matrix Collision 107
5.4 Performance Analysis: Theoretical vs. Empirical Optimizations 110
5.4.1 Empirical Micro-Benchmarking and Stall Analysis 110
5.4.2 Cluster-Level Speedup and Combined Scaling Efficiency 113
6 Conclusion 114
6.1 Summary of the Research 114
6.2 Key Architectural Insights and Contributions 114
6.3 Future Work 115
References 117

Table of Figures

Figure 2.1: Block diagram of the JPEG encoder 5
Figure 2.2: Forms of Image Data Redundancy through JPEG Compression 6
Figure 2.3: Average human optical Sensitivity to each Frequency (Mannos–Sakrison model, Equation (2.2)) 8
Figure 2.4: 3D visualization of characteristic colors in RGB format, with the pixels of the test image 10
Figure 2.5: Human sensitivity differences to each Color (CIE 1924 photopic luminous efficiency function) 11
Figure 2.6: Decomposition of an RGB image into the Y, Cb and Cr channels 13
Figure 2.7: The different levels of Chroma Subsampling in JPEG 14
Figure 2.8: Extraction of an 8×8 pixel Block from an image 16
Figure 2.9: DCT advantage to DFT 18
Figure 2.10: Visualization of the different 64 possible frequencies of an 8×8 Block 20
Figure 2.11: Energy compaction of the 2D-DCT: the DC coefficient captures the majority of the energy 22
Figure 2.12: Blocking Artifacts, the major JPEG visual problem with high quantization 24
Figure 2.13: Lossless and Lossy parts of JPEG pipeline 25
Figure 2.14: Visualization of Luminance Quantization Table 26
Figure 2.15: Visualization of Chroma Quantization Table 26
Figure 2.16: Trade-off between Quality of an image and Compression Ratio, measured on the test image 28
Figure 2.17: Energy compaction and sparsity generation through quantization (Qf = 50) 29
Figure 2.18: Visualization of Zig-Zag Scanning 31
Figure 2.19: Complete Theoretical JPEG pipeline 36
Figure 3.1: RISCV Instruction Categorization 39
Figure 3.2: RISCV Instruction Formats 40
Figure 3.3: From 1 Core to Multiple Core Systems 42
Figure 3.4: Snitch Cluster Architecture Visualization 44
Figure 3.5: Snitch Cluster Compute Core Architecture 46
Figure 3.6: Snitch Cluster visualization of Data Block Movement 48

Figure 3.7: Visualization of the movement of Data through 2 Ping-Pong Buffers 50
Figure 3.8: Snitch Cluster Memory management Architecture 52
Figure 3.9: Address pointing through TCDM.....54
Figure 3.10: Data Movement through FPU SSR without delay. 58
Figure 3.11: Visualization of computation gap between Standard Riscv and Optimized Computation.....61
Figure 4.1: Performance comparison per execution stage between the different implementations.....90
Figure 5.1: Graph showing the complete multicore movement of data through JPEG execution.....99
Figure 5.2: Detailed Memory of TCDM belonging to each specific Core 103
Figure 5.3: Inter-core bank staggering during lockstep execution: Core i accesses Bank 4i, so no two cores collide.....106
Figure 5.4: Theoretical model of the SSR bank accesses of one core during Pass 2 of the 2D-DCT 108
Figure 5.5: Theoretical model of the SSR bank accesses and collisions of all 8 cores during Pass 2 of the 2D-DCT.....109
Figure 5.6: Stall cycles per category across the three optimization steps.....111
Figure 5.7: Huffman Memory Optimization.....112

List of Tables

Table 2.1: Comparative Evaluation of Lossless and Lossy compression methods.....7
Table 2.2: Comparison of RGB and YCbCr color spaces.....13
Table 2.3: Comparison between chroma subsampling schemes.....14
Table 2.4: Comparative evaluation of transforms for image compression.....18
Table 2.5: Huffman Size Categories and Corresponding integer value range.....33
Table 3.1: Key standard extensions of the RISC-V ecosystem.....39
Table 3.2: Comparative analysis of a single-core versus a multi-core architecture....42
Table 3.3: RISC-V extensions used in the two types of Snitch Cluster cores 48
Table 3.4: Memory bank contention versus 64-bit word access stride S_{stride}.....55
Table 3.5: Performance comparison and overhead between a conventional RISC-V inner loop and the accelerated SSR + FREP Snitch streaming pipeline.....60
Table 4.1: Overview of the optimisation techniques applied per pipeline stage (FP64 vs. Packed SIMD FP16).....66
Table 4.2: Cycle count and speedup comparison of the YCbCr conversion stage 71
Table 4.3: Performance evaluation between the different implementations.....89
Table 4.4: Micro-benchmark results of the FP16 unpacking alternatives 93
Table 4.5: Performance of the hybrid FP16/FP64 pipeline compared with the FP64 and FP16 implementations.....95
Table 5.1: Total Compute comparison per one MCU per Core for a single Core Baseline vs an 8 core system using the memory optimizations.....113

List of Abbreviations

AC	Alternating Current (non-zero-frequency DCT coefficient)
AGU	Address Generation Unit
ALU	Arithmetic Logic Unit
ASIC	Application-Specific Integrated Circuit
AV1	AOMedia Video 1
AXI	Advanced eXtensible Interface
CCD	Charge-Coupled Device
CIE	Commission Internationale de l'Éclairage (International Commission on Illumination)
CMOS	Complementary Metal-Oxide-Semiconductor
CPU	Central Processing Unit
CR	Compression Ratio
CSF	Contrast Sensitivity Function
CSR	Control and Status Register
DC	Direct Current (zero-frequency DCT coefficient)
DCT	Discrete Cosine Transform
DFT	Discrete Fourier Transform
DHT	Define Huffman Table (JPEG marker)
DM	Data Mover
DMA	Direct Memory Access
DPCM	Differential Pulse-Code Modulation
DQT	Define Quantization Table (JPEG marker)
DRAM	Dynamic Random-Access Memory
DSA	Domain-Specific Architecture
DSP	Digital Signal Processing
DWT	Discrete Wavelet Transform
EOB	End of Block
EOI	End of Image (JPEG marker)
ETH	Eidgenössische Technische Hochschule (Swiss Federal Institute of Technology)
F2I	Float-to-Integer
FFT	Fast Fourier Transform
FIFO	First In, First Out
FLOP	Floating-Point Operation
FMA	Fused Multiply-Add
FP16	16-bit (half-precision) Floating Point
FP64	64-bit (double-precision) Floating Point
FP8	8-bit (quarter-precision) Floating Point
FPGA	Field-Programmable Gate Array

FPU	Floating-Point Unit
FREP	Floating-Point Repetition
GCD	Greatest Common Divisor
GFLOPS	Giga Floating-Point Operations per Second
GPU	Graphics Processing Unit
HEIF	High Efficiency Image File Format
HEVC	High Efficiency Video Coding
HVS	Human Visual System
IDCT	Inverse Discrete Cosine Transform
IEC	International Electrotechnical Commission
IEEE	Institute of Electrical and Electronics Engineers
IJG	Independent JPEG Group
IoT	Internet of Things
IPC	Instructions Per Cycle
ISA	Instruction Set Architecture
ISO	International Organization for Standardization
ISSR	Indirect Stream Semantic Register
ITU	International Telecommunication Union
JPEG	Joint Photographic Experts Group
JSON	JavaScript Object Notation
KLT	Karhunen–Loève Transform
LMS	Long, Medium, Short (wavelength cone types)
LRU	Least Recently Used
LSU	Load/Store Unit
LUT	Look-Up Table
MAC	Multiply-Accumulate
MCU	Minimum Coded Unit
MSE	Mean Squared Error
NaN	Not a Number
OPT	Optimized (vectorized Packed SIMD FP16 implementation)
PNG	Portable Network Graphics
PSNR	Peak Signal-to-Noise Ratio
PULP	Parallel Ultra-Low-Power
RAM	Random-Access Memory
RAW	Read-After-Write
RGB	Red, Green, Blue
RISC	Reduced Instruction Set Computer
RLE	Run-Length Encoding
SIMD	Single Instruction, Multiple Data
SOI	Start of Image (JPEG marker)
SOS	Start of Scan (JPEG marker)

SPM	Scratchpad Memory
SPMD	Single Program, Multiple Data
SRAM	Static Random-Access Memory
SSR	Stream Semantic Register
TCDM	Tightly Coupled Data Memory
TDP	Thermal Design Power
TIFF	Tagged Image File Format
VLC	Variable-Length Coding
YCbCr	Luminance (Y), Blue-difference (Cb) and Red-difference (Cr) Chrominance
ZRL	Zero Run Length

1 Introduction

1.1 Motivation: The Demands of Low-Power Computing

Internet of Things (IoT) devices, autonomous drones and embedded sensor networks have changed where visual data is produced and processed. Images are increasingly captured at the *edge of the network*, on small battery-powered devices that must process or transmit them under tight power and bandwidth budgets. Image compression is therefore a **core workload in low-power systems**. Its most common form is the JPEG standard (Joint Photographic Experts Group), the most widely used format for photographic images.

Running computationally intensive algorithms on such devices is an architectural challenge. General-purpose processors devote much of their silicon area and energy to control mechanisms rather than arithmetic. Examples are *deep cache hierarchies*, *dynamic branch prediction* and *out-of-order execution*. These features speed up arbitrary software. In an energy-constrained device, however, they mean that a large share of the power is spent managing and moving data rather than computing on it.

To reach the energy efficiency and compute density that low-power systems require, research has turned toward specialized, *heterogeneous multi-core architectures*. The Snitch cluster is a representative example. It is an open-source design built on the RISC-V instruction set architecture (ISA) and developed by ETH Zurich and the University of Bologna [1]. Snitch **removes hardware data caches entirely** and replaces them with three mechanisms:

- **Tightly Coupled Data Memory (TCDM).** A small *scratchpad memory* with single-cycle access, shared by all cores and managed explicitly by software.
- **Stream Semantic Registers (SSRs).** Floating-point registers mapped to memory streams. Reading or writing them loads or stores data automatically, without explicit load and store instructions.
- **Data Mover (DM) core.** A dedicated core that controls the cluster's Direct Memory Access (DMA) engine and transfers data between main memory and the TCDM independently of the compute cores.

This design achieves high energy efficiency, but at a cost. The responsibility for placing data in memory, synchronizing the cores and avoiding pipeline stalls **shifts almost entirely from the hardware to the software developer**.

1.2 Problem Statement

Porting a conventional algorithm to a cache-less, heterogeneous architecture such as Snitch is **far from trivial**. Standard JPEG implementations are written for processors whose caches hide memory latency and whose hardware handles data alignment transparently. On Snitch, none of this happens automatically, and three difficulties arise:

- **Explicit data movement.** Data transfers must be programmed explicitly to keep the Floating-Point Units (FPUs) supplied with operands. Otherwise, the compute cores stall while waiting for memory.
- **Memory bank contention.** When the pipeline is distributed across compute cores that execute the same instructions in *lockstep*, their simultaneous memory requests tend to target the same TCDM bank. The interconnect must then serialize these accesses.
- **Uneven fit to the accelerators.** Not every JPEG stage maps well to the available hardware. Some stages are regular floating-point arithmetic, while others, such as entropy coding, are serial and operate on individual bits.

As a result, the *performance bottleneck* shifts from arithmetic throughput to memory organization, inter-core synchronization and the constraints of the TCDM interconnect. High hardware utilization therefore requires restructuring the JPEG algorithm through **hardware-software co-design**: shaping the software around the specific capabilities and limits of the hardware, at both the instruction level and the cluster level.

1.3 Thesis Objectives and Contributions

The objective of this thesis is to **optimize and parallelize the JPEG compression pipeline** for the RISC-V Snitch cluster, and to show how software can be tailored to exploit specialized hardware accelerators. The main contributions are:

- **Precision analysis.** A cycle-accurate comparison of each JPEG stage in double precision (FP64, 64-bit floating point) and in *packed half precision* (FP16, 16-bit floating point processed as SIMD vectors, Single Instruction Multiple Data). FP16 is

faster in four of the six stages. However, converting packed vectors back into serial integers in the Zig-Zag reordering stage costs more than these gains. FP64 is therefore selected as the single-core baseline. It completes one Minimum Coded Unit (MCU, a 16×16 pixel block) in **33,129 cycles**, compared with 121,962 cycles for the initial C implementation.

- **Zero-cost memory routing.** Four-dimensional SSR address patterns transpose matrices in the 2D Discrete Cosine Transform (2D-DCT) without moving data. Indirect SSRs (ISSRs) perform type conversion and Zig-Zag reordering through *table look-ups*. Hardware queues between the FPU and the integer Arithmetic Logic Unit (ALU) allow the two units to operate concurrently.
- **Decoupled multi-core orchestration.** A tile-by-tile *producer-consumer* execution model in which the DM core performs all DMA transfers, isolating the eight compute cores from main-memory latency.
- **Memory layout for contention avoidance.** A *Dual-Level Bank Staggering* scheme offsets each core's workspace, and the buffers within each core, across different TCDM banks. This avoids lockstep bank collisions **without spending memory on padding**. The thesis also identifies a remaining architectural limit: the power-of-two stride of the column-wise DCT pass.
- **Stall analysis and cluster scaling.** Profiling showed that the dominant bottleneck was load-use latency in the scalar Huffman encoder, not bank contention. *Software pipelining* of the encoder reduced these stalls by approximately 59%. Combined with eight-core parallel execution, the cluster processes 8 MCUs in 32,508 cycles, an **8.15× speedup** over the single-core baseline. The portion above 8× comes from the Huffman optimization, which is not included in the baseline measurement.

1.4 Thesis Organization

The thesis moves from the theoretical foundations of image compression to the details of the final multi-core implementation:

- **Chapter 2 – The JPEG Compression Algorithm.** Presents the theoretical and mathematical basis of each stage of the JPEG pipeline, from color space transformation to entropy coding.
- **Chapter 3 – The RISC-V Architecture and the Snitch System.** Introduces the RISC-V instruction set and the components of the Snitch cluster, including the

TCDM, the SSRs, the hardware loop instruction FREP (Floating-Point Repetition) and the DM core.

- **Chapter 4 – Single-Core Implementation.** Develops the optimized single-core pipeline, compares the FP64 and FP16 implementations stage by stage, and selects the baseline for parallelization.
- **Chapter 5 – Multi-Core Architecture and Memory Optimization.** Describes the transition to the eight-core cluster, the DMA synchronization framework, the TCDM memory layout and bank-staggering scheme, and the final performance analysis.
- **Chapter 6 – Conclusion.** Summarizes the findings and outlines directions for future work.

2 Compression Algorithm JPEG

This chapter presents the theoretical and mathematical foundations of the JPEG compression algorithm. Figure 2.1 shows the JPEG encoder as a chain of processing blocks and the section of this chapter that describes each one: color space conversion and chroma subsampling (Section 2.2), division into 8×8 blocks and the 2D-DCT (Section 2.3), quantization (Section 2.4), and entropy coding (Section 2.5). Section 2.6 summarizes the complete pipeline in more detail.

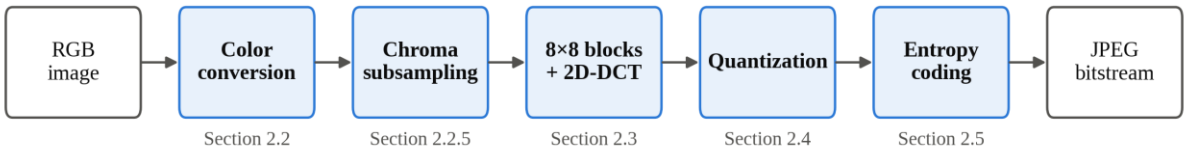


Figure 2.1: Block diagram of the JPEG encoder

2.1 Introduction to Image Compression

In modern digital imaging and signal processing, raw, uncompressed image data require a massive amount of storage space and high bandwidth for transmission and handling. A typical RGB (Red, Green, Blue) digital photograph consists of a two-dimensional grid of pixels with dimensions $M_{\text{img}} \times N_{\text{img}}$, where each pixel is represented by three 8-bit color channels: Red, Green, and Blue. Consequently, the total size S_{img} of an uncompressed image in bits is expressed by Equation (2.1):

$$S_{\text{img}} = M_{\text{img}} \times N_{\text{img}} \times B_{\text{px}} \tag{2.1}$$

where M_{img} and N_{img} are the height and width of the image in pixels, respectively, and B_{px} is the total color depth in bits per pixel (24 bits or 3 bytes for standard RGB images). For example, a 4K resolution image (3840 × 2160 pixels) requires approximately 24.88 MB of uncompressed storage space, while real-time video transmission at 30 frames per second requires a bitrate exceeding 5.9 Gbps, **making the use of efficient compression algorithms imperative**.

Effective image compression is made possible because natural images are not random sequences of data; rather, they contain a significant degree of correlation and structural repeatability. In information theory, image data redundancy is classified into three primary

forms: *coding redundancy*, which arises from using fixed-length codewords for values with differing probabilities of occurrence; *spatial redundancy*, which stems from the strong local correlation among neighboring pixels; and *psychovisual redundancy*, which involves details present in the data that are imperceptible to the human eye [2].

Figure 2.2 maps each of the three forms of redundancy to the stage of the JPEG (Joint Photographic Experts Group) algorithm that removes it: **coding redundancy** is removed by entropy coding (Huffman coding), **spatial redundancy** by the transformation to the frequency domain, and **psychovisual redundancy** by quantization, which rejects high-frequency detail. Removing all three is what allows JPEG to reach compression ratios of 10:1 to 50:1.

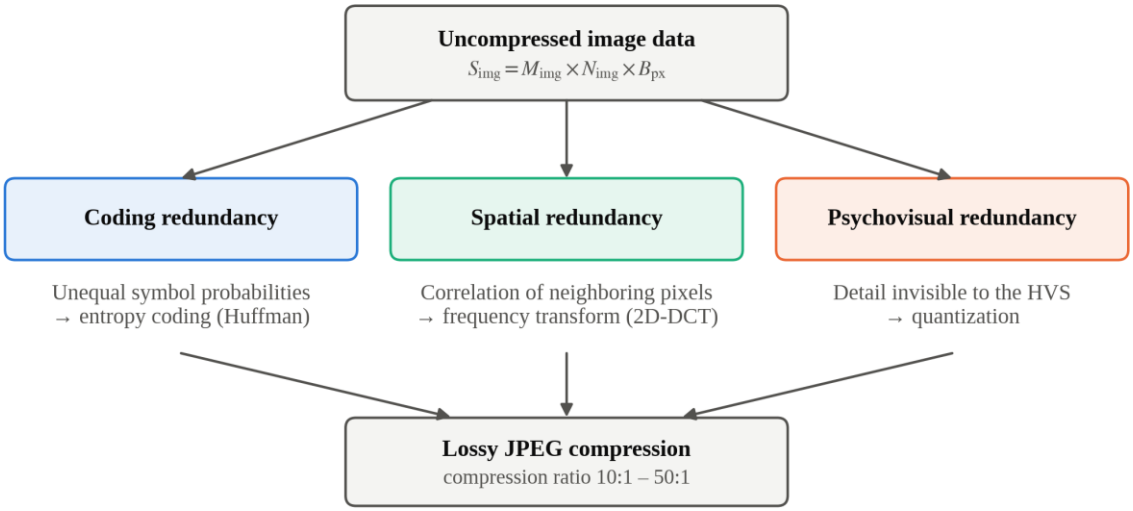


Figure 2.2: Forms of Image Data Redundancy through JPEG Compression

Image compression techniques are divided into two main categories: **lossless and lossy compression**. Lossless compression (e.g., PNG – Portable Network Graphics, lossless WebP) exploits only statistical and spatial redundancy in pixel data to reduce file size without altering a single bit of the original image, with the achievable compression ratio typically limited to 2:1 up to 3:1 [2]. In contrast, lossy compression (e.g., JPEG) achieves much higher compression ratios—often exceeding 10:1 to 50:1 with negligible visual degradation—by additionally exploiting the physiological limitations of the Human Visual System (HVS).

Table 2.1 compares the two categories in terms of reconstruction fidelity, achievable compression ratio, the forms of redundancy they eliminate, and typical file formats (TIFF – Tagged Image File Format; HEIF – High Efficiency Image File Format).

Table 2.1: Comparative Evaluation of Lossless and Lossy compression methods

Feature	Lossless Compression	Lossy Compression
Reconstruction Fidelity	100% (Exact replica)	Approximate reconstruction
Compression Ratio (CR)	2:1 – 3:1	10:1 – 50:1+
Redundancy Elimination	Coding, Spatial	Coding, Spatial, Psychovisual
Typical Standards	PNG, WebP (lossless), TIFF	JPEG, WebP (lossy), HEIF

The design of the JPEG standard is rooted in the physiology of human vision. The Human Visual System exhibits much higher sensitivity to changes in luminance (brightness) than to chrominance (color) information, allowing for *chroma subsampling* (e.g., 4:2:0, Section 2.2.5), which reduces color resolution to one-fourth without perceptible distortion. Simultaneously, the **Contrast Sensitivity Function (CSF)** demonstrates that the eye cannot discern errors in areas of high spatial frequency, such as sharp edges and fine textures [3].

Figure 2.3 plots the CSF, i.e., the relative sensitivity of the eye to contrast as a function of the spatial frequency ν, measured in *cycles per degree* (the number of light–dark stripe pairs that fit into one degree of the visual field). The curve is plotted from the analytical model of Mannos and Sakrison [3], given in Equation (2.2):

$$A(\nu) = 2.6(0.0192 + 0.114\nu)e^{-(0.114\nu)^{1.1}} \tag{2.2}$$

The linear factor $(0.0192 + 0.114\nu)$ models the rising sensitivity at low frequencies, while the exponential factor models the rapid loss of sensitivity at high frequencies, caused by the optics of the eye and the finite density of its photoreceptors. Setting $dA/d\nu = 0$ places the maximum at $\nu \approx 7.9$ cycles/degree, where $A \approx 0.98$. At 15 cycles/degree the sensitivity has already fallen to 0.74, at 30 cycles/degree to 0.19, and decays toward zero (0.017 at 50 cycles/degree). JPEG exploits this region by **quantizing high-frequency coefficients more coarsely** (Section 2.4).

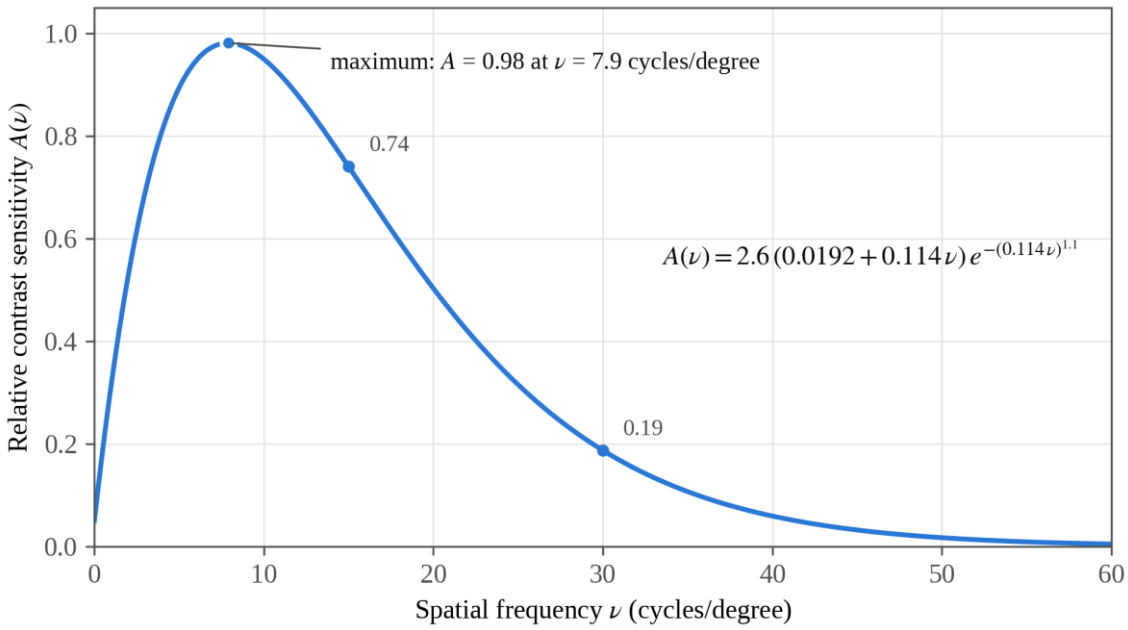


Figure 2.3: Average human optical Sensitivity to each Frequency (Mannos–Sakrison model, Equation (2.2))

The Joint Photographic Experts Group (JPEG) standard, officially standardized as ITU-T T.81 | ISO/IEC 10918-1 [4] (ITU-T: International Telecommunication Union – Telecommunication Standardization Sector; ISO/IEC: International Organization for Standardization / International Electrotechnical Commission), is the **most widely accepted lossy compression algorithm** for continuous-tone still images. Rather than treating an image as an arbitrary sequence of bytes in the spatial domain, the JPEG processing pipeline systematically transforms spatial pixel data into the *frequency domain*. Through this transformation, it selectively discards high-frequency information that the human eye cannot perceive, while preserving the essential structural characteristics and luminance of the image intact.

2.2 Color Space Transformation

2.2.1 The RGB Color Space: Nature and Inherent Compression Limitations

In modern digital imaging, the capture, storage, and display of visual information are primarily performed using the additive RGB (Red, Green, Blue) color model. In a typical 24-bit uncompressed image, each pixel is represented by the vector $[R, G, B]$ with discrete integer values ranging from 0 to 255 per channel. Although this model is optimized for hardware implementation (CMOS – Complementary Metal-Oxide-Semiconductor – and CCD – Charge-Coupled Device – image sensors, and displays), it is **fundamentally inefficient for data compression algorithms**.

The primary limiting factor is the exceptionally high **inter-channel correlation** among the three color channels. In natural images, illumination changes affect the Red, Green, and Blue values simultaneously and proportionally. This interdependence is demonstrated by the *Pearson linear correlation coefficient*, across the (R, G), (G, B), and (R, B) pairs of natural images. Retaining the image in the RGB color space forces the algorithm to store the same structural and luminance information threefold, resulting in substantial representation redundancy.

Figure 2.4 represents the RGB color space as a cube in which every pixel is a point (R, G, B) with 8-bit coordinates, so black lies at (0, 0, 0) and white at (255, 255, 255). The colored dots are 4,000 randomly selected pixels of the 512×512 test image used for all measured figures of this chapter (the "astronaut" photograph, NASA, public domain, distributed with the *scikit-image* library [5]). All shades of gray lie on the main diagonal $R = G = B$ (the *achromatic axis*). A change in illumination moves a pixel approximately along this diagonal, changing all three coordinates together, which is the geometric reason for the high inter-channel correlation described above: the pixels of the test image form an elongated cloud along the diagonal, with Pearson coefficients of 0.84 for (R, G), 0.96 for (G, B), and 0.73 for (R, B). The luminance Y of Equation (2.4) is constant on planes perpendicular to the direction $(0.299, 0.587, 0.114)$; it therefore increases along the gray diagonal, but the diagonal is not itself the Y axis.

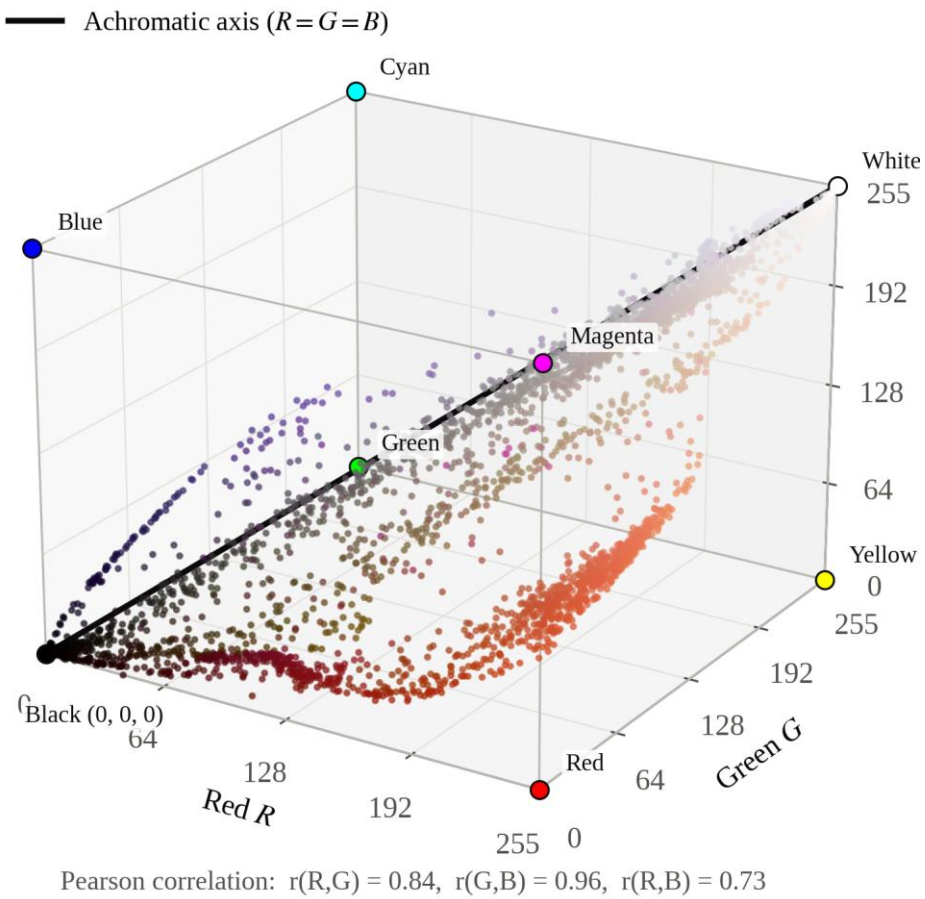


Figure 2.4: 3D visualization of characteristic colors in RGB format, with the pixels of the test image

2.2.2 Psychophysiology of the Human Visual System (HVS)

The second limitation of the RGB model is its **inability to separate luminance from chrominance**, which directly contradicts the physiology of the Human Visual System (HVS). Biological vision relies on *two distinct types of retinal photoreceptors*:

- **Rods**: ~120 million color-blind receptors responsible for *scotopic* (low-light) vision. They are extremely sensitive to luminance and contrast variations, providing high spatial acuity.
- **Cones**: ~6 million receptors (L, M, and S – long, medium, and short wavelength – corresponding to red, green, and blue wavelengths, respectively)[1], responsible for *photopic* (daylight) vision and color perception (chrominance).

Due to the overwhelming numerical dominance of rods over cones (~20:1), the human eye perceives spatial detail and edges **almost exclusively through luminance**, whereas its

[1] The three cone types are named after the part of the visible spectrum to which each is most sensitive: L (long wavelengths, peak sensitivity near 560 nm), M (medium wavelengths, near 530 nm) and S (short wavelengths, near 420 nm). The three-dimensional space formed by their responses is known as the LMS color space.

capacity to resolve fine color details is biologically limited [2]. Furthermore, the eye's luminous efficiency curve peaks in the green spectrum (~555 nm), decreases toward the red (~650 nm), and drops off sharply in the blue spectrum (~450 nm).

Figure 2.5 plots the *photopic luminous efficiency function* $V(\lambda)$, which gives the relative sensitivity of the eye to light of wavelength λ, normalized to 1 at its peak of 555 nm. The curve is the CIE 1924 *standard photopic observer* tabulated by the CIE (Commission Internationale de l'Éclairage) [6]. It is also the colour-matching function $\overline{y}(\lambda)$ of the CIE XYZ system, so the luminance of a light with spectral power distribution $\Phi(\lambda)$ follows from Equation (2.3), where $K_m = 683$ lm/W:

$$Y = K_m \int_{380\text{ nm}}^{780\text{ nm}} \Phi(\lambda) V(\lambda) d\lambda \tag{2.3}$$

The tabulated values $V(450\text{ nm}) = 0.038$ and $V(650\text{ nm}) = 0.107$ quantify the drop toward the blue and red ends of the spectrum described above. The BT.601 weights of Equation (2.4) follow from Equation (2.3): each weight is the luminance Y of one display primary (red, green, or blue) at full intensity, relative to the luminance of reference white.

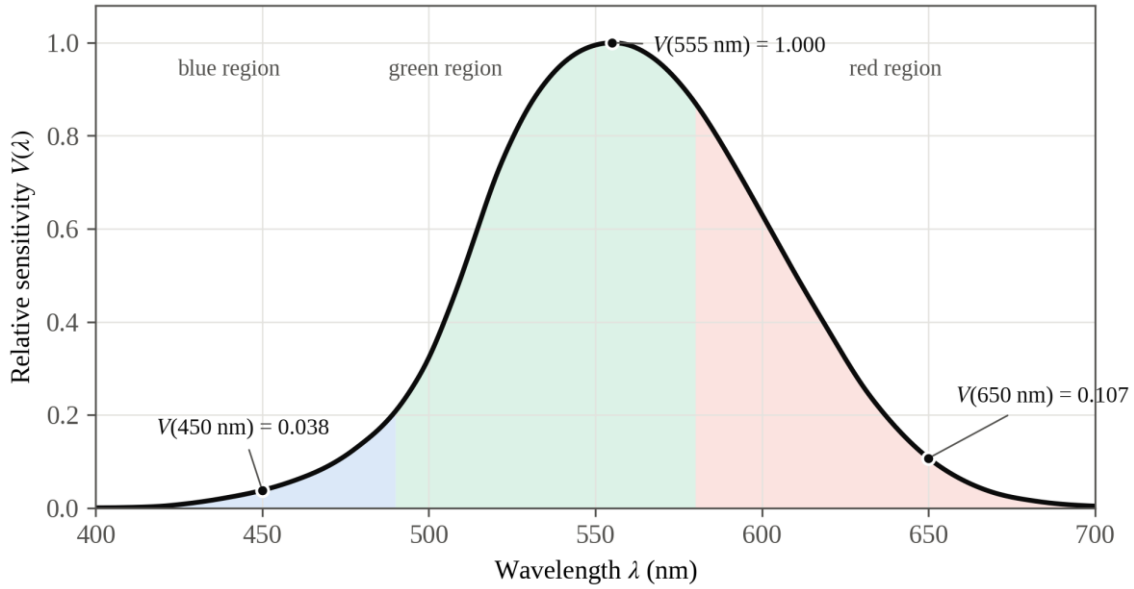


Figure 2.5: Human sensitivity differences to each Color (CIE 1924 photopic luminous efficiency function)

2.2.3 The YCbCr Color Space and the ITU-R BT.601 Standard

To eliminate inter-channel correlation and exploit biological asymmetry, the first processing stage of the JPEG (ITU-T T.81 | ISO/IEC 10918-1) standard is a linear transformation from RGB to the **YCbCr color space**, which separates the image into *one luminance component and two color-difference components*:

- **Y (Luminance)**: The effective light intensity, containing all achromatic structural information.
- **Cb (Blue Chrominance)**: The color difference between the blue channel and luminance $(B - Y)$.
- **Cr (Red Chrominance)**: The color difference between the red channel and luminance $(R - Y)$.

According to the ITU-R BT.601 recommendation [7] (ITU-R: International Telecommunication Union – Radiocommunication Sector), the conversion from 8-bit RGB to 8-bit YCbCr is defined by Equation (2.4):

$$\begin{bmatrix} Y \\ Cb \\ Cr \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.169 & -0.331 & 0.500 \\ 0.500 & -0.419 & -0.081 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 0 \\ 128 \\ 128 \end{bmatrix} \tag{2.4}$$

The weighting coefficients of matrix M (the 3×3 matrix in Equation (2.4)) reflect the psychophysical luminous efficiency curve of human vision (Green: 58.7%, Red: 29.9%, Blue: 11.4%).

The +128 constant is a normalization convention (DC offset, i.e., a constant shift of the zero level). Because the differences $(B - Y)$ and $(R - Y)$ can mathematically yield negative values (−127.5 to +127.5), this shift ensures that all data remain within the [0, 255] range of unsigned 8-bit integers (neutral gray corresponds to Cb = Cr = 128).

2.2.4 Inter-channel Decorrelation and Energy Compaction

Although transforming to YCbCr preserves the 24 bits/pixel data volume, it provides *two fundamental properties essential for subsequent compression stages*:

- **Decorrelation**: The *covariance matrix* (the matrix of pairwise covariances between the three channels) becomes nearly diagonal, eliminating statistical dependence between luminance (Y) and chrominance (Cb/Cr) and allowing independent processing of each channel.
- **Energy Compaction**: Approximately 85% to 90% of the total signal energy and variance is concentrated in the Y channel. The Cb and Cr channels contain only smooth, low-frequency variations centered around the value 128.

Figure 2.6 applies Equation (2.4) to the test image: the luminance channel Y keeps all structural detail and carries **87.5% of the total variance**, while the chrominance channels Cb

and Cr carry only 4.0% and 8.5% and contain mainly smooth, low-contrast variations around 128. These two channels are the input to chroma subsampling (Section 2.2.5).

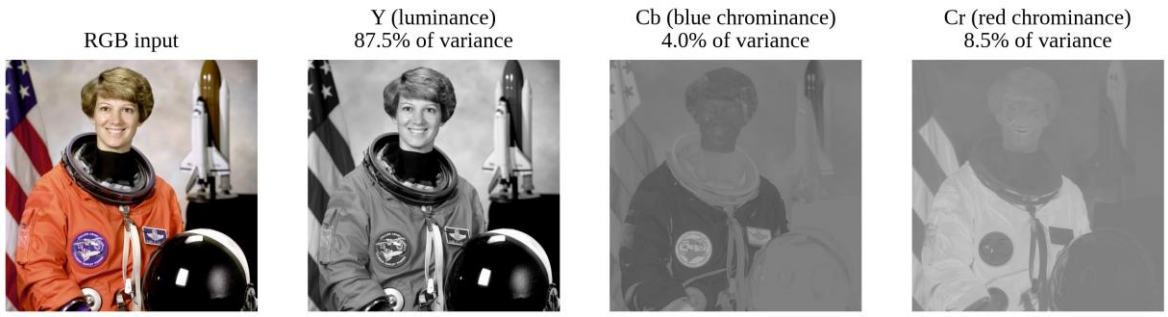


Figure 2.6: Decomposition of an RGB image into the Y, Cb and Cr channels

Table 2.2 compares the two color spaces with respect to the signal characteristics discussed above.

Table 2.2: Comparison of RGB and YCbCr color spaces

Signal Characteristic	RGB Color Space	YCbCr Color Space
Inter-channel Correlation	High (>0.85 among channels)	Negligible (Statistically independent channels)
Information Separation	Luminance & chrominance merged	Complete separation (Y achromatic, Cb/Cr chromatic)
Energy Distribution	Equal distribution among R, G, B	~90% energy in Y, ~10% in Cb/Cr
HVS Compatibility	Low	4:1 sensitivity to Y versus Cb/Cr
Compression Capability	Unsuitable for chroma rejection	Ideal for chroma subsampling

2.2.5 Chroma Subsampling: Direct Utilization of YCbCr

The greatest practical benefit of the YCbCr color space is that it enables the immediate application of **Chroma Subsampling**. Because the human eye cannot resolve fine spatial color details and the variance in the Cb/Cr channels is low, their spatial resolution can be drastically reduced without perceptible visual degradation.

Chroma subsampling is expressed using the three-part notation *J:a:b*, where J is the width in pixels of a reference region two pixels high (conventionally 4), a is the number of chroma samples in its first row, and b is the number of chroma samples in its second row. JPEG encoders use *three common schemes*:

- **4:4:4**: No subsampling (24 bits/pixel). Full Y, Cb, and Cr values are preserved for medical imaging or archival purposes.

- **4:2:2**: Horizontal subsampling by 1/2 (16 bits/pixel – 33.3% data reduction). The Cb and Cr channels are subsampled by a factor of 2 horizontally.
- **4:2:0**: Horizontal and vertical subsampling by 1/2 (12 bits/pixel – 50.0% data reduction). **The default sampling scheme in JPEG**. A 4-pixel (2×2) grid contains 4 distinct Y values but shares only 1 Cb value and 1 Cr value.

Figure 2.7 illustrates the three schemes on a 4×2 pixel window: every pixel keeps its own luminance sample (circles), whereas the number of chrominance samples (squares) drops from 8 to 4 and to 2. The effective depth per pixel follows from Equation (2.5), where N_Y and N_C are the numbers of luminance samples and of samples per chrominance channel in the window:

$$b_{\text{eff}} = 8\left(1 + 2\frac{N_C}{N_Y}\right) \text{ bits/pixel} \tag{2.5}$$

With N_C/N_Y equal to 1, 1/2, and 1/4, Equation (2.5) gives 24, 16, and 12 bits/pixel, i.e., 100%, 50%, and 25% of the original color resolution, as stated in the figure.

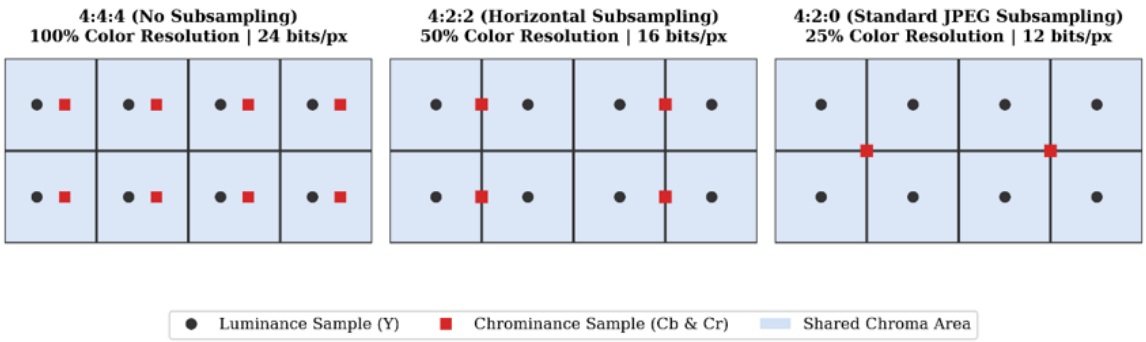


Figure 2.7: The different levels of Chroma Subsampling in JPEG

Table 2.3 lists the storage requirements and typical applications of each scheme.

Table 2.3: Comparison between chroma subsampling schemes

Scheme	Cb/Cr Resolution	Required Bytes per 4 Pixels	Effective Depth / Pixel	Immediate Reduction	Typical Applications
4:4:4	100% Horiz. & Vert.	12 Bytes (4 Y + 4 Cb + 4 Cr)	24 bits / pixel	0% (Reference)	Medical, Fine Art, Master
4:2:2	50% Horiz., 100% Vert.	8 Bytes (4 Y + 2 Cb + 2 Cr)	16 bits / pixel	33.3%	Professional videography
4:2:0	50% Horiz. & Vert.	6 Bytes (4 Y + 1 Cb + 1 Cr)	12 bits / pixel	50.0%	Standard JPEG, Web, Media

By utilizing the 4:2:0 scheme, the JPEG algorithm achieves an **immediate 50% reduction in image size** (a 2:1 compression ratio) during preprocessing, before applying the Discrete Cosine Transform (DCT) and quantization. This significant space savings without perceptible visual degradation makes the YCbCr color space a cornerstone of digital image compression.

2.3 2D Discrete Cosine Transform (2D-DCT)

2.3.1 Image Block Partitioning and DC Level Shift

Following color space transformation and chroma subsampling (e.g., 4:2:0), the digital image undergoes *spatial block partitioning*. Each color channel (Y, Cb, Cr) is systematically divided into non-overlapping square blocks of 8×8 pixels. The spatial dimension of $N = 8$ was selected as the optimal compromise between *two critical requirements*:

- **Computational Complexity**: A direct two-dimensional transform evaluation of size $N \times N$ requires $O(N^4)$ arithmetic operations, whereas optimized separable algorithms require $O(N^3)$ or $O(N^2 \log_2 N)$ operations, where $O(\cdot)$ denotes the order of growth of the operation count. A block size of $N = 8$ minimizes memory footprint and computational overhead, enabling real-time hardware processing.
- **Spatial Correlation**: Statistical inter-pixel correlation in natural images is exceptionally high for neighboring pixels but decays exponentially with distance. A 64-pixel (8×8) window captures the vast majority of local spatial correlation, whereas larger blocks (16×16 or 32×32) yield negligible improvements in *energy compaction* that do not justify the increased computational burden.

If the total dimensions of the original image are not exact multiples of 8, the perimeter boundaries (right and bottom edges) are artificially padded via *edge replication* of the final rows or columns so that the grid divides evenly into autonomous 8×8 blocks.

Figure 2.8 illustrates this partitioning: the image is divided into a grid of 8×8 blocks, and one block is extracted for processing. The right panel shows the extracted block $f(x, y)$, with the column index x running horizontally and the row index y vertically, after the DC level shift described below (Equation (2.7)).

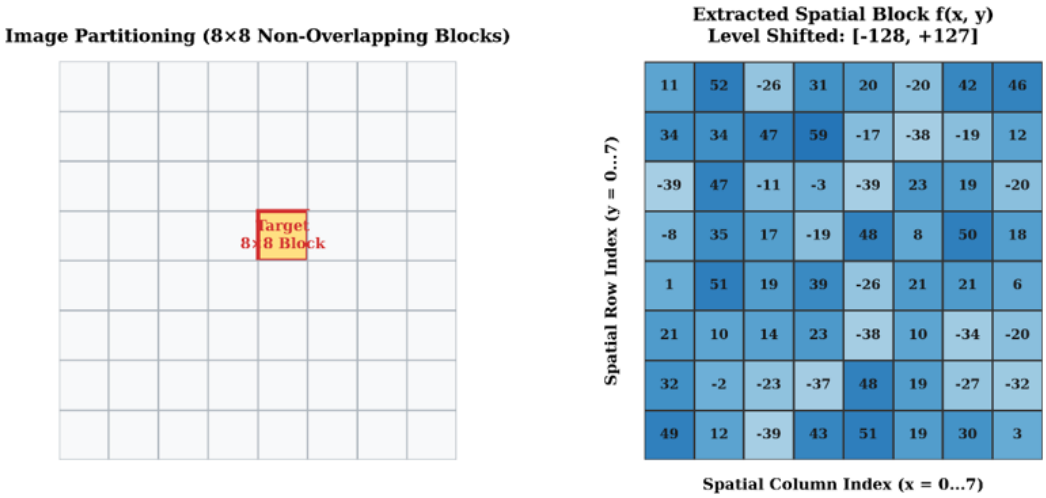


Figure 2.8: Extraction of an 8×8 pixel Block from an image

Prior to applying the mathematical transform, the block data undergo **DC Level Shifting**. In an 8-bit image, uncompressed luminance and chrominance values are expressed as unsigned integers in the range given by Equation (2.6):

$$f(x,y) \in [0,255],\ \forall x,y \in \{0,1,\dots,7\} \tag{2.6}$$

The cosine transform is designed to operate on *zero-mean signals*, allowing values to oscillate symmetrically around zero. Therefore, a constant value of 128 is subtracted from each spatial block sample $f(x,y)$, shifting the dynamic range to the signed integer interval of Equation (2.7):

$$f_{\text{shift}}(x,y) = f(x,y) - 128 \implies f_{\text{shift}}(x,y) \in [-128,+127] \tag{2.7}$$

This level shift prevents register overflow in the DC coefficient (Section 2.3.4) during transform calculations and balances the dynamic range of positive and negative frequency coefficients.

2.3.2 Theoretical Foundation: Selection of the DCT over Alternative Transforms

The primary objective of a transform is to find an orthogonal basis that completely decorrelates the block data and **concentrates the maximum possible signal energy into the fewest number of coefficients**. Three candidate transforms can be evaluated *against this objective*:

- **Karhunen–Loève Transform (KLT)**: Mathematically optimal (100% theoretical energy compaction limit), as its basis functions are derived from the eigenvectors of the signal's own covariance matrix. However, it is impractical for real-time applications

because it is data-dependent (requires basis re-computation per block/image) and lacks a fast computational algorithm.

- **Discrete Fourier Transform (DFT)**: Assumes the input signal is periodic. When a finite 8×8 block is periodically extended, boundary mismatches create artificial step discontinuities. This triggers the *Gibbs phenomenon* (oscillations around a discontinuity that do not disappear as more frequency terms are added), forcing the DFT to generate numerous high-frequency coefficients and severely degrading energy compaction.
- **Discrete Cosine Transform (DCT-II)**: Introduced by Ahmed, Natarajan, and Rao [8], the DCT-II (the second of the standard DCT variants, and the one used by JPEG) exploits the *even-symmetric extension* of the signal. *Mirror-image folding* at block boundaries eliminates edge discontinuities, preserving smooth signal transitions.

Figure 2.9 compares how the two transforms implicitly extend an 8-point block $x(n)$, $n = 0, \ldots, 7$. The DFT assumes the periodic extension of Equation (2.8), which repeats the block with period $N = 8$ and creates a step at every block boundary whenever $x(0) \neq x(7)$. The DCT-II instead assumes the even-symmetric extension of Equation (2.9), which mirrors the block and repeats it with period $2N = 16$, so the extended signal is continuous at the boundaries:

$$\tilde{x}_{\text{DFT}}(n) = x(n \bmod N) \tag{2.8}$$

$$\tilde{x}_{\text{DCT}}(n) = \begin{cases} x(n), & 0 \leq n \leq N-1 \\ x(2N-1-n), & N \leq n \leq 2N-1 \end{cases} \tag{2.9}$$

Because the Fourier coefficients of a signal with a jump decay only in proportion to $1/k$, whereas those of a continuous signal decay at least as fast as $1/k^2$ (where k is the frequency index), **the DCT needs far fewer significant coefficients to represent the same block**.

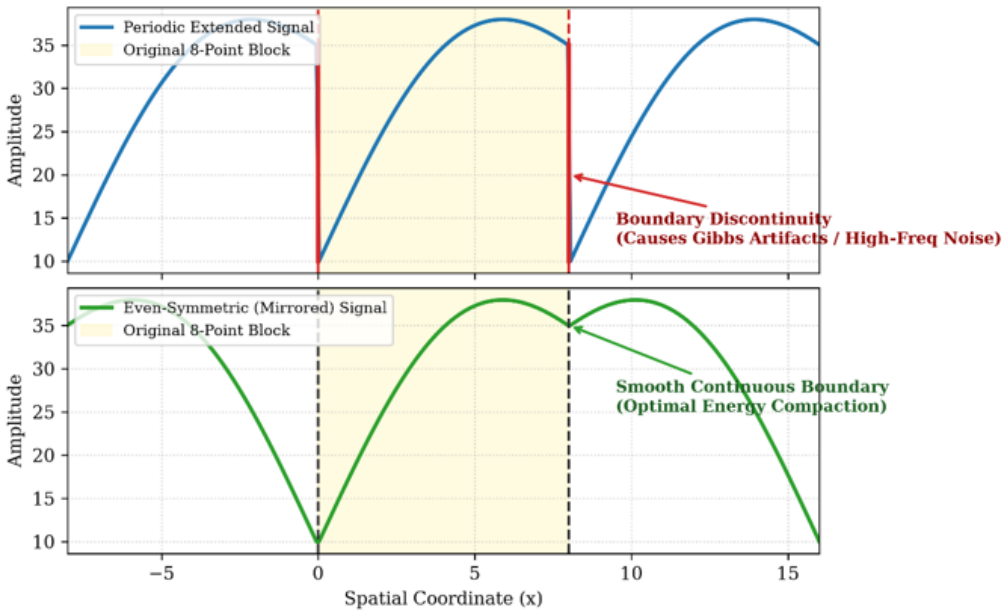


Figure 2.9: DCT advantage to DFT

For highly correlated spatial signals such as natural digital photographs, the performance of the DCT **asymptotically approaches that of the optimal KLT**, while offering fixed, data-independent basis functions and fast algorithmic implementations [8].

Table 2.4 summarizes the comparison of the three transforms.

Table 2.4: Comparative evaluation of transforms for image compression

Evaluation Criterion	Karhunen–Loève (KLT)	Discrete Fourier (DFT)	Discrete Cosine (DCT)
Energy Compaction	Optimal (100% theoretical limit)	Moderate (Gibbs phenomenon)	Near-Optimal (~98% of KLT)
Data Dependency	High (basis computation required)	None (fixed basis)	None (fixed basis)
Signal Extension	Not applicable	Periodic (boundary discontinuities)	Even-Symmetric (smooth)
Number Field	Real numbers	Complex numbers	Real numbers
Fast Algorithm	None ($O(N^2)$ complexity)	Yes (FFT – Fast Fourier Transform)	Yes (Fast DCT)

2.3.3 Mathematical Definition of the 2D-DCT and the Separability Property

The two-dimensional Discrete Cosine Transform (2D-DCT) converts a spatial block of intensity values $f(x,y)$ into an 8×8 spatial frequency matrix $F(u,v)$, as defined by Equation (2.10):

$$F(u,v) = \frac{1}{4}C(u)C(v)\sum_{x=0}^{7}\sum_{y=0}^{7} f(x,y)\cos\left(\frac{(2x+1)u\pi}{16}\right)\cos\left(\frac{(2y+1)v\pi}{16}\right) \qquad (2.10)$$

where $u, v \in \{0,1,\ldots,7\}$ represent horizontal and vertical spatial frequency indices, $x, y \in \{0,1,\ldots,7\}$ denote spatial pixel coordinates, and $C(k)$ is the orthonormal normalization factor defined in Equation (2.11):

$$C(k) = \begin{cases} \frac{1}{\sqrt{2}}, & \text{if } k = 0 \\ 1, & \text{if } k > 0 \end{cases} \tag{2.11}$$

Spatial domain image reconstruction is achieved via the Inverse Two-Dimensional Discrete Cosine Transform (2D-IDCT) of Equation (2.12):

$$f(x,y) = \frac{1}{4}\sum_{u=0}^{7}\sum_{v=0}^{7} C(u)C(v)F(u,v)\cos\left(\frac{(2x+1)u\pi}{16}\right)\cos\left(\frac{(2y+1)v\pi}{16}\right) \tag{2.12}$$

A) Separability Property

A fundamental mathematical property of the 2D-DCT is its **separability**: the two-dimensional transform kernel can be decomposed into the product of two independent one-dimensional (1D-DCT) kernels—one for rows and one for columns. This allows the 2D transform to be computed *in two sequential 1D steps*:

- **Row Pass**: Apply the 1D-DCT across the 8 horizontal rows of the block (Equation (2.13)).

$$G_{\text{row}}(u,y) = \frac{1}{2}C(u)\sum_{x=0}^{7} f(x,y)\cos\left(\frac{(2x+1)u\pi}{16}\right) \tag{2.13}$$

- **Column Pass**: Apply the 1D-DCT down the 8 vertical columns of the intermediate matrix $G_{\text{row}}(u,y)$ (Equation (2.14)).

$$F(u,v) = \frac{1}{2}C(v)\sum_{y=0}^{7} G_{\text{row}}(u,y)\cos\left(\frac{(2y+1)v\pi}{16}\right) \tag{2.14}$$

This property drastically reduces computational complexity from $8^4 = 4{,}096$ operations to just $2 \times 8^3 = 1{,}024$ operations per block (**a 75% reduction in computational overhead**).

B) Matrix Formulation

In modern digital architectures (DSP – Digital Signal Processor, FPGA – Field-Programmable Gate Array, GPU – Graphics Processing Unit), the transform is expressed compactly in the matrix multiplication form of Equation (2.15):

$$F = T \cdot f \cdot T^T \quad \text{and} \quad f = T^T \cdot F \cdot T \tag{2.15}$$

where f is the 8×8 spatial data matrix, F is the 8×8 frequency coefficient matrix, and T is the constant, orthonormal 8×8 DCT transform matrix with elements defined by Equation (2.16):

$$T(i,j) = \frac{1}{2}C(i)\cos\left(\frac{(2j+1)i\pi}{16}\right), \; i,j \in \{0,1,\ldots,7\} \tag{2.16}$$

Because matrix T is *orthogonal*, the inverse relationship $T^{-1} = T^{T}$ holds true, significantly simplifying decoding hardware implementation.

2.3.4 The 64 Basis Functions and Their Physical Interpretation

The 2D-DCT projects spatial pixel values $f(x, y)$ onto an orthogonal vector space defined by the 64 basis functions of Equation (2.17):

$$\Psi_{u,v}(x, y) = \cos\left(\frac{(2x+1)u\pi}{16}\right)\cos\left(\frac{(2y+1)v\pi}{16}\right) \tag{2.17}$$

Each coefficient $F(u, v)$ quantitatively represents the "weight" (amplitude projection) by which the corresponding *basis waveform* $\Psi_{u,v}(x, y)$ contributes to the reconstruction of the original spatial block.

Figure 2.10 displays the 64 basis functions as 8×8 images, arranged so that u increases from left to right and v from top to bottom. Each tile is obtained by evaluating Equation (2.17) at the 64 pixel positions (x, y); values near +1 are drawn white and values near −1 black. Across the 8 pixels of a row, the horizontal factor $\cos((2x + 1)u\pi/16)$ changes sign exactly u times, so the tile in column u shows u light–dark transitions horizontally, and likewise the tile in row v shows v transitions vertically. The top-left tile ($u = v = 0$) is constant.

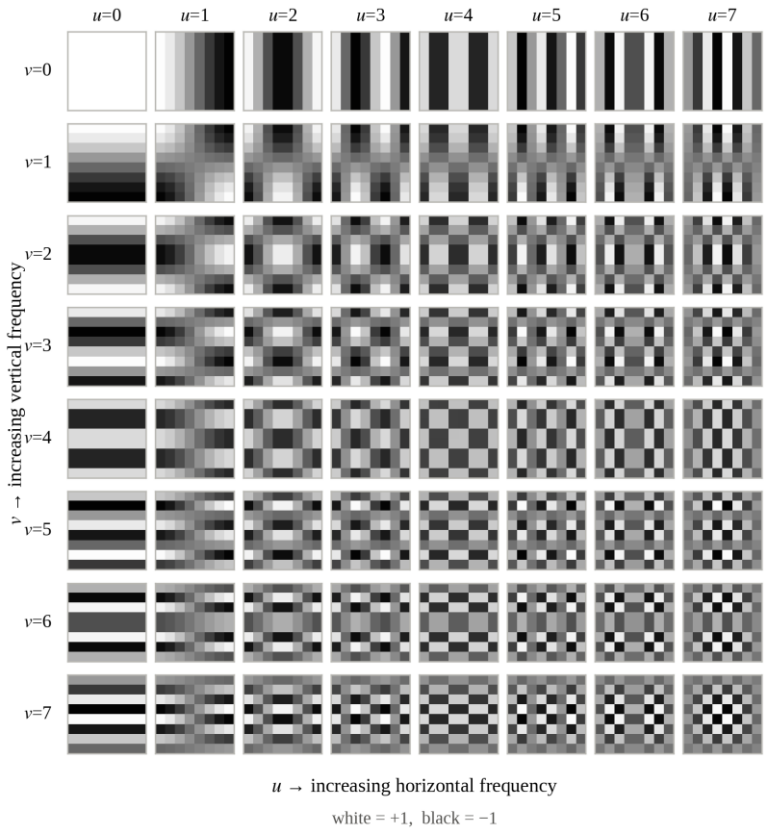


Figure 2.10: Visualization of the different 64 possible frequencies of an 8×8 Block

The arrangement of the 64 coefficients within the matrix follows a strict *spatial frequency hierarchy*, described in the two parts below.

A) The DC Coefficient - F(0,0)

Located at the top-left corner ($u = 0,\ v = 0$), the spatial frequency is zero ($\cos 0 = 1$). The term DC is borrowed from electrical engineering (*Direct Current*) and denotes the zero-frequency component. The DC coefficient mathematically represents the scaled average luminance value of the entire 8×8 block, as shown in Equation (2.18):

$$F(0,0) = \frac{1}{8}\sum_{x=0}^{7}\sum_{y=0}^{7} f(x,y) \tag{2.18}$$

Since $F(0,0)$ equals eight times the mean of the level-shifted block, and every sample lies in $[-128,127]$ (Equation (2.7)), the DC coefficient is bounded to the range $[-1024,1016]$.

B) The 63 AC Coefficients

The remaining 63 coefficients $F(u,v)$ (where $u + v > 0$) are termed AC coefficients (from *Alternating Current*, i.e., the non-zero-frequency components) and correspond to *progressively higher spatial frequencies*:

- **Horizontal Frequencies ($u > 0, v = 0$)**: Represent vertical edge structures.
- **Vertical Frequencies ($u = 0, v > 0$)**: Represent horizontal edge structures.
- **Diagonal Frequencies ($u > 0, v > 0$)**: Represent complex diagonal textures and corners.
- **High Frequencies ($u \to 7, v \to 7$)**: The bottom-right corner represents fine micro-textures, sharp high-contrast transitions, and random digital sensor noise.

2.3.5 The Energy Compaction Phenomenon

The primary justification for adopting the 2D-DCT in JPEG is its exceptional capacity for **energy compaction**. In natural photographic images, smooth and slowly varying surfaces dominate.

When a smooth 8×8 spatial block *undergoes the 2D-DCT*:

- **Low Frequencies**: Approximately 90% to 95% of the total optical signal energy is concentrated in the DC coefficient and 3–5 low-frequency AC coefficients located in the top-left corner of the matrix.

- **Mid Frequencies**: Mid-frequency coefficients assume small, nearly negligible numerical values.
- **High Frequencies**: High spatial frequency coefficients (bottom-right region) asymptotically approach or equal zero.

Figure 2.11 measures this effect on a natural image. The fraction of the block energy captured by a set K of coefficients is measured by Equation (2.19). Because the DCT is orthonormal, it preserves energy (*Parseval's theorem*), so the denominator also equals $\sum\sum f(x,y)^2$ of the spatial block:

$$\eta_K = \frac{\sum_{(u,v)\in K} F(u,v)^2}{\sum_{u=0}^{7}\sum_{v=0}^{7} F(u,v)^2} \tag{2.19}$$

Panel (a) of the figure shows, for the 4,096 luminance blocks of the test image, the share of the total energy carried by each coefficient $F(u,v)$ (summed over all blocks), and panel (b) the cumulative share η_K when the coefficients are taken in Zig-Zag order (Section 2.5.2). **The DC coefficient alone carries 89.6% of the energy**, the first 6 coefficients (DC and the five lowest AC) 97.7%, and the first 15 coefficients 99.1%.

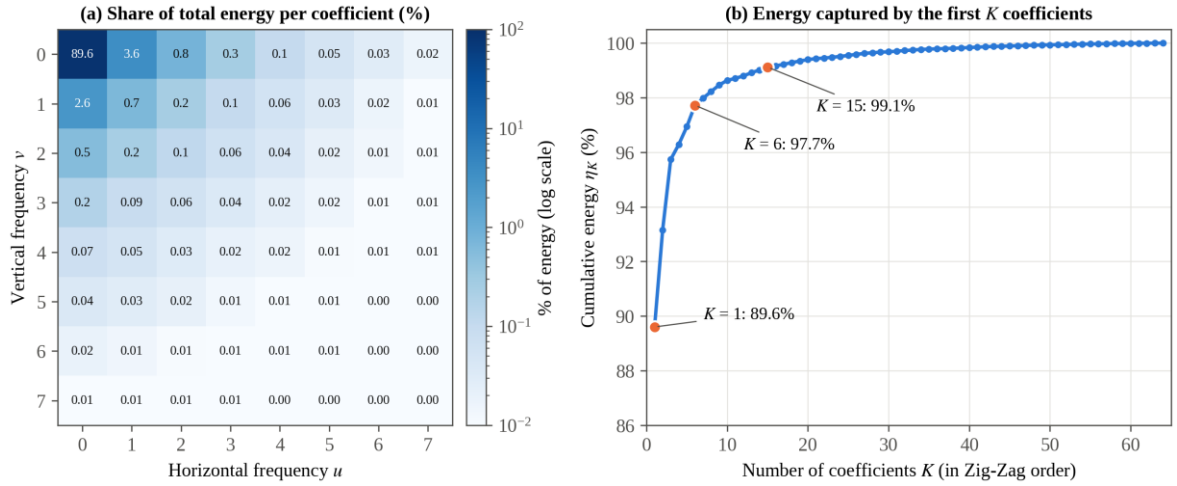


Figure 2.11: Energy compaction of the 2D-DCT: the DC coefficient captures the majority of the energy

The DCT transform is **not inherently lossy** (reconstruction via IDCT using floating-point precision yields zero mathematical error). However, effective energy compaction ideally conditions the signal for the subsequent JPEG stage: quantization, where imperceptible high-frequency coefficients are massively zeroed out without visible image degradation.

2.3.6 Computational Considerations, Fast DCT Algorithms, and Blocking Artifacts

A) Fast DCT Algorithms

The industrial standard implementation of JPEG relies on *Fast DCT Algorithms*, which reduce arithmetic operations *by exploiting cosine symmetry properties*:

- **Chen, Smith & Fralick Algorithm (1977)**: Reduced 8-point 1D-DCT requirements to 16 multiplications and 26 additions using *butterfly structures* (pairs of operations of the form $a + b$, $a - b$, as in the Fast Fourier Transform) [9].
- **Loeffler, Ligtenberg & Moschytz Algorithm (1989)**: Represents the theoretical optimization limit for the 8-point 1D-DCT, requiring exactly 11 multiplications and 29 additions with zero theoretical approximation error [10].

Modern software libraries (such as libjpeg, the reference library of the Independent JPEG Group, IJG) [11] implement *Integer/Fixed-Point DCT variants*. Cosine coefficients are scaled by powers of 2 and rounded, allowing transform execution entirely via fast bit-shift operations and integer additions while preventing floating-point rounding errors.

B) The Blocking Artifacts Phenomenon

The architectural choice in JPEG to partition an image into autonomous 8×8 blocks represents its **primary vulnerability at high compression ratios**. During transformation, each block is processed and quantized independently.

Under aggressive compression (low bitrate / low quality factor Q_{f}, Section 2.4.3), AC coefficients are heavily zeroed out, and each block is reconstructed almost exclusively from its DC coefficient (rendering it as a uniformly illuminated flat tile). Because no mathematical constraint enforces luminance continuity across block boundaries, abrupt *step discontinuities* emerge along the edges at $x, y \in \{0,7\}$.

Figure 2.12 contrasts a 64×64-pixel region of the test image with its reconstruction by a standard JPEG encoder at $Q_{\mathrm{f}} = 5$: most 8×8 blocks are reduced to almost uniform tiles or simple gradients, and step discontinuities appear along the block boundaries (red lines).

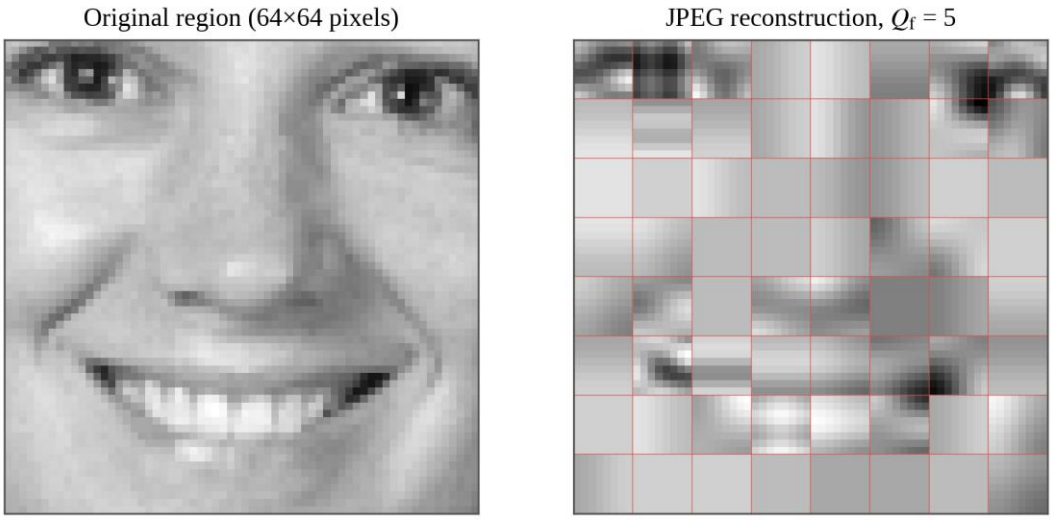


Figure 2.12: Blocking Artifacts, the major JPEG visual problem with high quantization

This structural error is termed a **blocking artifact** and is perceived as an artificial 8×8 grid structure that degrades visual continuity. Despite the emergence of newer *wavelet-based transforms* (such as the Discrete Wavelet Transform, DWT, in JPEG 2000 [12]), the 8×8 2D-DCT remains the industry-standard compromise between processing speed and decorrelation efficiency.

2.4 Quantization

2.4.1 The Psychovisual Foundation of Quantization

Apart from chroma subsampling, image processing up to the 2D-DCT stage **remains lossless**. The 8×8 frequency matrix $F(u, v)$ preserves the exact same information as the original spatial block $f(x, y)$.

Quantization is the **main inherently lossy stage of JPEG**, where permanent data compression occurs based on the physiological limitations of the Human Visual System (HVS). As demonstrated by the Contrast Sensitivity Function (CSF, Figure 2.3), visual acuity drops exponentially at higher spatial frequencies. The eye is highly sensitive to smooth illumination gradients and low-frequency edges, but biologically incapable of perceiving amplitude errors in high-frequency regions (e.g., fine textures or sensor noise).

Figure 2.13 marks this boundary in the pipeline: the 2D-DCT is lossless, quantization is lossy, and its output is a *sparse integer matrix* that feeds the lossless Zig-Zag and Run-Length Encoding stages (Section 2.5).

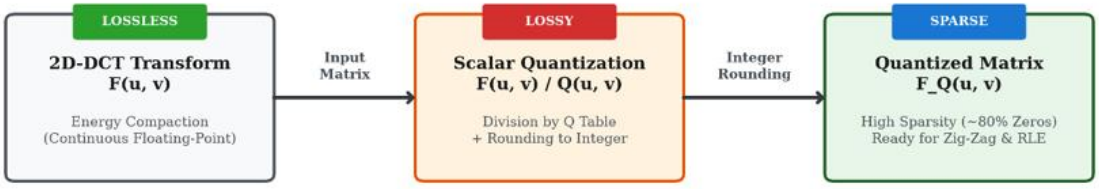


Figure 2.13: Lossless and Lossy parts of JPEG pipeline

Quantization is implemented via a *uniform scalar quantizer*. Each DCT coefficient $F(u,v)$ is divided element-wise by the step size $Q(u,v) \in [1,255]$ from a Quantization Table Q and rounded to the nearest integer, as given by Equation (2.20):

$$F_Q(u,v) = \text{round}\left(\frac{F(u,v)}{Q(u,v)}\right) = \left\lfloor \frac{F(u,v)}{Q(u,v)} + 0.5 \right\rfloor, \; u,v \in \{0,1,\dots,7\} \tag{2.20}$$

During decoding, *Dequantization* (Inverse Quantization) reconstructs the coefficient amplitude by multiplying back by the same step size (Equation (2.21)):

$$F'(u,v) = F_Q(u,v) \cdot Q(u,v) \tag{2.21}$$

Due to irreversible rounding, the reconstructed coefficient $F'(u,v)$ differs from the original $F(u,v)$. The resulting *Quantization Error* $\Delta F(u,v)$ is **strictly bounded**, as shown in Equation (2.22):

$$\Delta F(u,v) = F(u,v) - F'(u,v) \implies |\Delta F(u,v)| \leq \frac{Q(u,v)}{2} \tag{2.22}$$

Larger step sizes increase reconstruction distortion but significantly reduce the bit depth required for storage.

2.4.2 Luminance (Luma) and Chrominance (Chroma) Quantization Tables

The ITU-T T.81 | ISO/IEC 10918-1 (Annex K) standard [4] recommends two industry-standard quantization tables *derived from psychophysical subjective evaluation experiments*:

- **Luminance Table (Q_{Luma})**: Applied to the luminance channel Y (Figure 2.14).
- **Chrominance Table (Q_{Chroma})**: Applied to the chrominance channels Cb and Cr (Figure 2.15).

Figure 2.14 shows the Annex K luminance table as a *heat map*: each cell contains the step size $Q(u,v)$, and its color encodes the magnitude, from blue (fine quantization) to red (coarse quantization).

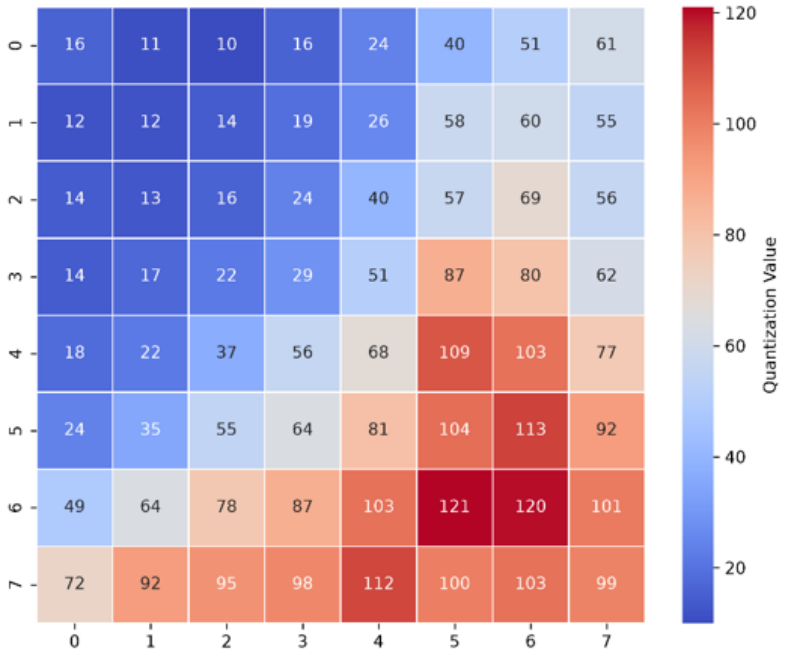


Figure 2.14: Visualization of Luminance Quantization Table

Figure 2.15 shows the corresponding chrominance table on the same color scale; outside the top-left 4×4 region, every entry equals 99.

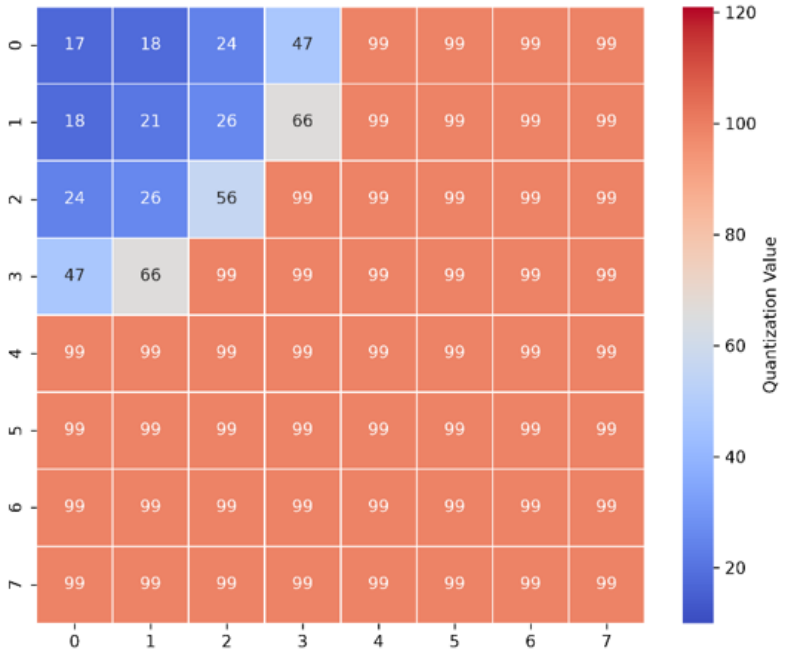


Figure 2.15: Visualization of Chroma Quantization Table

The two tables follow *three design principles*:

- **Low Frequencies**: Top-left step sizes in Q_{Luma} remain small (10 to 16) to preserve mean luminance and structural edges.
- **High Frequencies**: Bottom-right step sizes increase sharply (60 to 121) to suppress imperceptible high-frequency details.

- **Chrominance (Q_{Chroma})**: Due to poor human chromatic acuity, step sizes scale even more aggressively, reaching the recommended maximum (99) by the 4th row/column.

2.4.3 Quality Factor (Q_{f}) and Scaling

Standard Annex K tables correspond to a 50% quality level. To provide dynamic control over file size and image fidelity, the IJG library [11] scales these tables via a dimensionless Quality Factor $Q_{\mathrm{f}} \in [1{,}100]$ *in two steps*:

- **Scaling Factor Calculation**: The scaling factor S_{scale} is computed from Q_{f} by Equation (2.23).

$$S_{\mathrm{scale}} = \begin{cases} \frac{5000}{Q_{\mathrm{f}}}, & \text{if } 1 \le Q_{\mathrm{f}} < 50 \\ 200 - 2Q_{\mathrm{f}}, & \text{if } 50 \le Q_{\mathrm{f}} \le 100 \end{cases} \tag{2.23}$$

- **Adjustment and Clipping**: Each base step size is scaled and clipped by Equation (2.24).

$$Q'(u,v) = \mathrm{clip}\left(\left\lfloor \frac{Q_{\mathrm{base}}(u,v)\cdot S_{\mathrm{scale}}+50}{100} \right\rfloor, 1, 255\right) \tag{2.24}$$

where $\mathrm{clip}(x, 1{,}255)$ constrains values to the integer range $[1{,}255]$, preventing division by zero.

At $Q_{\mathrm{f}} = 100$ ($S_{\mathrm{scale}} = 0$), all step sizes become $Q'(u,v) = 1$, **effectively disabling quantization**. At $Q_{\mathrm{f}} = 50$ ($S_{\mathrm{scale}} = 100$), the base Annex K tables are used unaltered. As $Q_{\mathrm{f}} \to 1$, step sizes maximize ($Q' \to 255$), inducing severe blocking artifacts.

Figure 2.16 shows how Q_{f} controls the trade-off between reconstruction fidelity and compression. Fidelity is measured by the *Peak Signal-to-Noise Ratio* (PSNR) between the original image I and the reconstructed image $\hat{\mathrm{I}}$ of size $M_{\mathrm{img}} \times N_{\mathrm{img}}$ (Equation (2.25)), and compression by the *Compression Ratio* (CR) of Equation (2.26), where S_{img} is the uncompressed size of Equation (2.1) and S_{comp} the size of the compressed bitstream in bits. Both were measured on the test image with the IJG encoder (libjpeg) and 4:2:0 subsampling:

$$\mathrm{PSNR} = 10\log_{10}\frac{255^2}{\mathrm{MSE}}, \quad \mathrm{MSE} = \frac{1}{M_{\mathrm{img}}N_{\mathrm{img}}}\sum_x\sum_y\left(I(x,y) - \hat{\mathrm{I}}(x,y)\right)^2 \tag{2.25}$$

$$\mathrm{CR} = \frac{S_{\mathrm{img}}}{S_{\mathrm{comp}}} \tag{2.26}$$

As Q_{f} decreases, the step sizes of Equation (2.24) grow, more coefficients are rounded to zero, and the CR increases while the PSNR decreases. For the test image, the default $Q_{\mathrm{f}} = 50$ gives

a PSNR of 32.1 dB at a CR of 28.3:1, while $Q_f = 95$ gives 38.3 dB at 7.9:1 and $Q_f = 5$ gives 24.1 dB at 94.8:1.

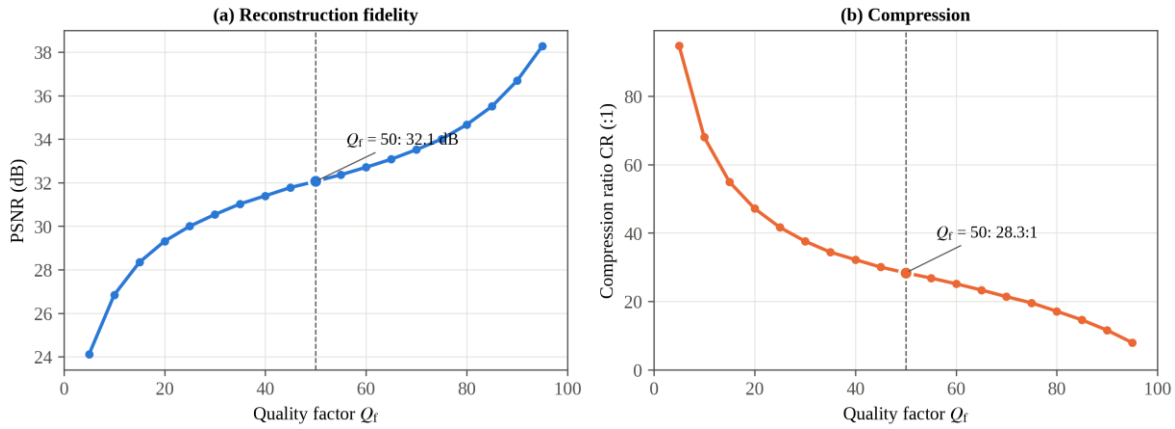


Figure 2.16: Trade-off between Quality of an image and Compression Ratio, measured on the test image

2.4.4 Sparsity Generation and Entropy Coding Preparation

Due to DCT energy compaction, high-frequency AC coefficients exhibit small numerical amplitudes. When divided by the large step sizes in the bottom-right region of $Q(u,v)$, their quotients fall below 0.5 and **round to zero** (Equation (2.27)):

$$\text{If } |F(u,v)| < \frac{Q(u,v)}{2} \implies F_Q(u,v) = 0 \tag{2.27}$$

Figure 2.17 applies Equation (2.20) with the luminance table of Figure 2.14 to one luminance block of the test image. Each value in the left matrix is divided by the step size at the same position and rounded; for example, $F(0,0) = -418.0$ gives $-418.0/16 = -26.1 \to -26$, and $F(1,0) = 80.7$ gives $80.7/11 \approx 7.3 \to 7$. Every coefficient whose magnitude is below half of its step size becomes zero, so **49 of the 64 coefficients (76.6%)** of this block become zero and the matrix on the right is sparse.

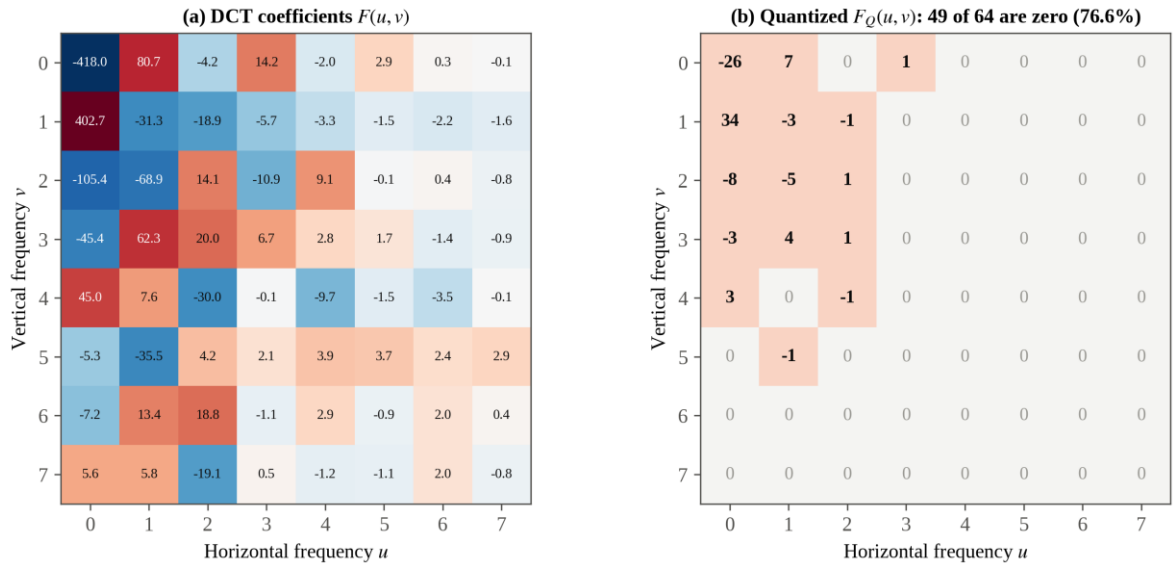


Figure 2.17: Energy compaction and sparsity generation through quantization ($Q_f = 50$)

This converts $F_Q(u, v)$ into a *Sparse Matrix*. At default quality ($Q_f = 50$), 50 to 58 of the 63 AC coefficients are completely zeroed out.

This sparsity is the **foundational requirement for the final compression stages**: Zig-Zag Scanning and Run-Length Encoding (RLE). Grouping consecutive zero-valued high frequencies into run-length pairs rapidly reduces signal entropy, allowing Huffman coding to achieve compression ratios exceeding 15:1 without perceptual degradation.

2.5 Entropy Encoding

2.5.1 Theoretical Background: Statistical Entropy and Lossless Compression

Entropy Encoding is the final stage in the JPEG compression processing pipeline. In stark contrast to the preceding steps—such as quantization and chroma subsampling—entropy encoding is a **completely lossless compression process**. Its primary objective is not the further discarding of visual information, but the optimal representation of the already quantized integer coefficients using the minimum possible number of bits by exploiting the statistical redundancy of the data source.

The theoretical compression limit of a discrete symbol alphabet is defined by *Shannon's Source Coding Theorem* [13]. For a discrete symbol source $S = \{s_1, s_2, \dots, s_n\}$ with corresponding occurrence probabilities p_i, the average informational entropy $H(S)$, expressed in bits per symbol, is defined by Equation (2.28):

$$H(S) = -\sum_{i=1}^{n} p_i \log_2 p_i \tag{2.28}$$

Entropy $H(S)$ represents the **theoretical lower bound** of the average codeword length $\overline{L}$ required to represent the source (Equation (2.29)):

$$H(S) \leq \overline{L} = \sum_{i=1}^{n} p_i l_i < H(S) + 1 \tag{2.29}$$

where l_i is the length in bits of the *prefix code* assigned to symbol s_i (in a prefix code, no codeword is the beginning of another, so the bitstream can be decoded without separators). The success of the JPEG standard lies in its combination of spatial rearrangement (Zig-Zag scanning), differential prediction (DPCM), and run-length encoding (RLE) to transform the initial coefficient distribution into a new statistical source with extremely low entropy $H(S)$, which is then optimally approximated using *Variable-Length Coding* (VLC) via Huffman codes.

2.5.2 Zig-Zag Scanning and Data Serialization

The quantized spatial frequency matrix of dimensions 8×8 constitutes a two-dimensional array. To undergo sequential entropy encoding, it must be *serialized* into a one-dimensional vector of 64 elements.

Typical linear scanning (row-by-row or column-by-column) **fails to exploit the frequency topology** of the Discrete Cosine Transform (DCT). As analyzed in the quantization stage, image energy is concentrated in the top-left region (low frequencies), whereas high-frequency elements toward the bottom-right corner generally evaluate to zero.

For this reason, the JPEG standard enforces a **diagonal Zig-Zag Scanning pattern**. The scan begins at the DC component at position (0, 0) and traverses the matrix diagonally, oscillating toward the highest frequency at position (7, 7). The sequence of coordinates (u, v) during traversal is given by Equation (2.30):

$$(0,0) \rightarrow (1,0) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (1,1) \rightarrow (2,0) \rightarrow (3,0) \rightarrow \cdots \rightarrow (7,7) \tag{2.30}$$

Figure 2.18 shows the scan order on the 8×8 grid (left) and the resulting one-dimensional array $Z[0 \ldots 63]$ for the quantized block of Figure 2.17 (right); after the last non-zero value $Z[23]$, all remaining 40 elements are zero and are replaced by a single EOB symbol (Section 2.5.4). Along each *anti-diagonal* the sum $u + v = d$ is constant, with $d = 0, \ldots, 14$. The scan visits the diagonals in increasing order of d and reverses direction on each one (u decreases along odd diagonals and increases along even ones), so the position of a coefficient in Z grows with its spatial frequency $u + v$.

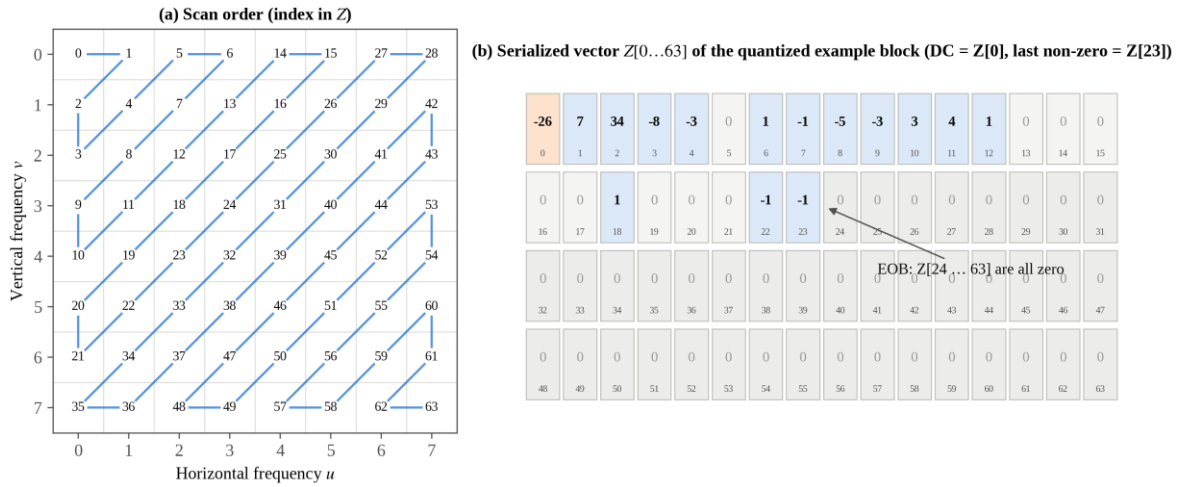


Figure 2.18: Visualization of Zig-Zag Scanning

This diagonal transformation rearranges the coefficients in **strictly increasing order of spatial frequency**. Consequently, all non-zero, low-frequency coefficients are clustered among the first elements of the one-dimensional vector, while almost all middle- and high-frequency zeros accumulate toward the end of the vector, forming extended, continuous chains of zeros. This clustering of zero elements forms the cornerstone of the highly efficient subsequent RLE encoding.

2.5.3 Differential Pulse-Code Modulation (DPCM) for the DC Coefficient

The first element of the Zig-Zag vector is the DC coefficient of the 8×8 block and represents the scaled average brightness or chrominance of the corresponding image region. Because natural images exhibit strong local continuity and lighting rarely changes abruptly from one 8×8 block to its immediate neighbor, the DC coefficients of consecutive blocks exhibit high *inter-block correlation*.

If the DC coefficient were encoded independently, it would require a wide representation range (e.g., 8 to 11 bits). To eliminate this inter-block redundancy, JPEG processes DC coefficients separately from AC coefficients using **Differential Pulse-Code Modulation (DPCM)**.

For each current 8×8 block with index i, the algorithm calculates and transmits exclusively the difference DIFF between the DC value of the current block (DC_i) and the DC value of the immediately preceding processed block (DC_{i-1}), as given by Equation (2.31):

$$\text{DIFF} = DC_i - DC_{i-1} \tag{2.31}$$

For the first block of each Scan (one complete pass over the compressed image data) or at the beginning of each Restart Interval (a group of MCUs after which the encoder resets its

prediction, so that a decoder can resynchronize after a transmission error), the reference value is conventionally reset to zero ($DC_0 = 0$), resulting in $DIFF_1 = DC_1$.

The differential operation shifts the probability distribution of the signal: whereas absolute DC values are uniformly distributed across a wide spectrum, the differences DIFF follow a *Laplace* (double-exponential) distribution [14] with an extremely sharp peak around zero ($DIFF \approx 0$). This **dramatic reduction in variance** minimizes the source entropy, allowing the majority of DC differences to be represented with a very small number of bits.

2.5.4 Run-Length Encoding (RLE) for AC Coefficients

The remaining 63 coefficients of the Zig-Zag vector are termed AC coefficients. Due to the preceding quantization and Zig-Zag rearrangement, the AC coefficient vector is **extremely sparse**, consisting of isolated non-zero values separated by long runs of zeros.

A) Run-Length Pair Format

The JPEG standard applies a specialized two-dimensional form of **Run-Length Encoding (RLE)**. Each non-zero AC coefficient is not stored individually; instead, it is paired with the number of zeros preceding it, producing the data pair of Equation (2.32):

$$(\text{RUNLENGTH, SIZE})(\text{AMPLITUDE}) \tag{2.32}$$

The syntax of this pair obeys *the following representation rules*:

- **RUNLENGTH (4 bits)**: Records the number of consecutive zero coefficients preceding the current non-zero AC coefficient. Because it is expressed in 4 bits, it takes integer values from 0 to 15 (a maximum run of 15 zeros).
- **SIZE / Category (4 bits)**: Records the minimum number of bits required for the exact binary representation of the signed value of the non-zero coefficient itself (the size of AMPLITUDE). It takes integer values from 1 to 10.
- **AMPLITUDE**: The actual signed integer value of the AC coefficient, expressed in a *modified one's complement* format (Section 2.5.5) with a bit-length equal to SIZE.

B) Special RLE Control Markers (ZRL and EOB)

Because the RUNLENGTH field is restricted to 4 bits, it cannot directly represent zero runs longer than 15 elements, and frequently the final elements of an 8×8 block are entirely zeros. The standard resolves these issues by defining *two special control symbols in the RLE table*:

- **ZRL (Zero Run Length — Hex: 0xF0)**: Corresponds to the symbol (15, 0). It denotes a run of 16 consecutive zeros. Multiple ZRL markers can be emitted sequentially to cover extended gaps. For example, a run of 20 zeros followed by the value 3 is encoded as one ZRL symbol (15, 0) followed by the symbol (4, 2) and the AMPLITUDE value 3.
- **EOB (End of Block — Hex: 0x00)**: Corresponds to the symbol (0, 0). This symbol is emitted immediately after representing the last non-zero coefficient of the Zig-Zag vector. It signals to the decoder that all remaining elements up to the 63rd position are zero, instantly terminating block processing without wastefully encoding the remaining empty positions.

2.5.5 Huffman Categories

To optimally manage coefficient amplitudes (for both DC differences DIFF and AC values), JPEG classifies possible integer values into the *Huffman Categories* 0 to 10. Each category k encompasses all signed values that can be represented with exactly k additional amplitude bits (AMPLITUDE bits).

Table 2.5 lists the categories together with the integer value range each one covers; for $k \geq 1$, category k contains the values v_{amp} with $2^{k-1} \leq |v_{\text{amp}}| \leq 2^k - 1$.

Table 2.5: Huffman Size Categories and Corresponding integer value range

Category	Integer Value Range (DC/AC Coefficients)	Required Amplitude Bits (AMPLITUDE)
0	0 (Used only for ZRL / EOB or DIFF = 0)	0
1	−1, 1	1
2	−3, −2, 2, 3	2
3	−7 … −4, 4 … 7	3
4	−15 … −8, 8 … 15	4
5	−31 … −16, 16 … 31	5
6	−63 … −32, 32 … 63	6
7	−127 … −64, 64 … 127	7
8	−255 … −128, 128 … 255	8
9	−511 … −256, 256 … 511	9
10	−1023 … −512, 512 … 1023	10

The binary representation of the additional amplitude bits (AMPLITUDE) follows a clever *bit-saving rule (modified one's complement)*:

- **Positive Values**: AMPLITUDE is the standard binary representation of the integer (e.g., +5 in Category 3 is written as 101). The first bit is always 1.
- **Negative Values**: AMPLITUDE is derived by subtracting the absolute value from $2^k - 1$ (or more simply, by inverting all bits of the positive value's binary representation). For example, −5 in Category 3 is written as 010. The first bit is always 0, allowing the decoder to identify the sign instantaneously without an extra sign bit.

2.5.6 Huffman Coding and Final Bitstream Formation

The final computational step of entropy encoding is the assignment of *Variable-Length Prefixes* via the Huffman algorithm [15]. The Huffman code is constructed by building a binary tree where the **most frequently occurring categories receive the shortest binary sequences** (e.g., 2 to 3 bits), while rarer categories receive longer ones (up to 16 bits). The JPEG standard does not enforce universal default Huffman tables; instead, it allows custom tables to be stored in the file header via the DHT (Define Huffman Table) marker.

The final bitstream for each block coefficient is formed by *concatenating two distinct components*:

- **DC Coefficient**: The input symbol to the DC Huffman table is exclusively the 4-bit category number SSSS of the difference DIFF, i.e., its SIZE category from Table 2.5. The decoder reads the Huffman prefix, determines the size SSSS, and then reads exactly SSSS additional bits from the data stream to reconstruct DIFF.
- **AC Coefficients**: The input symbol to the AC Huffman table is the composite 8-bit byte (RUNLENGTH, SIZE). The Huffman table outputs the variable-length prefix code for that specific pair, immediately followed by the SIZE bits of AMPLITUDE.

2.5.7 Bit-Writer Buffer and Byte Stuffing

Because the bit-lengths of Huffman prefixes and AMPLITUDE bits are arbitrary (not multiples of 8), writing the compressed bitstream to storage cannot be performed using conventional byte-addressing operations. The encoder utilizes a **Bit-Writer Buffer**: a 16-bit or 32-bit register into which successive encoded bits are pushed. As soon as 8 bits (1 octet / byte) accumulate, the completed byte is written to memory and the register shifts left.

Here, a critical hardware problem arises: the JPEG standard defines various *Control Markers* that delineate file structure, such as Start of Image (SOI — 0xFFD8), Define Quantization

Table (DQT — 0xFFDB), and Start of Scan (SOS — 0xFFDA). Every standard marker begins with the hexadecimal prefix 0xFF followed by a non-zero byte.

During the random generation of Huffman and AMPLITUDE bits, it is entirely possible for an octet with the value 0xFF to occur randomly within the compressed pixel data stream. If read as-is, the decoder would misinterpret the pixel data as a structural control marker, **causing decompression to crash**.

To prevent this, the JPEG standard enforces *a Byte Stuffing mechanism*:

- **In the Encoder**: Whenever the Bit-Writer outputs a byte with the exact value 0xFF within the entropy-coded image data, the algorithm obligatorily inserts a zero byte (0x00) immediately after it, as expressed by Equation (2.33).

$$0xFF \rightarrow 0xFF\ 0x00 \tag{2.33}$$

- **In the Decoder**: While reading the bitstream, the hardware inspects every byte equal to 0xFF *and distinguishes two cases*:
  - **Stuffed Byte (0x00)**: If the immediately following byte is 0x00, the decoder recognizes it as valid pixel data, strips (discards) the 0x00, and retains the 0xFF in the bit buffer.
  - **Marker Byte (non-zero)**: If the following byte is any non-zero value (e.g., 0xD9 for End of Image — EOI), the decoder recognizes that the pixel stream has terminated and executes the appropriate control marker routine.

2.6 Summary of the Compression Pipeline

2.6.1 The Three Phases of the Pipeline

The JPEG compression algorithm transforms spatial pixel data into a compact, entropy-coded bitstream by **decoupling lossy visual redundancy reduction from lossless statistical compression** across *three consolidated phases*:

- **Color & Resolution Preprocessing**: Raw RGB pixels (0–255) are converted to the YCbCr color space. Applying 4:2:0 chroma subsampling reduces color resolution (Cb/Cr) by 75%, grouping pixels structurally into 8×8 blocks and 16×16 Minimum Coded Units (MCUs), the smallest group of blocks that the encoder processes as one unit (four Y blocks, one Cb block, and one Cr block in 4:2:0).

- **Forward 2D-DCT & Quantization (Lossy Stage)**: After level-shifting (subtracting 128), the 2D-DCT transforms pixel blocks into spatial frequency coefficients $F(u, v)$. These are element-wise divided by quantization tables (Q_{Luma} / Q_{Chroma}) and rounded to integers, eliminating high-frequency noise to yield a highly sparse matrix $F_Q(u, v)$.
- **Entropy Encoding (Lossless Stage)**: The sparse 2D matrix is serialized into a 64-element vector via Zig-Zag scanning. Inter-block correlation is compressed using DPCM for the DC coefficient, while zero-chains are condensed via RLE (ZRL / EOB) for AC coefficients. Finally, Huffman coding assigns variable-length prefixes, and byte stuffing (0xFF → 0xFF 0x00) prevents structural marker collisions in the final bitstream.

Figure 2.19 summarizes the complete pipeline as a flow diagram, from the raw RGB input to the final JPEG bitstream: the blue stages prepare the data, the red stage (quantization) is the lossy step, and the green stages perform lossless entropy coding.

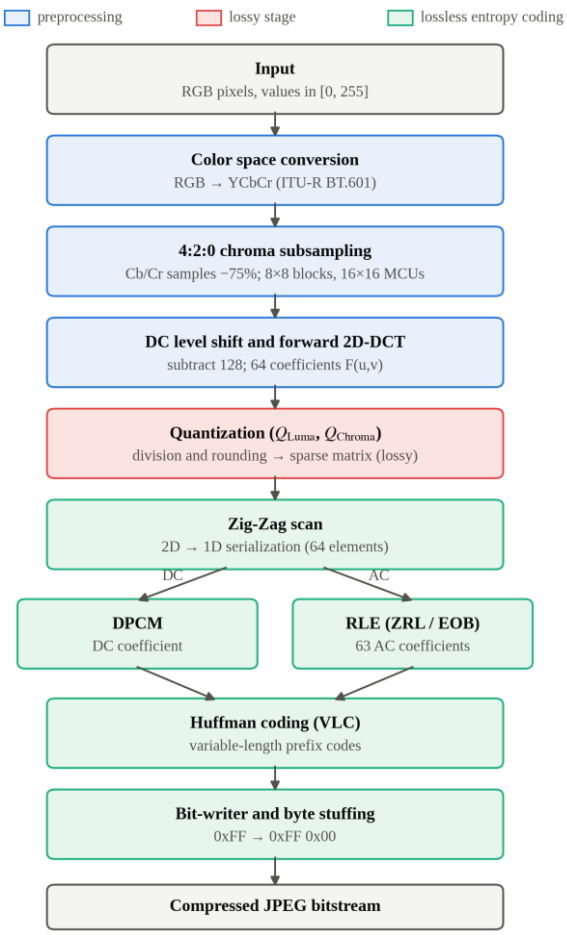


Figure 2.19: Complete Theoretical JPEG pipeline

2.6.2 Architectural Mapping on RISC-V Snitch

This mathematical pipeline serves as the direct blueprint for *our single- and multi-core RISC-V Snitch hardware implementations* (Chapter 3 introduces the RISC-V instruction set and the Snitch cluster):

- **Memory Locality**: Maximized by tiling data into self-contained 16×16 MCUs.
- **Vectorization**: SIMD (Single Instruction, Multiple Data) acceleration applied heavily across 2D-DCT matrix multiplications and element-wise quantization divisions.
- **Parallel Scaling**: Multi-core throughput scaled by distributing independent 8×8 blocks or entire Restart Intervals across available compute cores.

3 The RISC-V Architecture and the Snitch System

3.1 The RISC-V Architecture

3.1.1 The RISC-V Instruction Set Architecture (ISA)

RISC-V (*Reduced Instruction Set Computer - V*) is an open-source Instruction Set Architecture (ISA) originally developed at UC Berkeley and now governed by RISC-V International. Unlike proprietary architectures (such as x86 or ARM), RISC-V is *royalty-free*, allowing developers to design, modify, and implement custom processors without licensing fees [16], [17].

The core strength of RISC-V is its **modular design**. Rather than forcing a single, monolithic instruction set onto every processor, it separates the architecture into a *mandatory Base Integer ISA and optional Standard Extensions*:

- **Base Integer ISA** (e.g., RV32I / RV64I): A small, efficient set of ~40–50 basic integer instructions sufficient to run modern software and operating systems.
- **Standard Extensions**: Modular add-ons that can be included based on the system's performance and power requirements.

Figure 3.1 illustrates this modular organization. At its center is the Base Integer ISA (RV32I/RV64I), from which the standard extensions branch out: M for integer multiplication and division, A for atomic operations, F/D for floating-point arithmetic, C for compressed 16-bit instructions, and V for vector (SIMD) processing. Beyond the standard set, the ISA leaves room for custom extensions, such as the DSP and JPEG-oriented accelerators shown on the right of the figure, which are discussed in Section 3.1.3.

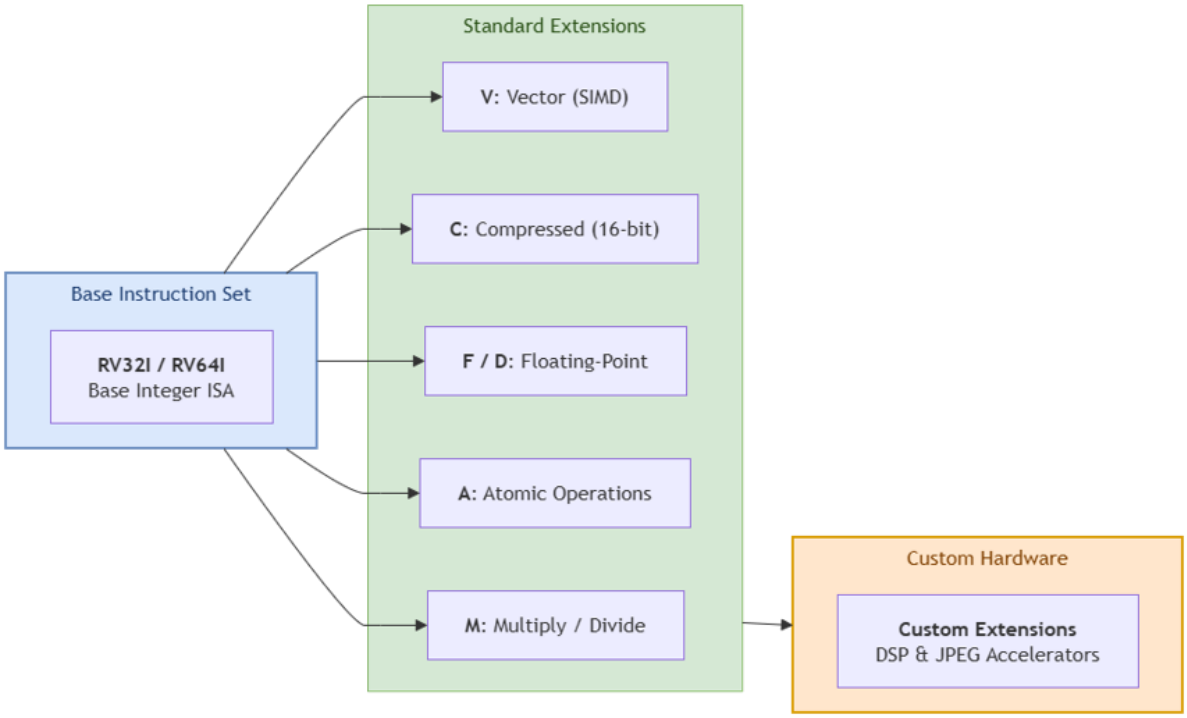


Figure 3.1: RISCV Instruction Categorization

3.1.2 Register File and Standard Extensions

The standard RV32I/RV64I architecture provides 32 General-Purpose Registers (x0–x31). Notably, x0 is *hardwired to zero*, which simplifies instruction decoding and hardware execution.

When building a processor, designers can combine the base integer set (I) with standard extensions. The bundle of I + M + A + F + D is commonly referred to as G (*General-Purpose*). The most important standard extensions and their functions are summarized in Table 3.1 [16].

Table 3.1: Key standard extensions of the RISC-V ecosystem

Extension	Name	Core Function & Benefit
I / E	Base Integer	Minimal integer instruction set. (E is a 16-register version for tiny embedded microcontrollers.)
M	Multiply / Divide	Adds hardware integer multiplication and division instructions.
A	Atomic	Synchronization instructions for multi-core processors.
F / D	Floating-Point	Single- (32-bit) and Double-precision (64-bit) floating-point arithmetic.
C	Compressed	Encodes common instructions in 16 bits (instead of 32 bits), reducing code size by ~25%.

Extension	Name	Core Function & Benefit
V	Vector (SIMD)	Data-parallel operations for multimedia, image processing, and DSP workloads.

3.1.3 Custom Extensions and Domain-Specific Acceleration

RISC-V reserves specific opcode spaces explicitly for *Custom Instructions* [16]. This allows hardware engineers to design Domain-Specific Architectures (DSA)—custom hardware accelerators integrated directly into the CPU pipeline—**without breaking standard compiler toolchains**.

This is possible because every RISC-V instruction follows a small set of fixed 32-bit formats [16], shown in Figure 3.2. In all formats, the opcode occupies the lowest seven bits [6:0], and the register fields (rs1, rs2, rd) stay at the same bit positions. R-type instructions operate on two source registers. I-type instructions replace the second source register with a 12-bit immediate, used for arithmetic with constants and for loads. S-type instructions split the immediate around rs2 to encode store offsets. Because the fields are always in the same place, decoding is simple, and **a custom instruction can reuse an existing format under a reserved opcode** without changing the rest of the pipeline.

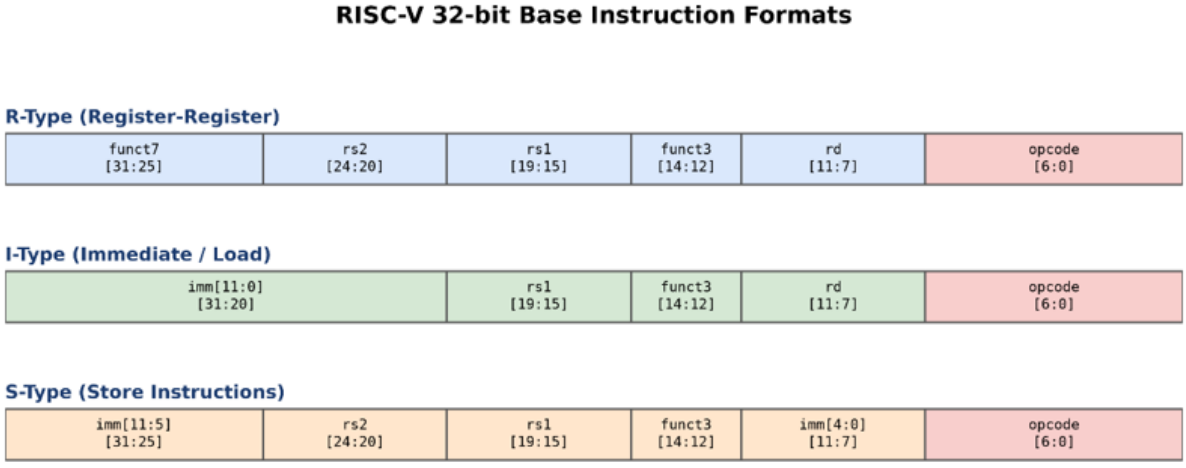


Figure 3.2: RISCV Instruction Formats

This flexibility bridges general-purpose programmability with high-performance hardware acceleration, and it is exactly the mechanism exploited by the Snitch system described in Section 3.2.

3.1.4 Energy and Thermal Constraints in Edge Computing

In embedded edge computing systems, data processing is executed locally and in real time, imposing *two strict architectural constraints* on the hardware:

- **Strict Power and Energy Budgets**: The absence of active cooling (fans, bulky heatsinks) and frequent reliance on battery power enforce operation within a strictly defined Thermal Design Power (TDP). The critical metric is not merely execution time, but minimizing the energy consumed per computational operation (Energy per Operation).
- **Deterministic Low Latency**: Image and video processing applications require guaranteed throughput performance without the unpredictable network latencies introduced by remote cloud computing.

3.1.5 The Architectural Advantage of Multi-core Clusters

The dynamic power consumption of digital CMOS circuits depends on the supply voltage and the clock frequency. Because the supply voltage must scale roughly linearly with frequency, **dynamic power grows with the cube of the clock frequency**, as expressed in Equation (3.1) [18]:

$$P_{dyn} \propto {V_{dd}}^2 \cdot f_{\text{clk}} \propto {f_{\text{clk}}}^3 \tag{3.1}$$

For this reason, modern hardware design abandons the power-hungry overclocking of complex single-core processors (*Out-of-Order*) in favor of Multi-core Clusters composed of simpler, energy-efficient *In-Order* cores.

The core concept relies on *Voltage-Frequency Scaling*: instead of a single core executing the workload at the maximum frequency f_{clk}, the workload is partitioned in parallel across N_{cores} simpler cores operating at a fraction of the frequency (f_{clk}/N_{cores}) and at a lower supply voltage (V_{low}). Because dynamic power scales cubically, the combined power consumption of all N_{cores} cores **remains a fraction of the original single-core power**, while total data throughput is maintained or even increased.

Figure 3.3 summarizes this trade-off. At the top, a single complex out-of-order core must run at a high frequency f_{clk} and supply voltage V_{dd} to meet its throughput target, pushing its dynamic power toward the f_{clk}^3 thermal wall. At the bottom, the same workload is partitioned across N_{cores} simple in-order cores, each running at f_{clk}/N_{cores} and a reduced voltage V_{low}.

Together they deliver the same throughput at a fraction of the power. This is the principle on which clusters such as Snitch are built.

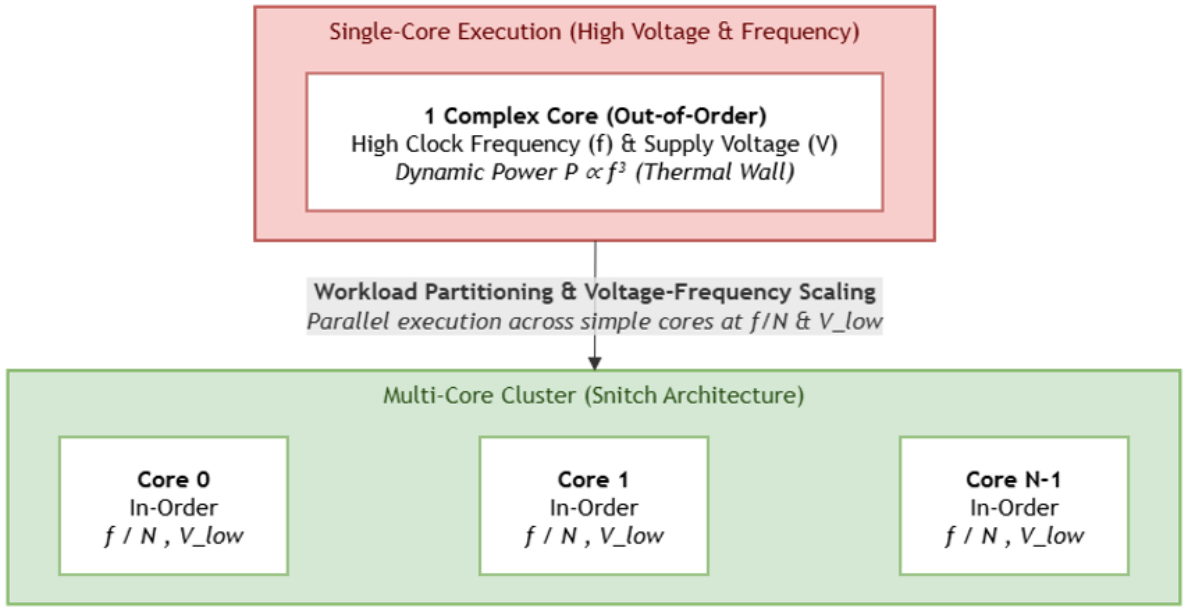


Figure 3.3: From 1 Core to Multiple Core Systems

Table 3.2 contrasts the two design approaches parameter by parameter.

Table 3.2: Comparative analysis of a single-core versus a multi-core architecture.

Parameter	**High-Frequency Single-Core**	**Multi-core Cluster (N_{cores} Cores)**
Operating Frequency (f_{clk})	High (f_{clk})	Low (f_{clk}/N_{cores})
Supply Voltage (V_{dd})	High (V_{dd})	Low (V_{low})
Core Architecture	Complex (Out-of-Order, Speculative)	Simple, Energy-Efficient (In-Order)
Total Power (P)	$\propto f_{clk}^3$ (Cubic increase)	$\propto N_{cores} \cdot (f_{clk}/N_{cores})^3 = f_{clk}^3/N_{cores}^2$ (Drastic reduction)
Energy per Operation	High	Minimal

3.1.6 Exploiting Data Parallelism in Image Algorithms

The performance of a multi-core cluster depends directly on the degree of algorithm parallelism, governed by *Amdahl's Law* [19]. Image and video processing algorithms—such

as JPEG compression—represent an ideal workload for multi-core systems, as they possess high inherent *Data Parallelism*:

- **Spatial Block Autonomy**: In the JPEG standard, an image is partitioned into independent 8x8 pixel blocks (or 16x16 MCU blocks). The most computationally demanding phases—such as the 2D Discrete Cosine Transform (2D-DCT) and Quantization—are executed independently on each block without inter-block data dependencies.
- **Zero Inter-Core Synchronization Overhead**: Data independence eliminates the need for inter-core messaging or complex cache coherency overhead during the transformation stages.
- **Near-Linear Speedup**: Distributing the total set of blocks across N_{cores} available cores reduces execution time almost linearly, as shown in Equation (3.2):

$$T_{exec}(N_{\text{cores}}) \approx \frac{T_{exec}(1)}{N_{\text{cores}}} \tag{3.2}$$

- **Memory Locality Optimization**: In tightly-coupled multi-core clusters, cores access block data directly from a shared, low-latency on-chip scratchpad memory, avoiding power-hungry data movement from external system RAM. The Snitch implementation of this principle, the TCDM, is detailed in Section 3.2.3.

3.2 The Snitch Cluster Architecture

Snitch is an open-source research architecture for *ultra-low-power* multi-core computing systems, developed jointly by the Swiss Federal Institute of Technology in Zurich (ETH Zurich) and the University of Bologna within the framework of the PULP (Parallel Ultra-Low-Power) platform [1], [20], [21]. Snitch exploits the custom-extension flexibility of RISC-V described in Section 3.1.3 to add specialized instructions for digital signal processing and image encoding on top of a standard RISC-V core.

The design philosophy of Snitch is grounded in a critical observation regarding modern hardware efficiency: in conventional high-performance microprocessors, the vast majority of silicon area, along with **up to 80% of the energy budget**, is consumed by complex control structures and control-flow management. Mechanisms such as branch prediction, register renaming, dynamic scheduling, and out-of-order execution are dedicated to maintaining a

high Instructions Per Cycle (IPC) rate, leaving only a small fraction of the total power for executing actual mathematical operations [1], [22].

In contrast to general-purpose central processing units that prioritize single-thread latency through massive instruction-window buffers, embedded edge applications require predictable, high-throughput data processing under stringent thermal constraints. Snitch addresses this challenge by adopting a radical **control-compute decoupling paradigm**. Instead of relying on bulky general-purpose cores, it pairs a minimalistic, highly simplified integer RISC-V core (acting as a control-flow manager) with a disproportionately wide and powerful Floating-Point Unit (FPU). This approach maximizes the mathematical compute density per unit of silicon area (GFLOPS/mm^2) and minimizes the energy cost per operation, making it exceptionally well-suited for repetitive, compute-bound kernels such as those found in image compression and digital signal processing pipelines.

Figure 3.4 shows the reference *8+1 topology* of the cluster used in this thesis. Eight compute cores (RV32IMAFD, each with a 64-bit FPU supporting SIMD) share a 16 kB, 2-way instruction cache. Operand data is served from a 128 kB L1 TCDM scratchpad, organized as a 32-bank interleaved crossbar that supports double-buffering. A ninth core, the DM core, has its FPU and SIMD units disabled and instead controls a 512-bit DMA engine that moves data between the TCDM and the external L3 DRAM. Each of these components is examined in the sections that follow.

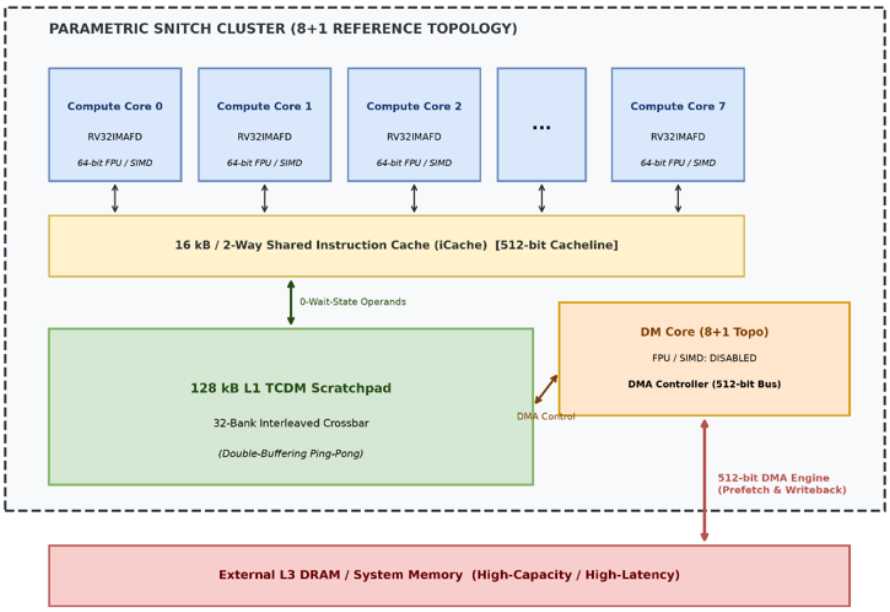


Figure 3.4: Snitch Cluster Architecture Visualization

Parametric Hardware Generation

One of the most defining features of the Snitch architecture is its **parametric hardware generation**. Unlike static architectures that enforce a rigid physical layout, Snitch utilizes an automated hardware generation framework driven by JSON configuration files (e.g., cfg/*.json) and SystemVerilog generators [21], [23].

Through this parametric flow, system architects can dynamically specify *key parameters at synthesis time*:

- **Core Count**: The total number of compute cores within the cluster (scaling from compact 4-core setups to large multi-cluster arrays).
- **Data Width**: The data bus width and integer register sizing (RV32 or RV64).
- **FPU Capabilities**: The capabilities and precision of the floating-point unit (ranging from standard 32-bit to 64-bit with packed SIMD subdivisions).
- **Memory Organization**: The memory hierarchy sizing and organization (number of TCDM banks, L1 iCache capacity).
- **Hardware Accelerators**: The integration of specialized hardware accelerators (such as STREAM, SSR, and FREP engines).

This flexibility enables the design of *Domain-Specific Architectures* (DSAs) that optimize the compute-to-area ratio specifically for targeted workloads. By tailoring the hardware configuration parameters directly to the architectural demands of algorithms like the JPEG 2D-DCT and quantization, designers can eliminate unneeded silicon overhead, achieving optimal energy efficiency profiles that static commercial processors cannot match.

3.2.1 Compute Cores and the Floating-Point Unit (FPU)

At the structural level, the compute cores are grouped into organizational units called *Hives*. Although the number of cores per Hive is configurable, a highly established reference topology in academic literature consists of a Hive with 8 Compute Cores [1].

To minimize silicon area overhead and eliminate the need for complex cache-coherency protocols, all cores in a Hive share a 16 kB Shared Instruction Cache (iCache). This cache is 2-way *set-associative* and features a wide 512-bit cacheline. Because scientific computing and digital signal processing workloads typically follow SPMD (Single Program, Multiple Data) execution models, the 8 cores run identical code paths with offset memory pointers, **making a**

shared iCache exceptionally efficient while drastically reducing the required instruction memory footprint across the physical silicon.

Compute Core Microarchitecture

Each compute core implements the RV32IMAFD (or RV64IMAFD) instruction set, and its internal organization is shown in Figure 3.5. An *in-order fetch and decode stage* issues instructions to two decoupled parts. On the right is the integer pipeline, whose address generation unit handles loop counters and memory pointers. On the left is the multi-pipelined FPU, which contains a packed SIMD pipeline (2× FP16 or 4× FP8, 1-cycle latency) and an FP64 pipeline for fused multiply-add (3-cycle latency). Both parts are connected directly to the TCDM: the FPU reads and writes operands, while the integer side supplies the data addresses.

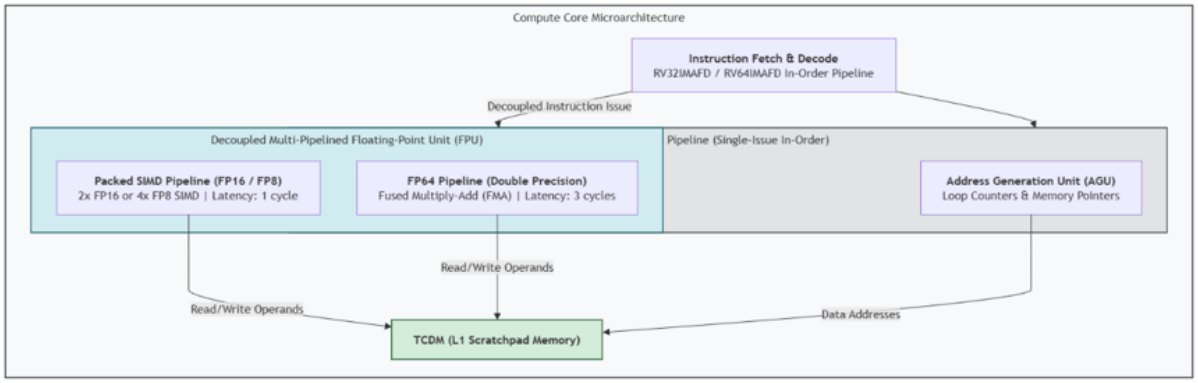


Figure 3.5: Snitch Cluster Compute Core Architecture

Each compute core therefore consists of *two distinct subsystems*:

- **Minimalistic RISC-V Integer Core**: Implements a simple single-issue, in-order execution pipeline. The role of this core is strictly confined to program control flow: it calculates address arithmetic, increments loop counters, and resolves branches.
- **Decoupled 64-bit FPU**: A multi-pipelined floating-point unit architecturally decoupled from the integer pipeline and optimized for Fused Multiply-Add (FMA) operations of the form given in Equation (3.3):

$$d = a \cdot b + c \tag{3.3}$$

where multiplication and addition complete with a single rounding step, ensuring high numerical precision.

To further streamline dense numerical loops, the compute cores integrate specialized hardware extensions such as the *Streamer* (SSR) and Floating-Point Repeat (FREP), which

are presented in detail in Section 3.3. The Streamer automates multi-dimensional address generation in hardware, removing explicit pointer arithmetic instructions from the inner execution loops. Meanwhile, FREP allows small blocks of instructions to be cached and re-executed in hardware loops without instruction-fetch overhead, ensuring that the pipeline remains perpetually saturated with mathematical workloads.

Asymmetric Execution Latency and SIMD Precision

The Snitch FPU exhibits **asymmetric execution latency** depending on the operand data type, balancing *numerical precision against computational throughput*:

- **Double Precision** (FP64): Standard 64-bit floating-point instructions complete with a latency of 3 clock cycles (lat_comp_fp64: 3). Providing FP64 precision guarantees stability for demanding linear algebra and scientific simulations by preventing cumulative rounding errors.
- **Packed SIMD Half and Quarter Precision** (FP16 / FP8): For workloads that do not require 64-bit precision—such as image processing, multimedia compression, and deep neural networks—the FPU supports SIMD (Single Instruction, Multiple Data) execution. These packed formats allow the simultaneous processing of two independent 16-bit operations (FP16) or four 8-bit operations (FP8) within a single 64-bit register. These operations are fully parallelized in hardware and execute in exactly 1 clock cycle (lat_comp_fp16: 1), doubling (for FP16) or quadrupling (for FP8) the theoretical peak throughput of the cluster in FLOPs per cycle [24].

3.2.2 Data Management Core (DM Core) and Direct Memory Access (DMA)

A fundamental challenge in parallel high-performance computing is the *Memory Wall* and the resulting *compute starvation*. As compute cores execute math-intensive kernels, their demand for operand data scales rapidly. If cores are forced to fetch data directly from external DRAM, the floating-point pipeline stalls while waiting for high-latency bus transactions to complete.

To overcome this bottleneck, the standard Snitch cluster topology embeds a ninth, highly differentiated core alongside the 8 compute cores: the **Data Management Core (DM Core)**, forming the established "8+1" architecture topology.

Figure 3.6 shows how data moves in this topology. The DM core, with its FPU and SIMD units disabled but its 512-bit DMA controller active, issues asynchronous transfers that bring data blocks from the high-latency external DRAM into the shared L1 TCDM. The eight

compute cores have the opposite configuration (FPU and SIMD active, DMA disabled). They never access external memory directly; instead, they read and write their operands in the TCDM with zero wait states.

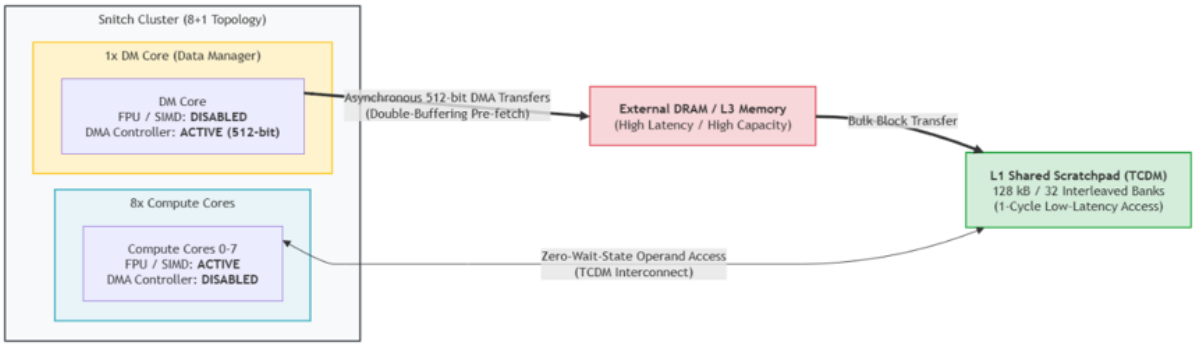


Figure 3.6: Snitch Cluster visualization of Data Block Movement

The DM Core is an independent RISC-V core running its own microcode and exhibiting **radical architectural asymmetry** compared to the Compute Cores. The exact configuration of each core type, taken from the reference cluster configuration [21], is listed in Table 3.3.

Table 3.3: RISC-V extensions used in the two types of Snitch Cluster cores

Feature / Subsystem	Compute Cores (Cores 0–7)	Data Management Core (DM Core)
Instruction Set Architecture	RV32IMAFD / RV64IMAFD	RV32IMAFD / RV64IMAFD
64-bit / 32-bit FPU	Active (fpu: true)	Disabled (fpu: false)
SIMD Unit (FP16 / FP8)	Active (xf16: true)	Disabled (xf16: false)
Streamer / Loop Extensions (SSR / FREP)	Active (xssr: true, xfrep: true)	Disabled (xssr: false, xfrep: false)
DMA Controller (xdma)	Disabled (xdma: false)	Active – Full Control (xdma: true)
Primary Architectural Role	100% Mathematical Computation	100% Data Movement Orchestration

Compute cores have no access to the DMA controller, dedicating every available clock cycle to executing computations on data already resident in local memory. Conversely, the DM Core is stripped of power-intensive floating-point pipelines and hardware extensions, **operating exclusively as an orchestrator for data movement**.

The DMA Subsystem, TCDM, and Double-Buffering

The Direct Memory Access (DMA) controller managed by the DM Core interfaces with the external system bus via an ultra-wide 512-bit channel (dma_data_width: 512). This allows high-speed bulk block transfers (*burst transactions*) between the external memory hierarchy and local storage with minimal per-byte overhead [25].

The local memory of the Snitch cluster consists of a 128 kB shared Tightly-Coupled Data Memory (TCDM) scratchpad. The TCDM is organized into 32 interleaved memory banks connected to the compute cores via a low-latency *crossbar interconnect*, allowing simultaneous read and write accesses across different banks. Its organization is examined in detail in Section 3.2.3.

By leveraging this memory organization, the DM Core coordinates an asynchronous **double-buffering (ping-pong buffering)** scheme. While the compute cores execute vector transformations and mathematical kernels on data residing in one section of the TCDM, the DM Core simultaneously prefetches the next batch of working tiles from the external L3 DRAM into the alternate TCDM section via the wide DMA channel. This continuous overlapping of communication and computation **effectively hides external memory latency**, ensuring that the computational pipelines of the 8 compute cores remain continuously saturated.

The primary software responsibility of the DM Core is to execute this Double-Buffering (Ping-Pong Buffering) schedule within the TCDM, achieving *Compute-Communication Overlap*:

- **Buffer Partitioning**: The L1 TCDM scratchpad address space is logically partitioned into two equal regions: *Buffer A* and *Buffer B*.
- **Asynchronous Prefetching**: While the 8 Compute Cores process data blocks resident in *Buffer A*, the DM Core issues non-blocking DMA requests to prefetch the next data tile from L3 DRAM into *Buffer B*.
- **Ping-Pong Toggle**: Once the Compute Cores finish processing Buffer A and the DMA completes loading Buffer B, the roles of the buffers swap. The Compute Cores begin processing Buffer B, while the DM Core writes the completed results from Buffer A back to main memory and prefetches the next memory tile.

Figure 3.7 shows this schedule as a sequence of three phases. In Phase 1, the DM core loads the first tile (Tile 0) into Buffer A (steps 1–2). In Phase 2, it fetches Tile 1 into Buffer B (steps

3–4), while in parallel the compute cores read Tile 0 from Buffer A, execute the 2D-DCT and quantization, and write the results back (steps 5–6). In Phase 3, the buffers swap roles: the DM core writes the processed Tile 0 back to DRAM and loads Tile 2 into Buffer A (steps 7–8), while the compute cores work on Tile 1 in Buffer B (steps 9–10). From this point the pattern repeats, so **data transfer is always overlapped with computation**.

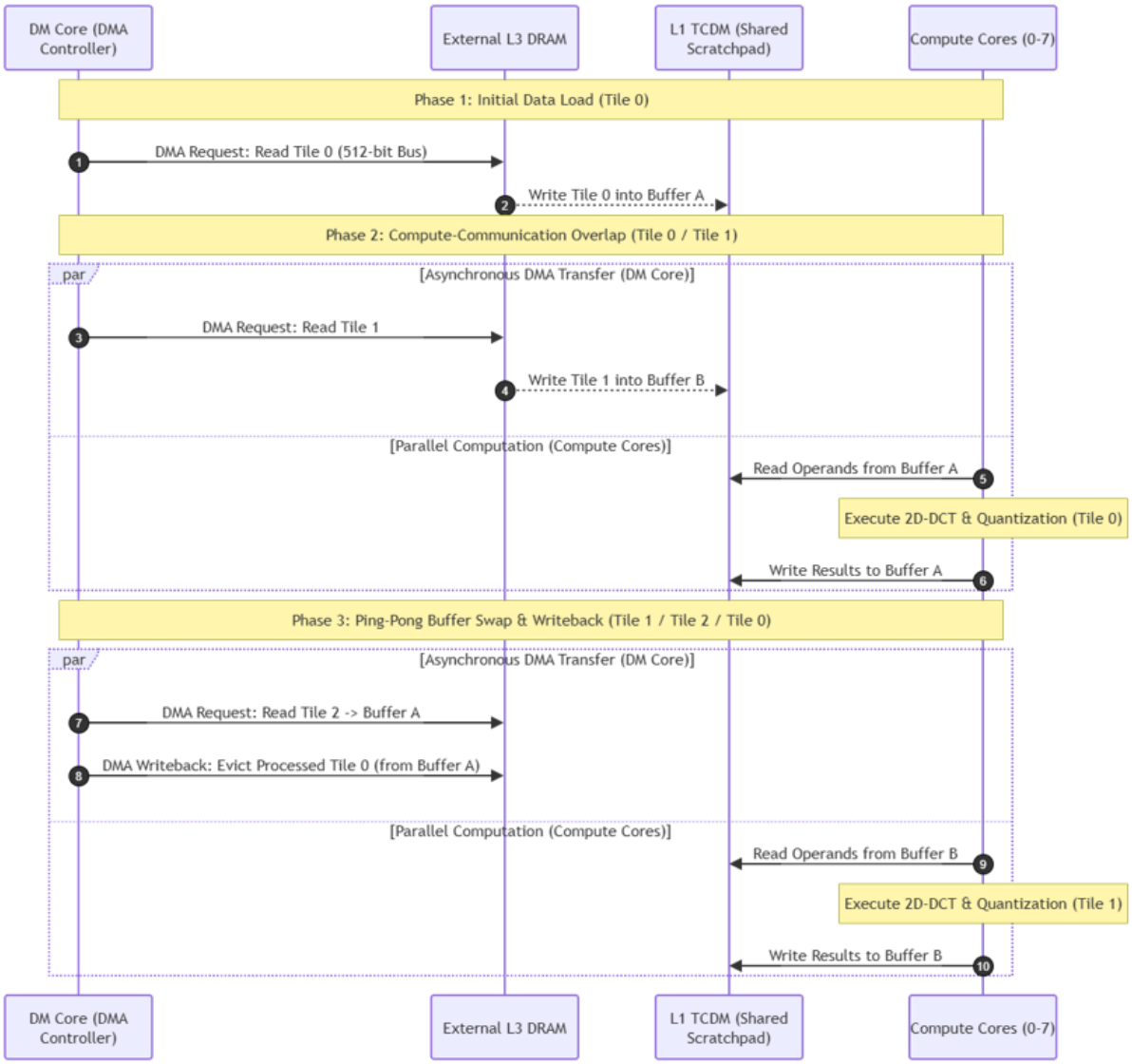


Figure 3.7: Visualization of the movement of Data through 2 Ping-Pong Buffers

By executing this scheme, the total execution time (T_{total}) is governed by the maximum of the computation duration ($T_{compute}$) and the data transfer duration (T_{dma}), **rather than their sum**, as expressed in Equation (3.4):

$$T_{total} = \sum_{i=0}^{N_{tiles}-1} max(T_{compute,i}, T_{dma,i}) \quad (3.4)$$

When the workload is tuned such that computation time matches or exceeds data transfer time ($T_{compute} \geq T_{dma}$), **external memory latency is hidden**, ensuring that the 8 compute cores maintain continuous execution on active operands.

3.2.3 Local Memory Hierarchy

Memory hierarchy design in multi-core edge accelerators presents a classic engineering trade-off: high computational throughput requires ultra-fast operand delivery, yet standard hardware-managed caches introduce non-deterministic latencies, silicon area overhead, and power-hungry tag-comparison logic. The Snitch Cluster resolves this by replacing conventional data caches with a **software-managed, low-latency scratchpad memory architecture** optimized for predictable execution, high-bandwidth streaming, and dense mathematical inner loops.

A) Tightly-Coupled Data Memory (TCDM)

In contrast to conventional high-performance processors that rely on standard Level-1 Data Caches (L1 D-Cache) with complex, hardware-driven *line replacement policies* (such as LRU or Pseudo-LRU) and *cache eviction logic*, the Snitch cluster utilizes a shared scratchpad memory known as the Tightly-Coupled Data Memory (TCDM).

In standard 64-bit reference configurations (such as those in the ETH Zurich / University of Bologna Occamy architecture[2]), the cluster features a TCDM capacity of 128 KiB (size: 128), which functions as the L1 data store for all cores within the cluster [26]. The TCDM is directly connected to both the 8 Worker Compute Cores and the 1 Data Management Core (DM Core) through a high-bandwidth, single-cycle *combinational crossbar interconnect*.

This organization is shown in Figure 3.8. The eight compute cores connect through their 64-bit ports to a low-latency crossbar, which routes each request in a single cycle to one of the 32 TCDM banks (Bank 0 to Bank 31, each 64 bits wide and 4 KiB deep). The DM core does not share these narrow ports: its 512-bit DMA bus accesses eight consecutive banks in parallel, so **bulk transfers do not consume the bandwidth of the compute cores**.

[2] Occamy is a dual-chiplet RISC-V system developed by ETH Zurich and the University of Bologna. It integrates 48 Snitch clusters (432 cores in total), each built around the cluster configuration described in this section [26].

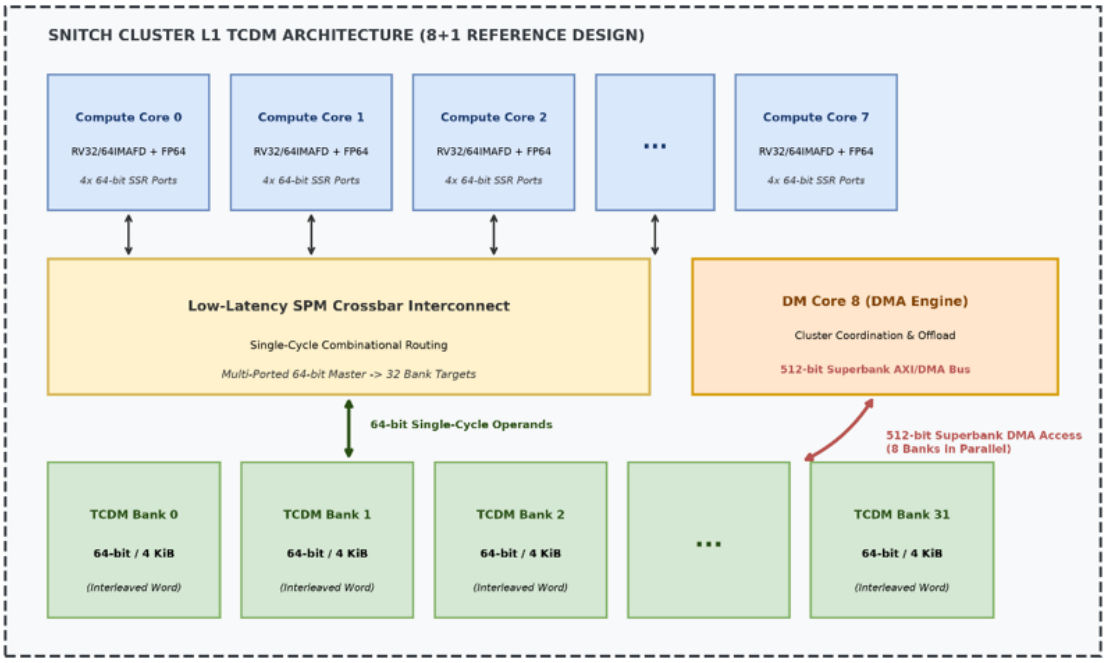


Figure 3.8: Snitch Cluster Memory management Architecture

Key Architectural Characteristics of the TCDM

The TCDM design is defined by the following *key architectural characteristics*:

- **Single-Cycle Access Latency**: Every successful memory transaction (read or write) issued to an uncontended TCDM bank completes in exactly one clock cycle. This zero-wait-state access profile eliminates pipeline stalls during operand fetch phases.
- **Multi-Ported 64-bit Core Streaming**: To saturate the double-precision (FP64) floating-point pipelines without half-word serialization bottlenecks, each worker core connects to the SPM crossbar via 64-bit ports. These ports simultaneously feed Stream Semantic Registers (SSRs) and standard Load/Store Units (LSUs).
- **512-bit Superbank DMA Engine**: The DM Core controls a 512-bit wide AXI/DMA engine[3]. Rather than competing for individual 64-bit crossbar ports, the DMA accesses blocks of 8 consecutive banks in parallel (superbanks, or 64 bytes per cycle) via a wide secondary interconnect, enabling massive bulk data transfers without stalling worker cores.
- **Deterministic Execution Timing**: Because scratchpad memory lacks hardware cache misses, execution timing becomes strictly deterministic. Hardware engineers and

[3] AXI (Advanced eXtensible Interface) is an on-chip bus protocol of the Arm AMBA specification. It supports burst transfers and multiple outstanding transactions, which makes it well suited to DMA engines.

compiler toolchains can precisely calculate algorithm cycle counts, enabling targeted micro-architectural optimizations.

- **Explicit Software Management**: Rather than relying on reactive cache allocation, software explicitly orchestrates data placement via the DM Core's DMA engine. This eliminates tag arrays, status bits, and eviction controllers, converting saved silicon area directly into raw arithmetic execution units.

B) Memory Interleaving and Bank Contention

To support concurrent access requests from up to 9 cores (and their multi-ported SSRs) without creating a bottleneck on a centralized bus, the 128 KiB TCDM is physically partitioned into **32 independent Memory Banks** (banks: 32). In a 64-bit Snitch cluster, each bank has a word width of 64 bits (8 bytes) and an individual capacity of 4 KiB (32 × 4 KiB = 128 KiB).

64-bit Word Interleaving Address Scheme

Addresses are mapped using a *64-bit Word Interleaving* scheme, where consecutive 8-byte (64-bit) memory words are distributed cyclically across the 32 physical banks. The physical Bank Index for any given byte address is calculated using Equation (3.5):

$$Bank\ Index\ = \left(\frac{Address}{8}\right)\ mod\ 32 \tag{3.5}$$

In binary addressing, this calculation corresponds to extracting bits [7:3] of the target byte address (since bits [2:0] address the 8 individual bytes within a 64-bit word). The resulting mapping is:

```
Address 0x0000 → Bank 0      Address 0x0008 → Bank 1
Address 0x0010 → Bank 2      ...
Address 0x00F8 → Bank 31     Address 0x0100 → Bank 0 (Wraparound)
Address 0x0108 → Bank 1
```

Figure 3.9 illustrates how a byte address is decoded. The lowest three bits [2:0] select a byte within the 64-bit word and are not used for routing, while the next five bits [7:3] form the bank index. The crossbar uses this index to send the request to the corresponding bank. As a result, consecutive 8-byte words fall on consecutive banks, and the pattern **wraps back to Bank 0 every 256 bytes**.

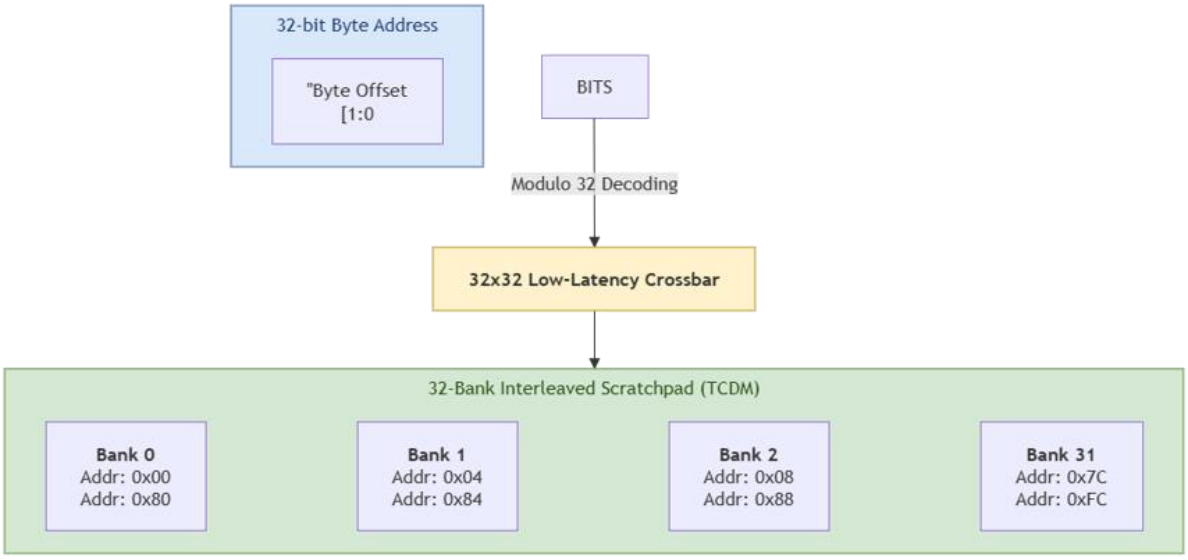


Figure 3.9: Address pointing through TCDM

The Bank Contention Problem

The Snitch cluster achieves maximum throughput when all active core ports access addresses residing in different physical memory banks during the same clock cycle. Under these ideal conditions, the crossbar routes all transactions in parallel, sustaining a **peak memory bandwidth of 32 words (256 bytes) per cycle**.

However, when multiple initiator ports target the same bank simultaneously, an architectural hazard known as *TCDM Bank Contention* (or a Bank Conflict) occurs:

- **Bank Conflict**: A bank conflict arises when two or more cores (or SSR ports) attempt to read or write to different memory addresses that map to the *same physical memory bank* during the exact same clock cycle.
- **Hardware Arbitration and Stalls**: Because each single-ported memory bank can service only one 64-bit transaction per cycle, the crossbar interconnect enforces *serial arbitration*. One requesting port is granted access, while all other conflicting ports are stalled by hardware, waiting for subsequent clock cycles to complete their accesses.

C) Mathematical Modeling of Bank Conflicts and Stride Alignment

To analyze the impact of memory access patterns on multi-core performance, consider a parallel loop executing across N_{cores} cores where each core $i \in \{0, 1, \ldots, N_{cores}-1\}$ accesses a 1D vector with a constant memory access stride S_{stride} (measured in 64-bit words / 8 bytes). The physical memory address accessed by core i at iteration k is given by Equation (3.6):

$$Address_{i,k} = Base_Address + 8 \cdot (k \cdot N_{cores} \cdot S_{stride} + i \cdot S_{stride}) \quad (3.6)$$

Combining Equations (3.5) and (3.6), the corresponding target bank index $B_{i,k}$ for core i is derived as:

$$B_{i,k} = \left(\frac{Base_Address}{8} + k \cdot N_{\text{cores}} \cdot S_{\text{stride}} + i \cdot S_{\text{stride}}\right) mod\ 32 \quad (3.7)$$

Assuming an aligned base address (Base_Address mod 256 = 0), the bank index for core i simplifies to:

$$B_i = (i \cdot S_{\text{stride}})\ mod\ 32 \quad (3.8)$$

Conflict Conditions and Degree of Contention

From Equation (3.8), the number of unique memory banks targeted simultaneously by N_{cores} cores executing with stride S_{stride} is given by Equation (3.9):

$$Unique\ Banks = \frac{32}{gcd(S_{\text{stride}},32)} \quad (3.9)$$

where gcd(S_{stride}, 32) represents the greatest common divisor between the word stride S_{stride} and the total bank count (32). From this relationship, we classify four primary access regimes, summarized in Table 3.4:

Table 3.4: Memory bank contention versus 64-bit word access stride S_{stride}

Access Stride (S_{stride}, in 64-bit words)	**gcd(S_{stride},32)**	**Unique Banks**	**Conflict Degree**	**Hardware Impact**
Unit Stride (S_{stride}=1)	1	32	1 (No Conflicts)	Maximum Throughput: All cores access distinct banks; transactions complete in 1 cycle.
Odd Stride (S_{stride}=3,5,7)	1	32	1 (No Conflicts)	Full Parallelism: Cores map to a permutation of all 32 banks; zero stalls incurred.
Power-of-Two Stride (S_{stride}=2,4,8)	2, 4, 8	16, 8, 4	2x, 4x, 8x Stalls	Severe Contention: Multiple cores map to the same bank, multiplying cycle counts.
Critical Stride (S_{stride}=32)	32	1	N_{cores}-fold Conflict	Worst Case: All N_{cores} cores hit Bank 0 simultaneously; execution is entirely serialized.

This model is applied in Chapter 5 (Section 5.3) to analyze and **resolve the bank conflicts of the multi-core JPEG implementation**.

3.3 Hardware Accelerators in Snitch

The architectural philosophy of Snitch is rooted in the principle of *control-compute decoupling*. In a conventional processing system, the execution of mathematically demanding kernels—such as the 2D Discrete Cosine Transform (2D-DCT) and Quantization—is fundamentally constrained by the classic *Von Neumann bottleneck.*

In a standard in-order execution pipeline, a substantial fraction of the execution time and energy budget is expended not on actual floating-point arithmetic, but on *administrative control-flow overhead*:

- **Instruction Fetch & Decode**: Fetching and decoding every instruction from memory.
- **Pointer Arithmetic**: Address generation and memory pointer updates.
- **Explicit Loads and Stores**: Data load and store instructions (fld, fsd).
- **Loop Control**: Loop-bound checking and branch resolution (addi, bne).

To eliminate this administrative overhead without resorting to rigid, monolithic ASIC designs, Snitch embeds two native, domain-specific hardware accelerators within its compute core microarchitecture: **Stream Semantic Registers (SSR) and Floating-Point Repetition (FREP)**.

3.3.1 Stream Semantic Registers (SSR)

A) The Conventional Memory Access Problem

In a standard RISC-V architecture (RV32IMAFD / RV64IMAFD), executing a vector Fused Multiply-Add (FMA) operation of the form $d_i = a_i \cdot b_i + c_i$ inside an inner loop requires executing at least seven to ten distinct instructions per iteration, as shown in Listing 3.1:

Listing 3.1: Conventional FMA inner loop in standard RISC-V (RV64D).

```
loop:
   fld  ft0, 0(a0)          # 1. Load operand A from L1 TCDM memory
   fld  ft1, 0(a1)          # 2. Load operand B from L1 TCDM memory
   fld  ft2, 0(a2)          # 3. Load operand C (accumulator)
   fmadd.d ft2, ft0, ft1, ft2  # 4. FPU arithmetic: ft2 = ft0 * ft1 + ft2
   fsd  ft2, 0(a2)          # 5. Store result back to memory
   addi a0, a0, 8           # 6. Increment pointer A (Pointer Arithmetic)
   addi a1, a1, 8           # 7. Increment pointer B
   addi a2, a2, 8           # 8. Increment pointer C
   addi t0, t0, -1          # 9. Decrement loop counter
   bne  t0, zero, loop      # 10. Check branch condition
```

As demonstrated in Listing 3.1, **only 1 out of 10 instructions (10%)** performs useful mathematical computation in the FPU. The remaining nine instructions are wasted on data movement and loop management, consuming valuable I-Cache bandwidth, introducing pipeline stalls, and increasing overall energy dissipation.

B) SSR Microarchitecture and Operation (xssr: true)

Stream Semantic Registers (SSR) completely eliminate explicit `fld` / `fsd` instructions and manual pointer arithmetic. The core concept relies on overloading specific floating-point registers in the General Register File, converting them into *virtual streaming portals* [27].

In the reference Snitch implementation, three FPU registers are architecturally reserved as *Stream Semantic Registers*:

- **ft0** (Stream 0 - Read): Every read from ft0 automatically fetches the next word from the first input queue (Input FIFO 0).
- **ft1** (Stream 1 - Read): Every read from ft1 automatically fetches the next word from the second input queue (Input FIFO 1).
- **ft2** (Stream 2 - Write): Every write to ft2 automatically pushes the result into the output queue (Output FIFO) for direct writeback to TCDM memory.

Figure 3.10 shows the data path behind this mechanism. Each stream has its own 4D address generation unit (AGU), which issues requests to the TCDM crossbar without involving the integer core. Data read by SSR 0 and SSR 1 is buffered in input FIFOs and appears in registers ft0 and ft1. Results written to ft2 enter an output FIFO and are written back to memory through SSR 2. The FPU therefore only sees registers, **without any explicit load or store instruction**.

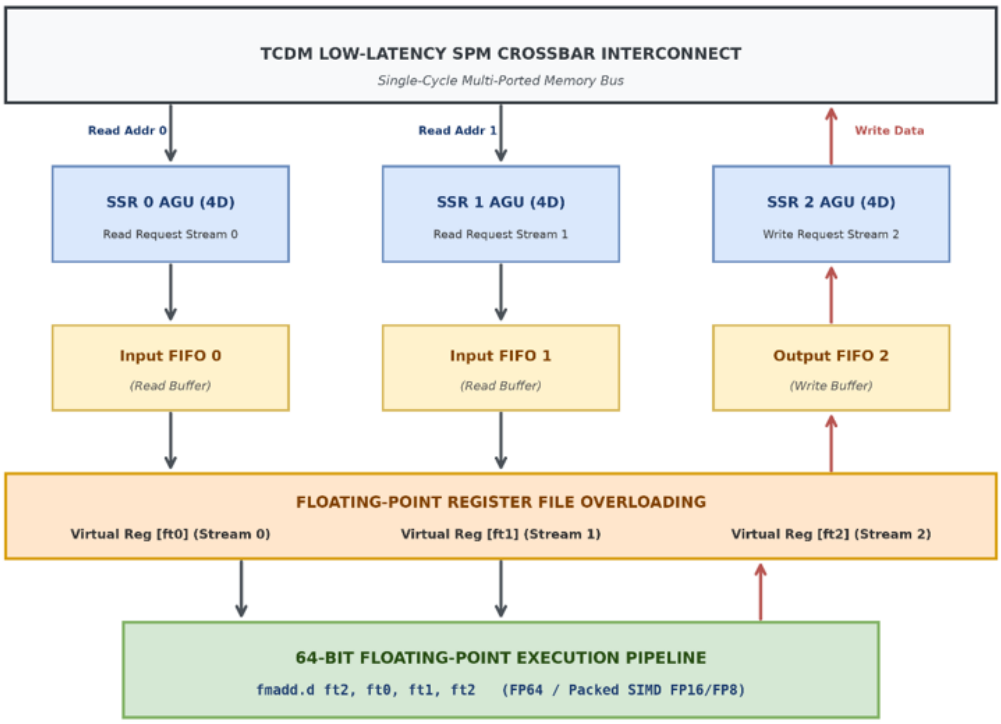


Figure 3.10: Data Movement through FPU SSR without delay.

When the accelerator is enabled (xssr: true), hardware logic intercepts the FPU instruction decode stage and bridges it directly to the TCDM memory bus. A single FPU instruction:

```
fmadd.d ft2, ft0, ft1, ft2
```

is automatically interpreted by the hardware as: *"Pop the next word from Stream 0, pop the next word from Stream 1, multiply them, add the next word from Stream 2, and write the final result back to memory via Stream 2"*. Compared to Listing 3.1, this single instruction replaces the explicit loads, the store and the three pointer increments.

C) Multi-Dimensional Address Generation Units (4D AGUs)

To support complex memory access patterns without intervention from the integer core, each Stream Semantic Register is equipped with an autonomous, programmable Address Generation Unit (AGU), shown as the top row of blocks in Figure 3.10, supporting up to 4 dimensions (1D to 4D Nested Loops) [27]. Later extensions add *indirect, index-driven streams* (ISSRs) [28], [29], which are used in Chapter 4.

The generated memory address Addr for a 4-dimensional nested loop scheme is given by Equation (3.10):

$$Addr(i_0, i_1, i_2, i_3) = Base + \sum_{d=0}^{3} i_d \cdot Stride_d \quad (3.10)$$

The terms of Equation (3.10) are *defined as follows*:

- **Base**: The base starting address of the array in TCDM memory.
- **d** ∈ {0, 1, 2, 3}: The loop dimension level (from the innermost loop d = 0 to the outermost loop d = 3).
- $\mathbf{i_d} \in [0, \text{Bound}_d - 1]$: The current integer index of dimension d.
- **Stride$_d$**: The memory access stride (in bytes) for dimension d.
- **Bound$_d$**: The total iteration count for dimension d.

The AGU hardware automatically updates its internal index counters on every read/write cycle. As soon as index i_0 reaches $\text{Bound}_0 - 1$, it wraps around to zero instantaneously while incrementing index i_1, repeating hierarchically. This capability allows the hardware to *autonomously execute*:

- **Sub-matrix extraction**: For example, reading an 8x8 pixel tile from a larger 512x512 image array.
- **On-the-fly Matrix Transposition**: Configuring Stride_0 equal to a column stride instead of a row stride, eliminating explicit transpose instructions in memory.
- **Strided Sampling**: Accessing elements at arbitrary strides (such as 4:2:0 Chroma Subsampling) without requiring branch or jump instructions.

3.3.2 Floating-Point Repetition (FREP)

A) Eliminating Control-Flow Overhead (xfrep: true)

While the SSR accelerator resolves the data load and store bottleneck, the processor core is still penalized by loop control instructions (counter decrements and branch checks). To eliminate this control overhead entirely, Snitch introduces the Floating-Point Repetition (FREP) hardware extension (xfrep: true) [1].

The frep instruction is a *dedicated hardware loop instruction* taking *two primary arguments*:

- **Iteration Count** (N_{iter}): The number of loop iterations (up to $2^{11} - 1$).
- **Block Size** (I_{blk}): The number of FPU instructions (typically $I_{blk} = 1$ to $I_{blk} = 4$) immediately following the frep instruction.

Listing 3.2 shows the syntax, applied to the FMA loop of Listing 3.1:

Listing 3.2: The FMA loop of Listing 3.1 expressed with FREP and SSRs.

```
# Syntax: frep.o [iteration_count], [block_instruction_count], [stalling_rates]
frep.o  a0, 1, 0, 0
fmadd.d ft2, ft0, ft1, ft2
```

When the integer pipeline decodes a frep instruction, it locks the subsequent FPU instruction (fmadd.d) into the Instruction Decode Buffer and re-issues it continuously $N_{iter}+1$ times directly to the FPU pipeline, **requiring zero subsequent instruction fetches** from the L1 iCache. The ten-instruction loop body of Listing 3.1 is thus reduced to a single instruction that is fetched only once.

B) The SSR + FREP Architectural Synergy: Streaming Engine Paradigm

The combined operation of SSR and FREP fundamentally transforms the compute core microarchitecture: moving away from a conventional Von Neumann processor and into an **Autonomous Streaming Engine**.

Integrated Streaming Pipeline Workflow

The resulting execution follows an *integrated streaming pipeline workflow*:

- **Stream Configuration**: The programmer configures the Base, Bound, and Stride registers of the 4D AGUs for the required input/output data streams (Equation (3.10)).
- **Loop Issue**: The integer core executes a single frep instruction followed by a target floating-point instruction (e.g., fmadd.d ft2, ft0, ft1, ft2).
- **Integer Core Release**: The RISC-V integer pipeline enters an idle state (or is freed to execute auxiliary control tasks [30]).
- **Saturated Execution**: The FPU executes continuously at 1 FMA per clock cycle, automatically popping operands from SSR 0/1 queues and pushing results to SSR 2, achieving 100% FPU Pipeline Saturation.

Table 3.5 summarizes the overhead of a conventional RISC-V inner loop compared to the accelerated SSR + FREP streaming pipeline.

Table 3.5: Performance comparison and overhead between a conventional RISC-V inner loop and the accelerated SSR + FREP Snitch streaming pipeline

Architectural Metric	Standard RISC-V (RV64D)	Snitch with SSR + FREP	Improvement / Impact
Instructions per Loop Iteration	~10 instructions	0 instructions (Zero loop body overhead)	10x code footprint reduction
I-Cache Fetch Overhead (I-Fetch)	100% (every cycle)	0% (Instruction locked in decode buffer)	Drastic instruction energy savings
Pointer Arithmetic Overhead	~3–4 addi instructions	0 instructions (Autonomous 4D AGUs)	Releases integer ALU

Architectural Metric	Standard RISC-V (RV64D)	Snitch with SSR + FREP	Improvement / Impact
FPU Pipeline Saturation	10% – 25%	100% (Continuous 1 FMA / cycle)	4x – 10x effective GFLOPS
Energy Dissipation / FLOP	High (Decode/Fetch dominated)	Minimal (Confined to FPU/TCDM operations)	Optimal Energy Efficiency

Figure 3.11 illustrates these differences with a concrete example: a 64-element FMA loop. On a standard in-order RV64D core, the loop takes 448 cycles, of which only 64 perform useful floating-point work. The rest is spent on explicit loads and stores and on loop control, pointer arithmetic and branching, giving an FPU utilization of just 14.3%. With SSR and FREP enabled, the same loop completes in 65 cycles, one FMA per cycle plus a single cycle of setup, reaching 98.5% FPU utilization. This is **roughly a 6.9× reduction in execution time**.

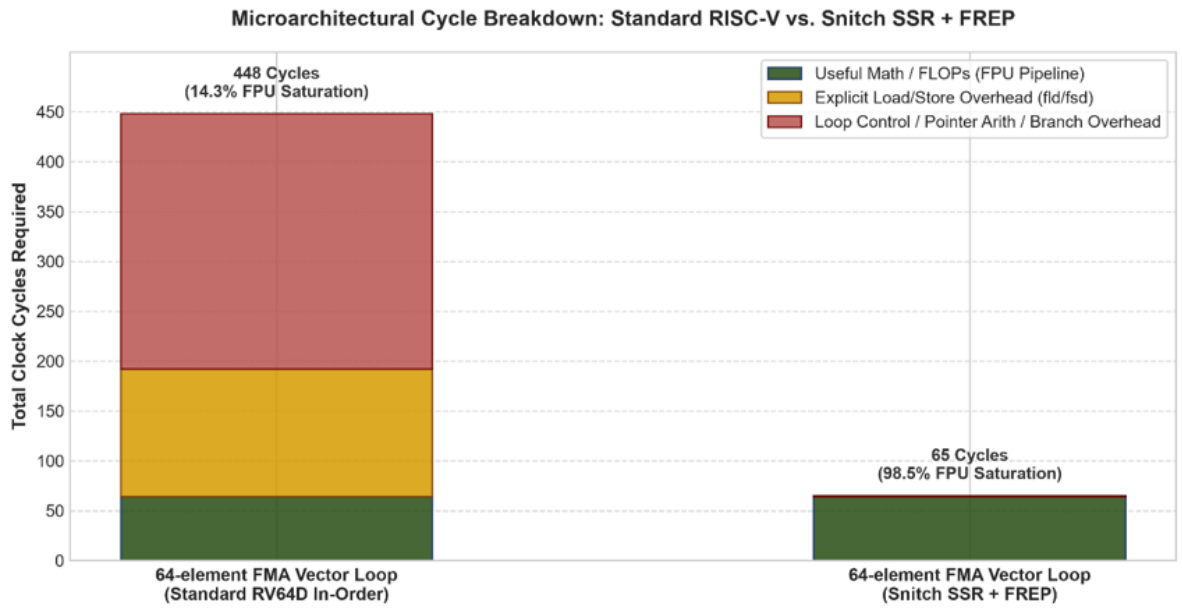


Figure 3.11: Visualization of computation gap between Standard Riscv and Optimized Computation

These two accelerators form the basis of all optimized kernels of the JPEG pipeline presented in Chapter 4.

4 Single-Core Implementation

This chapter presents the implementation and optimisation of the JPEG compression pipeline on a single compute core of the Snitch cluster and establishes the single-core reference on which the multi-core design of Chapter 5 is built. Section 4.1 describes the Initial Design, a functional scalar C implementation of the six pipeline stages. Sections 4.2 to 4.6 then present, stage by stage, the detailed hardware-oriented optimisations: for Color Space Conversion (Section 4.2), Chroma Subsampling (Section 4.3), the 2D-DCT (Section 4.4), Quantization (Section 4.5) and Zig-Zag Reordering with Differential Encoding (Section 4.6), a double-precision (FP64) and a packed SIMD half-precision (FP16) variant are implemented, compared architecturally and evaluated in clock cycles. Finally, Section 4.7 consolidates the per-stage measurements, analyses the overall trade-offs and **selects the FP64 implementation as the single-core baseline** for the multi-core expansion. Section 4.8 then revisits the Zig-Zag stage of the FP16 variant, traces its slowdown to the method used to unpack the packed FP16 values, and presents a **hybrid FP16/FP64 pipeline** that removes it.

4.1 Initial Design

To develop the compression algorithm, a complete and functional implementation was first written entirely in C. This version is referred to throughout this chapter as the **Initial Design** (labelled "Naive" in the measurements). Available *open-source code* [31] was used as a starting point and was heavily modified to meet the constraints of block-based data processing. The pipeline processes Minimum Coded Units (MCUs) of 16×16 pixels and is divided into six execution stages, which are described in Sections 4.1.1 to 4.1.6.

4.1.1 Color Space Conversion: Look-Up-Table Type Conversion and the Luminance Dependency

In the Initial Design, the raw 8-bit color components are not converted to floating point through arithmetic. Instead, each byte is used as an index into a pre-computed Look-Up Table (LUT) that directly returns the corresponding double-precision (FP64) value. This choice deliberately moves the cost of the type conversion away from the Arithmetic Logic Unit (ALU) and **replaces it with a single memory read per component**.

The conversion itself follows Equations (4.1) to (4.3). Equation (4.1) computes the luminance as a weighted sum of the three color components, while Equations (4.2) and (4.3) compute the two chrominance components as scaled color differences.

$$Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B \tag{4.1}$$

$$C_b = 0.564 \cdot (B - Y) \tag{4.2}$$

$$C_r = 0.713 \cdot (R - Y) \tag{4.3}$$

Equations (4.2) and (4.3) reveal an **inherent data dependency at register level**: Cb and Cr cannot be computed in parallel with Y, because the luminance value produced by Equation (4.1) is a required operand of both floating-point subtractions. The +128 offset of the ITU-R BT.601 definition (Section 2.2.3) is not part of Equations (4.2) and (4.3); how and where it is applied is explained in Section 4.3.2.

4.1.2 Chroma Subsampling: 2×2 Averaging through Multiplication Instead of Division

The 4:2:0 chroma subsampling stage reduces the spatial resolution of the chrominance channels by condensing every 2×2 group of pixels into a single value. As expressed by Equation (4.4), the subsampled value is the average of the four samples of the group.

$$\overline{C_{\text{chr}}}(i,j) = 0.25 \cdot \sum_{m=0}^{1} \sum_{n=0}^{1} C_{\text{chr}}(2i+m, 2j+n) \tag{4.4}$$

The average is computed by multiplying the sum by the constant 0.25 instead of dividing it by 4.0. This *micro-optimization* avoids the long latency of the floating-point division instruction and keeps the throughput of the Floating-Point Unit (FPU) high.

4.1.3 2D-DCT: Level Shift and Separable Row–Column Multiply-Accumulate Passes

The 2D-DCT is a strictly *compute-bound* stage. Before the transform, every sample is centred around zero by the DC level shift of Equation (4.5). In the Initial Design this subtraction is not a separate pass; it is integrated into the copy of the block into the working buffer.

$$f_s(x,y) = f(x,y) - 128 \tag{4.5}$$

The transform exploits the separability property (Section 2.3.3) and is written in the matrix form of Equation (4.6), where T is the constant 8×8 DCT matrix.

$$F = T \cdot f_s \cdot T^T \tag{4.6}$$

Equation (4.6) is evaluated as two successive series of one-dimensional Multiply-Accumulate (MAC) operations: a horizontal pass over the rows, followed by a vertical pass over the columns. No output element of the resulting 8×8 block depends on any other output element, so every coefficient can be computed independently. This absence of data dependencies makes the 2D-DCT the **ideal candidate for vectorization (SIMD)** and for the hardware streaming techniques applied in Section 4.4, where multiple FPU MACs can be executed concurrently.

4.1.4 Quantization: Reciprocal Multiplication, Rounding and Integer Conversion

The quantization stage applies the element-wise transformation of Equation (4.7) to the 64 coefficients of each block, where $R_{rec}(u,v)$ is the reciprocal of the quantization step Q(u,v).

$$F_Q(u,v) = \text{round}(F(u,v) \cdot R_{\text{rec}}(u,v)), \quad R_{\text{rec}}(u,v) = \frac{1}{Q(u,v)} \tag{4.7}$$

Inside a 64-iteration loop, every coefficient passes through *the following three operations*:

- **Multiplication:** The coefficient is multiplied by the corresponding entry of the reciprocal matrix, which stores the inverted quantization coefficients. The slow division is therefore replaced by a much faster floating-point multiplication.
- **Rounding:** The double-precision product is rounded to the nearest integer by the nearbyint() function.
- **Type Casting:** The rounded value is converted from a 64-bit floating-point number (double) to a 16-bit signed integer (int16_t) and is stored in the output array.

4.1.5 Zig-Zag Reordering and DC Differential Encoding through a Precomputed Index Map

This stage processes the blocks sequentially (64 values per block) and performs *two restructuring and arithmetic operations*:

- **Zig-Zag Reordering:** The 64 coefficients are rearranged from their 2D layout into a 1D array according to a predefined index table (zigzag_map). The traversal places the significant low-frequency coefficients at the beginning of the array and gathers the

trailing high-frequency zeros at its end, which favours the subsequent run-length encoding.

- **Differential Encoding (DC DPCM):** The first coefficient of the reordered block (the DC coefficient) is replaced by its difference from the DC coefficient of the previous block (current_dc − *prev_dc), as defined by Equation (4.8). The current absolute DC value is then stored to serve as the reference for the next block.

In Equation (4.8), DCi denotes the DC coefficient of block i and DCi−1 that of the block processed immediately before it.

$$\mathrm{DIFF}_i = \mathrm{DC}_i - \mathrm{DC}_{i-1} \tag{4.8}$$

4.1.6 Entropy Coding: A Serial, Integer-Only Stage Kept in Scalar C

The final stage compresses the reordered blocks with Run-Length Encoding (RLE) and Huffman coding. Its execution profile differs fundamentally from that of the preceding compute-bound stages, such as the 2D-DCT and Quantization. Variable-length coding and bitstream generation are **inherently serial**, rely almost exclusively on integer bit-level operations (shifts and masks), and exhibit *irregular control flow* with frequent branches. Since the FPU cannot accelerate integer bit manipulation, FPU or SIMD optimizations bring no benefit to this stage.

For this reason, the stage was implemented in scalar C, and this implementation was kept unchanged in all single-core variants of this chapter. It therefore defines the **serial boundary of the compression pipeline**. The only later change, an instruction-scheduling (prefetching) improvement applied in the multi-core version, is discussed in Section 5.4.1.

4.2 Color Space Conversion (YCbCr) and Data Precision Exploration (Double vs. Packed FP16)

Sections 4.2 to 4.6 present the detailed optimisations applied to each stage of the Initial Design. For every stage, an FP64 and a Packed SIMD FP16 variant are developed and compared, and each section follows the same structure: the FP64 implementation, the FP16 implementation, a comparative analysis, and a cycle-count evaluation. Before the individual stages are examined, Table 4.1 gives an overview of the optimisation techniques used in every stage for both precisions, together with the main limitation met by the FP16 variant; it serves as a *roadmap* for the sections that follow.

Table 4.1: Overview of the optimisation techniques applied per pipeline stage (FP64 vs. Packed SIMD FP16)

Stage	FP64 optimisations	FP16 optimisations	Main FP16 limitation	Section
Color Space Conversion	ISSR + LUT zero-cost type cast; 4D SSR tiled output; FREP; 2 pixels per iteration	ISSR + LUT; 4-lane packing (vfcpka/vfcpkb); vector MACs (vfmac.r.h); FREP; 4 pixels per iteration	Packing overhead and register pressure	4.2
Chroma Subsampling 4:2:0	4D SSR read, two SSR writes; FREP; ×0.25 and +128 fused in one fmadd.d	Two row streams; vfadd.h and vfsum.h	No sub-64-bit SSR writes: ALU stores (sw), no FREP	4.3
2D-DCT	Zero-cost transposition with 4D SSR strides; nested FREP; 8 accumulators	Vector multiplications (vfmul.h) with lane reduction	Pass 1: ALU stores (sh); Pass 2: FREP limit, branch loop	4.4
Quantization	Reciprocal multiplication; magic-number rounding; three SSR streams; FREP	Same technique on 4 packed lanes; FREP	None (element-wise operation)	4.5
Zig-Zag & DPCM	ISSR zig-zag gather; copift F2I hardware queue; FREP conversion	Scalar fallback (hybrid: flh widening to FP64, Section 4.8)	No ISSR, NaN-boxing, FPU–ALU ping-pong (removed by the hybrid pipeline)	4.6, 4.8
Huffman	Scalar C (Initial Design)	Scalar C (Initial Design)	Serial, integer-only stage	4.1.6

The first stage of the optimized pipeline converts the raw pixels from the RGB color space to YCbCr. Moving from the serial C implementation to an implementation that fully exploits the Snitch cluster requires a fundamental restructuring of data management. The goal of this stage is to **decouple the Arithmetic Logic Unit (ALU) from address generation**, so that the Floating-Point Unit (FPU) can operate without interruption.

To evaluate the best use of the available resources, this stage was implemented in two precision variants: one using double-precision floating-point (FP64) and one using half-precision vector instructions (Packed SIMD FP16). Both variants share a common set of micro-architectural optimizations, described in Section 4.2.1, before they diverge in their register-management strategies (Sections 4.2.2 and 4.2.3).

4.2.1 Common Architectural Optimizations (ISSR & 4D Tiling)

Three hardware mechanisms are shared by both variants: the Indirect SSR for type conversion, the four-dimensional SSR address generator for the tiled output, and the FREP hardware loop.

A) Zero-Cost Type Casting via ISSR

The most critical optimization concerns the way input data are fed to the processor. The raw image data occupy 1 byte per channel (uint8_t), whereas the FPU requires floating-point operands. In a conventional processor, a type cast inside the loop would consume a large number of clock cycles. To eliminate this cost, the implementation leverages the Indirect Stream Semantic Register (ISSR) hardware extension in *two complementary steps*:

- **LUT Placement:** A pre-computed Look-Up Table (LUT) of 256 entries is placed in the local TCDM memory. Entry k holds the value k already converted to floating-point format.
- **Index-Based Fetching:** Through the snrt_issr_read function, the DM0 stream is configured to read the serial 8-bit data. Instead of sending the bytes to the FPU, the hardware uses them as indices (offsets) relative to the base address of the LUT.

With this strategy, the memory subsystem fetches the ready floating-point value and forwards it directly to the FPU. The operation therefore constitutes a *hardware-accelerated zero-cost type cast*: no integer or floating-point instruction is spent on the conversion.

B) Generating 4D Layouts without Pointer Arithmetic (Tiling)

The luminance (Y) must be produced as independent 8×8 blocks (*tiled layout*) for the subsequent DCT stage, although the pixels of an MCU arrive in *raster order*. The snrt_ssr_loop_4d function programs the DM1 output stream to follow a four-dimensional address pattern. In the FP64 variant (Listing 4.1), the bounds are (8, 2, 8, 2) and the strides (8, 512, 64, 1024) bytes, so the address of each written Y sample is generated as in Equation (4.9).

$$\text{Addr} = \text{Base} + 8 \cdot i_0 + 512 \cdot i_1 + 64 \cdot i_2 + 1024 \cdot i_3 \tag{4.9}$$

In Equation (4.9), i_0 walks over the 8 pixels of a block row, i_1 jumps from the left to the right 8×8 block (one block occupies 64 × 8 = 512 bytes), i_2 moves to the next row inside the block, and i_3 jumps to the lower pair of blocks. As the FPU produces results, the stream

automatically applies these strides and arranges the pixels **without the ALU executing a single pointer-arithmetic instruction**.

C) Zero-Overhead Loops with FREP

In both variants the computational kernel is wrapped in the frep.o (Floating-point Repetition) instruction, which re-issues the enclosed block of FPU instructions in hardware for the requested number of iterations. The loop counter, the branch and the instruction fetch are thus removed from the critical path (**zero loop overhead**), and the integer core is released while the FPU works.

4.2.2 Double Precision Implementation (FP64)

The first approach uses double (64-bit) variables. The code processes two pixels per iteration (*loop unrolling*), and since every value occupies an entire 64-bit register, the data flow is straightforward: the multiply-accumulate operations (fmadd.d) execute in parallel with the automatic SSR reads and writes, with no need to rearrange data inside the registers.

Listing 4.1 shows the complete FP64 kernel. Like every listing of this chapter, it is divided into numbered blocks, and the right-hand column summarizes each block. The first part (blocks 1–2) configures the three streams (the ISSR on DM0 for the LUT fetch, the 4D write pattern of Equation (4.9) on DM1 for Y, and a 1D write on DM2 for the interleaved Cb/Cr values); the second part (block 3) is the 22-instruction FREP body that computes Equations (4.1) to (4.3) for two pixels per iteration.

Listing 4.1: *YCbCr implementation in Assembly using FP64 precision (Double).*

1	**void** opt_YCBCR_conversion(**uint8_t** *input_idx,	**1.** Signature; the five YCbCr coefficients are pinned in FPU registers ft3–ft7.
2	**double** *out_Y,	
3	**double** *out_CbCr,	
4	**double** *lut,	
5	**int** total_pixels) {	
6	**register double** r_c_r **asm**("ft3") = 0.299;	
7	**register double** r_c_g **asm**("ft4") = 0.587;	
8	**register double** r_c_b **asm**("ft5") = 0.114;	
9	**register double** r_c_cb **asm**("ft6") = 0.564;	
10	**register double** r_c_cr **asm**("ft7") = 0.713;	
11		**2.** Stream setup: the ISSR on DM0 reads the RGB bytes through the 256-entry LUT (zero-cost cast); DM1 writes Y in 8×8-tiled 4D order (Eq. 4.9); DM2 writes Cb/Cr.
12	snrt_issr_read(SNRT_SSR_DM0, lut, input_idx, *// DM0: RGB → FP64 (LUT)*	
13	total_pixels * 3, SNRT_SSR_IDXSIZE_U8);	
14	snrt_ssr_loop_4d(SNRT_SSR_DM1, 8, 2, 8, 2, 8, *// DM1: Y, tiled 4D*	
15	8 * 8 * 8, 8 * 8, 1024);	
16	snrt_ssr_write(SNRT_SSR_DM1, SNRT_SSR_4D, out_Y);	
17	snrt_ssr_loop_1d(SNRT_SSR_DM2, total_pixels * 2, **sizeof**(**double**));	
18	snrt_ssr_write(SNRT_SSR_DM2, SNRT_SSR_1D, out_CbCr); *// DM2: Cb/Cr*	
19	snrt_ssr_enable();	
20		**3.** 22-instruction FREP body, two pixels per iteration: Y (Eq. 4.1), then B−Y, R−Y and the scaled Cb, Cr (Eq. 4.2–4.3).
21	**asm volatile**(	
22	"frep.o %[n], 22, 0, 0 \n" *// 22 instr./iteration*	
23	"fmv.d ft8, ft0 \n" *// pop R,G,B of 2 pixels*	
24	"fmv.d ft9, ft0 \n"	
25	"fmv.d ft10, ft0 \n"	

```
26          "fmv.d ft11, ft0                \n"
27          "fmv.d f8,   ft0                \n"
28          "fmv.d f9,   ft0                \n"
29          "fmul.d f10, %[c_r], ft8        \n"  // Y = 0.299R + ...
30          "fmul.d f11, %[c_r], ft11       \n"
31          "fmadd.d f10, %[c_g], ft9,  f10  \n"
32          "fmadd.d f11, %[c_g], f8,   f11  \n"
33          "fmadd.d f10, %[c_b], ft10, f10  \n"
34          "fmadd.d f11, %[c_b], f9,   f11  \n"
35          "fmv.d ft1, f10                 \n"  // push Y (DM1)
36          "fmv.d ft1, f11                 \n"
37          "fsub.d ft10, ft10, f10         \n"  // B−Y, R−Y
38          "fsub.d f9,   f9,   f11         \n"
39          "fsub.d ft8,  ft8,  f10         \n"
40          "fsub.d ft11, ft11, f11         \n"
41          "fmul.d ft2, %[c_cb], ft10      \n"  // push Cb, Cr (DM2)
42          "fmul.d ft2, %[c_cr], ft8       \n"
43          "fmul.d ft2, %[c_cb], f9        \n"
44          "fmul.d ft2, %[c_cr], ft11      \n"
45          :                                                          4. Operands: iteration count,
46          : [n]    "r" (total_pixels / 2 - 1),                       coefficient registers and
47            [c_r]  "f" (r_c_r),  [c_g] "f" (r_c_g), [c_b] "f" (r_c_b),  clobbered registers.
48            [c_cb] "f" (r_c_cb), [c_cr] "f" (r_c_cr)
49          : "ft0", "ft1", "ft2", "ft8", "ft9", "ft10", "ft11",
50            "f8", "f9", "f10", "f11", "memory"
51      );
52      snrt_fpu_fence();                                              5. Wait for the FPU and release
53      snrt_ssr_disable();                                            the streams.
54  }
```

4.2.3 Vectorized Implementation (Packed SIMD FP16)

Aiming to **double the theoretical throughput**, an alternative implementation was developed that uses half-precision vector instructions (Packed SIMD FP16). In this strategy, four 16-bit values are packed into one 64-bit register, which allows the architecture to execute vector MACs (vfmac.r.h) on several pixels simultaneously. Theoretically, the FREP loop now processes four pixels per iteration (total_pixels / 4 − 1).

Listing 4.2 presents the FP16 kernel. Before the loop, the five coefficients are converted to half precision (fcvt.h.s, block 2); the Cb and Cr coefficients (0.141 and 0.17825) are the FP64 values 0.564 and 0.713 divided by four, because the ×0.25 averaging factor of the subsequent subsampling stage is folded into them (Section 4.3.3). Inside the 20-instruction FREP body (block 4), the pairs vfcpka.h.d/vfcpkb.h.d pack the samples into the vector lanes before the vector arithmetic is executed.

Listing 4.2: *YCbCr implementation using Packed SIMD FP16, vector instructions, and sub-indexing.*

```
1   void opt_YCBCR_conversion(uint8_t *input_idx,                      1. Signature; Cb/Cr coefficients
2                             float16_t *out_Y,                        are pre-scaled by 0.25 for the
3                             float16_t *out_CbCr,                     subsampling stage.
4                             double *lut,
5                             int total_pixels) {
6       float c_r_f  = 0.299f;  float c_g_f  = 0.587f;  float c_b_f  = 0.114f;
7       float c_cb_f = 0.141f;  float c_cr_f = 0.17825f;
8                                                                      2. Coefficients converted to FP16
9       // Load and convert constants to Half-Precision (FP16)         into registers f3–f7.
10      asm volatile(
11          "fcvt.h.s f3, %0 \n" "fcvt.h.s f4, %1 \n" "fcvt.h.s f5, %2 \n"
```

```
        "fcvt.h.s f6, %3 \n" "fcvt.h.s f7, %4 \n"
        :: "f"(c_r_f), "f"(c_g_f), "f"(c_b_f), "f"(c_cb_f), "f"(c_cr_f)
        : "f3","f4","f5","f6","f7"
    );

    snrt_issr_read(SNRT_SSR_DM0, lut, input_idx,  // DM0: RGB via LUT
                   total_pixels * 3, SNRT_SSR_IDXSIZE_U8);
    snrt_ssr_loop_4d(SNRT_SSR_DM1, 2, 2, 8, 2, 8,  // DM1: Y (FP16)
                     8*2*8, 8*2, 8*8*2*2);
    snrt_ssr_write(SNRT_SSR_DM1, SNRT_SSR_4D, out_Y);
    snrt_ssr_loop_1d(SNRT_SSR_DM2, total_pixels / 2, sizeof(uint64_t));
    snrt_ssr_write(SNRT_SSR_DM2, SNRT_SSR_1D, out_CbCr);  // DM2: Cb/Cr
    snrt_ssr_enable();

    asm volatile(
        "frep.o %[n], 20, 0, 0 \n"  // 20 instr./iteration
        "fmv.d f8, f0 \n fmv.d f9, f0 \n fmv.d f10, f0 \n"  // pop samples
        "vfcpka.h.d f8, f8, f0 \n vfcpka.h.d f9, f9, f0 \n vfcpka.h.d f10, f10, f0 \n"
        "fmv.d f14, f0 \n fmv.d f15, f0 \n fmv.d f16, f0 \n"
        "vfcpkb.h.d f8, f14, f0 \n vfcpkb.h.d f9, f15, f0 \n vfcpkb.h.d f10, f16, f0 \n"
        "vfmul.r.h f11, f8, f3 \n vfmac.r.h f11, f9, f4 \n vfmac.r.h f11, f10, f5 \n"
        "vfsub.h f10, f10, f11 \n vfsub.h f8, f8, f11 \n"
        "fmv.d f1, f11 \n"  // push Y
        "vfmul.r.h f2, f10, f6 \n vfmul.r.h f2, f8, f7 \n"  // push Cb, Cr
        :: [n] "r"(total_pixels / 4 - 1)
        : "f0","f1","f2","f3","f4","f5","f6","f7","f8","f9","f10","f11",
          "f14","f15","f16","memory"
    );
    snrt_fpu_fence();
    snrt_ssr_disable();
}
```

3. Stream setup as in Listing 4.1, with FP16 output layouts.

4. 20-instruction FREP body, four pixels per iteration: lane packing (vfcpka/vfcpkb), then vector MACs.

5. Operands and clobbered registers.

6. Wait for the FPU and release the streams.

4.2.4 Comparative Analysis: Packing Overhead and Register Complexity

Despite the mathematical superiority offered by Packed SIMD vectorization, its practical application in this stage introduced considerable architectural complexity. The comparison of the two approaches revealed **two fundamental bottlenecks** that reduced the theoretical advantage of FP16.

A) Configuration Cost (Packing Overhead)

Because the data arrive sequentially from the ISSR, they are not aligned by default for SIMD operations. In the FP64 implementation, the data are forwarded immediately to the fmadd.d instructions. In the FP16 implementation, in contrast, the processor must spend clock cycles on data formatting (*sub-indexing*). As shown in Listing 4.2, move instructions (fmv.d f8, f0) followed by the specialized packing instructions vfcpka.h.d and vfcpkb.h.d are required to merge several independent values into the same register. This overhead **directly lengthens every iteration of the frep.o loop**.

B) Register Interleaving Complexity and Register Pressure

The packing of color elements imposed strict dependencies between the RGB channels of different pixels. The Snitch hardware executes SIMD operations lane by lane. To multiply the

red channel (R) of the first pixel simultaneously with the R of the second pixel, the programmer must perform a dense "*register interleaving*" that aligns the lanes inside the input registers. The need to keep intermediate states for sub-indexing forced the FP16 implementation to use **15 distinct floating-point registers** (f0–f11 and f14–f16, Listing 4.2).

4.2.5 Cycle Count Analysis and Performance Evaluation

To quantify the real-world impact of these architectural decisions, the execution cycles of the YCbCr conversion stage were measured for the three implementations. The speedup S_{speedup} of an implementation is defined in Equation (4.10) as the ratio of the cycle count of a reference implementation to that of the optimized one.

$$S_{\text{speedup}} = \frac{C_{\text{ref}}}{C_{\text{opt}}} \tag{4.10}$$

Table 4.2 lists the measured cycles together with the speedups of Equation (4.10) relative to the Initial Design and to the FP64 implementation.

Table 4.2: Cycle count and speedup comparison of the YCbCr conversion stage

Implementation	Cycles	Speedup vs. Initial Design	Speedup vs. FP64
Initial Design (Naive)	14,503.00	1.00×	–
Double FP64	4,039.50	3.59×	1.00×
Vectorized OPT FP16	2,103.00	6.90×	1.92×

Despite the packing overhead and the register-interleaving complexity outlined in Section 4.2.4, the localized theoretical advantage of the Packed SIMD FP16 approach holds for this computational kernel. By processing four pixels per iteration, the OPT FP16 implementation reduces the YCbCr conversion latency to **2,103.00 cycles**.

As Table 4.2 shows, this is an approximately **1.92× speedup over the Double FP64 implementation** (4,039.50 cycles) and a 6.90× speedup over the Initial Design (14,503.00 cycles). For straightforward, compute-bound kernels such as color space conversion, half-precision vector operations therefore double the throughput and absorb the initial sub-indexing and data-alignment costs.

4.3 Chroma Subsampling (4:2:0)

The next stage of the algorithm performs the spatial reduction of the chrominance information (Cb and Cr). The 4:2:0 subsampling scans the data in blocks of 2×2 pixels, calculates their

average (Equation (4.4)), and stores the results in new, smaller arrays. As in the previous stage, the implementation was explored at two precision levels (FP64 and Packed FP16), this time highlighting the constraints that the hardware imposes on the memory communication subsystem.

4.3.1 Hardware Constraints: The 3 Data Mover Limit

A fundamental architectural constraint of the Snitch cluster is the limited number of Data Movers (DM) in the Stream Semantic Register (SSR) subsystem. Each core features **exactly three SSR channels** (DM0, DM1 and DM2). This strict limit dictates how concurrent memory reads and writes can be configured and forces the programmer to distribute these three resources carefully between the input and output arrays.

4.3.2 Double Precision Implementation (FP64)

In the double-precision implementation, the three available SSR resources are fully utilized, *with the following perfect mapping*:

- **DM0 (Input Stream):** Configured with a 4D loop that reads the interleaved Cb/Cr data from memory and automatically extracts the 2×2 blocks.
- **DM1 and DM2 (Output Streams):** Configured as 1D loops that write the final, separated values to the out_cb_f and out_cr_f output arrays, respectively.

This allocation enables the use of the frep.o instruction (zero loop overhead). Within the loop, the four values of each block are added with fadd.d instructions, and the division by four is replaced by a single fused multiply-add (fmadd.d). As expressed by Equation (4.11), this instruction multiplies the sum by the constant 0.25 and simultaneously adds the chroma offset of 128.0, directly producing the final FP64 result.

$$\overline{C_{\text{chr}}} = 0.25 \cdot (c_{00} + c_{01} + c_{10} + c_{11}) + 128 \tag{4.11}$$

Listing 4.3 presents the FP64 kernel: in block 2, DM0 is programmed with the 4D read pattern, DM1 and DM2 with the two 1D write patterns, and the 16-instruction FREP body (block 3) processes one Cb and one Cr block per iteration, ending with the two fmadd.d instructions of Equation (4.11).

Listing 4.3: *Chroma Subsampling implementation using FP64, fully utilizing the 3 available SSR streams (1 Read, 2 Writes).*

1	**void** opt_chroma_420_2d_ssr(**double** *in_cbcr, **double** *out_cb_f,	**1.** Signature; constants 0.25 and 128.0.
2	**double** *out_cr_f, **int** w, **int** h) {	
3	**register double** quarter **asm**("ft11") = 0.25;	

```
4     double c_128 = 128.0;
5
6     snrt_ssr_loop_4d(SNRT_SSR_DM0, 4, 2, 8, 8, 8, 32*8, 32, 32*8*2);
7     snrt_ssr_read(SNRT_SSR_DM0, SNRT_SSR_4D, in_cbcr);  // DM0: 2×2 blocks
8
9     snrt_ssr_loop_1d(SNRT_SSR_DM1, (w/2)*(h/2), sizeof(double));
10    snrt_ssr_write(SNRT_SSR_DM1, SNRT_SSR_1D, out_cb_f);  // DM1: Cb out
11
12    snrt_ssr_loop_1d(SNRT_SSR_DM2, (w/2)*(h/2), sizeof(double));
13    snrt_ssr_write(SNRT_SSR_DM2, SNRT_SSR_1D, out_cr_f);  // DM2: Cr out
14    snrt_ssr_enable();
15
16  #ifdef SNRT_SUPPORTS_FREP
17    asm volatile(
18        "frep.o %[tot], 16, 0, 0 \n"
19        "fmv.d ft8, ft0 \n fmv.d ft9, ft0 \n"  // pop 4 Cb + 4 Cr
20        "fmv.d ft3, ft0 \n fmv.d ft10, ft0 \n"
21        "fmv.d ft4, ft0 \n fmv.d ft7, ft0 \n"
22        "fmv.d ft5, ft0 \n fmv.d ft6, ft0 \n"
23        "fadd.d ft8, ft8, ft3 \n fadd.d ft9, ft9, ft10 \n"  // pairwise sums
24        "fadd.d ft4, ft4, ft5 \n fadd.d ft7, ft7, ft6 \n"
25        "fadd.d ft8, ft8, ft4 \n fadd.d ft9, ft9, ft7 \n"
26        "fmadd.d ft1, ft8, %[q], %[c128] \n"  // Cb = 0.25·Σ + 128
27        "fmadd.d ft2, ft9, %[q], %[c128] \n"  // Cr = 0.25·Σ + 128
28        :
29        : [tot] "r" ((w/2 * h/2) - 1),
30          [q] "f" (quarter), [c128] "f" (c_128)
31        : "ft0", "ft1", "ft2", "ft3", "ft4", "ft5",
32          "ft6", "ft7", "ft8", "ft9", "ft10", "memory"
33    );
34  #endif
35    snrt_fpu_fence();
36    snrt_ssr_disable();
37  }
```

2. Stream setup: DM0 reads the 2×2 blocks with a 4D pattern; DM1 and DM2 write Cb and Cr.

3. 16-instruction FREP body: sum of the four samples, then ×0.25 and +128 in one fmadd.d (Eq. 4.11).

4. Operands and clobbered registers.

5. Wait for the FPU and release the streams.

The +128 constant that appears in Listing 4.3 is one of several points where the pipeline shifts or converts its data. In the JPEG standard, the chrominance components carry a +128 offset (Section 2.2.3), so that the color differences (B − Y) and (R − Y), which lie approximately in [−127.5, +127.5], are stored as unsigned 8-bit values with neutral gray at 128. Conversely, the *DC level shift* of the DCT stage (Equation (4.5)) subtracts 128 from every sample, so that the transform operates on zero-centred data in [−128, +127].

In the proposed pipeline, the raw 8-bit samples are first converted to FP64 (uint8_t → double) by the 256-entry LUT fetched through the ISSR (Section 4.2.1), so no arithmetic instruction is spent on the cast. The color conversion of Listing 4.1 then computes Cb and Cr exactly as in Equations (4.2) and (4.3), that is, as zero-centred values without the +128 term. The offset is restored during chroma subsampling, inside the averaging instruction itself (fmadd.d ft1, ft8, 0.25, 128.0, Equation (4.11)), at no extra instruction cost; in the FP16 variant, the same constant is broadcast into all four lanes (c128_vec, value 0x5800) and added with vfadd.h (Listing 4.4). Before the transform, Listing 4.5 subtracts 128 from every sample (Equation (4.5)), and after quantization the coefficients are converted to integers by fcvt.w.d in the zig-zag stage (Listing 4.9) and stored as 16-bit values (double → int16_t).

For the chrominance channels, the +128 of the subsampling stage and the −128 of the level shift **cancel each other exactly**. Keeping both operations makes the intermediate YCbCr buffers identical to the standard representation, which simplifies verification against a reference encoder; removing them for Cb and Cr would be a possible further micro-optimisation.

4.3.3 Vectorized Implementation (Packed SIMD FP16)

In an effort to accelerate the process further, the second implementation uses half precision (FP16). Because of the three-DM limit, a radical reallocation of resources was chosen: DM0 reads the top row of the 2×2 block, while DM1 simultaneously reads the bottom row. The vector addition vfadd.h merges the data vertically, and the pairwise-sum instruction vfsum.h performs the horizontal addition of adjacent elements. No multiplication by 0.25 is needed in this kernel, because the factor has already been folded into the Cb/Cr coefficients of Listing 4.2.

The pairwise vfsum.h instruction, however, presents an *architectural peculiarity*: the valid sums are stored only in the lower lanes (the first two 16-bit positions) of the 64-bit register, while the upper lanes remain empty or contain invalid data (garbage). Listing 4.4 shows the resulting kernel; its second half (block 4) extracts the valid 32-bit part of each register to the integer core (fmv.x.w) and writes it with sw instructions inside a conventional branch-controlled loop.

Listing 4.4: *Vectorized Chroma Subsampling implementation with FP16. Notice the use of the ALU for manual data writing (sw) due to SSR stride incompatibilities.*

1	`void opt_chroma_420_2d_ssr(double *in_cbcr, float16_t *out_cb_f,`	**1.** Signature; 128 broadcast into the four FP16 lanes; pointers to the top and bottom rows.
2	`float16_t *out_cr_f) {`	
3	`  double zero_val = 0.0;`	
4	`  double *p_zero = &zero_val;`	
5	`  float16_t val128 = float_to_fp16(128.0f); // 0x5800`	
6		
7	`  uint64_t c128_vec __attribute__((aligned(8))) =`	
8	`  ((uint64_t)val128 << 48) | ((uint64_t)val128 << 32) |`	
9	`  ((uint64_t)val128 << 16) | val128;`	
10		
11	`  void *p_top = (void*)in_cbcr;`	
12	`  void *p_bot = (void*)((uintptr_t)in_cbcr + 64);`	
13	`  uint64_t *p_c128 = &c128_vec;`	
14		
15	`  snrt_ssr_loop_2d(SNRT_SSR_DM0, 8, 8, 8, 128);  // DM0: top row`	**2.** Stream setup: DM0 reads the top row, DM1 the bottom row; no write stream.
16	`  snrt_ssr_read(SNRT_SSR_DM0, SNRT_SSR_2D, p_top);`	
17		
18	`  snrt_ssr_loop_2d(SNRT_SSR_DM1, 8, 8, 8, 128);  // DM1: bottom row`	
19	`  snrt_ssr_read(SNRT_SSR_DM1, SNRT_SSR_2D, p_bot);`	
20		
21	`  snrt_ssr_enable();`	
22		**3.** Branch-controlled loop (no FREP): vertical vfadd.h, horizontal vfsum.h, +128.
23	`  uint32_t alu_cb1, alu_cr1, alu_cb2, alu_cr2, iters = 15;`	
24		
25	`  asm volatile(`	

```
26          "fld f31, 0(%[zero]) \n fld f30, 0(%[c128]) \n"
27          "csrsi 0x801, 0x1 \n 1: \n"  // loop start
28          "fmv.d f4, f0 \n fmv.d f5, f1 \n"
29          "fmv.d f8, f0 \n fmv.d f9, f1 \n"
30          "fmv.d f12, f0 \n fmv.d f13, f1 \n"
31          "fmv.d f16, f0 \n fmv.d f17, f1 \n"
32          "vfadd.h f6, f4, f5 \n vfadd.h f10, f8, f9 \n"  // vertical sums
33          "vfadd.h f14, f12, f13 \n vfadd.h f18, f16, f17 \n"
34          "fmv.d f7, f31 \n fmv.d f11, f31 \n"
35          "fmv.d f15, f31 \n fmv.d f19, f31 \n"
36          "vfsum.h f7, f6 \n vfsum.h f11, f10 \n"  // horizontal sums
37          "vfsum.h f15, f14 \n vfsum.h f19, f18 \n"
38          "vfadd.h f7, f7, f30 \n vfadd.h f11, f11, f30 \n"  // + 128
39          "vfadd.h f15, f15, f30 \n vfadd.h f19, f19, f30 \n"
40          "fmv.x.w %[acb1], f7 \n fmv.x.w %[acr1], f11 \n"  // FPU → ALU, then sw
41          "fmv.x.w %[acb2], f15 \n fmv.x.w %[acr2], f19 \n"
42          "sw %[acb1], 0(%[out_cb]) \n sw %[acb2], 4(%[out_cb]) \n"
43          "sw %[acr1], 0(%[out_cr]) \n sw %[acr2], 4(%[out_cr]) \n"
44          "addi %[out_cb], %[out_cb], 8 \n"
45          "addi %[out_cr], %[out_cr], 8 \n"
46          "addi %[iters], %[iters], -1 \n"
47          "bgez %[iters], 1b \n csrci 0x801, 0x1 \n"  // branch
48        : [out_cb] "+r"(out_cb_f), [out_cr] "+r"(out_cr_f),
49          [iters] "+r"(iters), [acb1] "=&r"(alu_cb1),
50          [acr1] "=&r"(alu_cr1), [acb2] "=&r"(alu_cb2),
51          [acr2] "=&r"(alu_cr2)
52        : [zero] "r"(p_zero), [c128] "r"(p_c128)
53        : "f4", "f5", "f6", "f7", "f8", "f9", "f10", "f11", "f12",
54          "f13", "f14", "f15", "f16", "f17", "f18", "f19", "f30",
55          "f31", "memory"
56    );
57    snrt_ssr_disable();
58  }
```

4. Valid lower 32 bits moved to the ALU (fmv.x.w) and stored with sw; pointer and loop updates.

5. Operands, clobbered registers and stream release.

4.3.4 Comparative Analysis: SSR Write Constraints and ALU Involvement

The fact that the useful result of the SIMD processing lands only in the lower 32 bits (two 16-bit values) of the register reveals a critical hardware limitation: the SSR write mechanisms **do not support spatially non-contiguous writes with a stride of 16 or 32 bits**; they mandate 64-bit aligned writes. If the third Data Mover (DM2) were assigned to write the register contents directly to memory, the empty (or garbage) data of the upper 32 bits would overwrite and corrupt adjacent memory locations.

To bypass this constraint, the FP16 implementation is forced to abandon frep.o completely and to involve the integer Arithmetic Logic Unit (ALU). Specifically, the ALU takes over *the following memory-routing tasks*:

- **Register Extraction:** The move instruction fmv.x.w copies the lowest valid 32-bit segment of the FPU register into an integer register (e.g., alu_cb1).
- **Manual Stores:** The ALU performs the memory routing with conventional sw (Store Word) instructions.
- **Loop Control:** Additional integer arithmetic (addi) increments the pointers and controls the loop (bgez).

In conclusion, the inability of the SSR to support fine-grained writes smaller than 64 bits, combined with the nature of the vfsum.h instruction, negates much of the benefit of vectorized computation. Activating the ALU and the main processor for memory operations inside the critical loop **transforms the kernel from compute-bound to instruction/memory-bound**, which again makes the FP64 implementation the structurally preferable choice.

4.3.5 Cycle Count Analysis and Performance Evaluation

To place the architectural constraints and memory-bound limitations of Section 4.3.4 in context, the execution cycles of the Chroma Subsampling stage were measured. The stage requires *the following cycle counts*:

- **Initial Design (Naive):** 2,303.75 cycles
- **Double FP64:** 1,456.25 cycles
- **Vectorized OPT FP16:** 725.00 cycles

Interestingly, the cycle counts reveal a localized performance advantage for the half-precision approach. Despite the forced activation of the ALU, the loss of zero-overhead loops (frep.o) and the manual memory routing required to bypass the SSR write constraints, the OPT FP16 implementation processes this kernel in 725.00 cycles. This **halves the execution time** compared to the Double FP64 implementation (1,456.25 cycles; speedup 2.01× according to Equation (4.10)).

This localized speedup occurs because the vector additions (vfadd.h) and pairwise sums (vfsum.h) process multiple pixels simultaneously, providing enough *raw computational throughput* to mask the latency of the manual integer instructions (fmv.x.w, sw, addi).

However, as concluded in Section 4.3.4, this isolated cycle reduction **comes at a heavy architectural cost**. By abandoning the automated SSR write capabilities and forcing the main processor to handle sub-64-bit aligned stores, the kernel becomes instruction-bound. Although the FP16 kernel executes fewer cycles in isolation, the structural strain and register complexity it introduces make the Double FP64 implementation the more elegant, scalable and suitable choice when the stages are integrated into the continuous execution pipeline.

4.4 Two-Dimensional Discrete Cosine Transform (2D-DCT)

The computation of the 2D-DCT is the **most mathematically intensive phase** of the JPEG compression pipeline. The algorithm is compute-bound and relies on the separability principle, which decomposes the two-dimensional transform of an 8×8 block into two sequential one-dimensional passes (row-wise, followed by column-wise), as expressed in Equation (4.6). This stage was aggressively optimized in both FP64 and Packed FP16 precision to evaluate the Multiply-Accumulate (MAC) throughput of the cluster.

4.4.1 Double Precision Implementation (FP64)

The FP64 implementation is a prime example of **hardware-software co-design**, as it offloads almost all memory and control-flow overhead to the Snitch microarchitecture. The execution is divided into *three fluid stages*:

- **Level Shifting:** A 1D SSR streams the data into the FPU, where a single fadd.d inside a frep.o loop subtracts 128.0 from each sample (Equation (4.5)), centring the dynamic range.
- **Zero-Cost Transposition via 4D SSRs (Pass 1 & Pass 2):** To perform the matrix multiplications of Equation (4.6) without explicitly transposing the intermediate matrix in memory, the Data Movers DM0 and DM1 are programmed with 4D nested strides. By simply swapping the stride parameters in the SSR configuration between Pass 1 and Pass 2, the hardware fetches the matrix elements by row in the first pass and by column in the second pass, which removes the need for transposition instructions.
- **Nested FREP Execution:** The core matrix multiplication uses 8 accumulation registers (f3–f7 and f28–f30) to compute an entire 8-element row or column simultaneously. Thanks to nested frep.o loops, the complete 1D-DCT pass is executed without loop-control instructions, achieving near 100% FPU utilization.

Listing 4.5 shows the three stages in order (blocks 2, 3–4 and 5–6). The comment of Stage 2 marks the only difference between the two passes, namely the altered SSR strides of DM1, which implement the **zero-cost transposition**.

Listing 4.5: *FP64 2D-DCT featuring zero-cost matrix transposition via 4D SSR strides and nested FREP loops.*

1 2 3	**void** opt_dct_forward(**double** *in, **double** *out, **const double** *dct_m) { **register double** offset **asm**("ft3") = -128.0;	**1.** Signature; level-shift offset −128.

```
    // STAGE 0: Level Shift
    snrt_ssr_loop_1d(SNRT_SSR_DM0, 64, sizeof(double));
    snrt_ssr_read(SNRT_SSR_DM0, SNRT_SSR_1D, (void*)in);
    snrt_ssr_loop_1d(SNRT_SSR_DM1, 64, sizeof(double));
    snrt_ssr_write(SNRT_SSR_DM1, SNRT_SSR_1D, (void*)out);
    snrt_ssr_enable();
    asm volatile(
        "frep.o %[n], 1, 0, 0 \n fadd.d ft1, ft0, %[off] \n"  // f − 128
        : : [n] "r"(63), [off] "f"(offset) : "ft0", "ft1", "memory"
    );
    snrt_fpu_fence(); snrt_ssr_disable();

    // STAGE 1: Pass 1 - Rows
    snrt_ssr_loop_4d(SNRT_SSR_DM0, 8,8,1,8, 0,8,0,64);
    snrt_ssr_read(SNRT_SSR_DM0, SNRT_SSR_4D, (void*)out);
    snrt_ssr_loop_4d(SNRT_SSR_DM1, 8,8,1,8, 64,8,128,0);  // DM1: rows of T
    snrt_ssr_read(SNRT_SSR_DM1, SNRT_SSR_4D, (void*)dct_m);
    snrt_ssr_loop_1d(SNRT_SSR_DM2, 64, sizeof(double));
    snrt_ssr_write(SNRT_SSR_DM2, SNRT_SSR_1D, (void*)in);
    snrt_ssr_enable();
    asm volatile(
        "frep.o %[cnts], 24, 0, 0 \n"  // outer: 8 rows
        "fsub.d f3, f3, f3 \n fsub.d f4, f4, f4 \n fsub.d f5, f5, f5 \n"
        "fsub.d f6, f6, f6 \n fsub.d f7, f7, f7 \n fsub.d f28, f28, f28 \n"
        "fsub.d f29, f29, f29 \n fsub.d f30, f30, f30 \n"
        "frep.o %[cnt], 8, 0, 0 \n"  // inner: 7 MAC rounds
        "fmadd.d f3, f0, f1, f3 \n fmadd.d f4, f0, f1, f4 \n"
        "fmadd.d f5, f0, f1, f5 \n fmadd.d f6, f0, f1, f6 \n"
        "fmadd.d f7, f0, f1, f7 \n fmadd.d f28, f0, f1, f28 \n"
        "fmadd.d f29, f0, f1, f29 \n fmadd.d f30, f0, f1, f30 \n"
        "fmadd.d f2, f0, f1, f3 \n fmadd.d f2, f0, f1, f4 \n"  // 8th MAC → DM2
        "fmadd.d f2, f0, f1, f5 \n fmadd.d f2, f0, f1, f6 \n"
        "fmadd.d f2, f0, f1, f7 \n fmadd.d f2, f0, f1, f28 \n"
        "fmadd.d f2, f0, f1, f29 \n fmadd.d f2, f0, f1, f30 \n"
        : : [cnt] "r"(6), [cnts] "r"(7)
        : "f0","f1","f2","f3","f4","f5","f6","f7","f28","f29","f30","memory"
    );
    snrt_fpu_fence(); snrt_ssr_disable();

    // STAGE 2: Pass 2 - Columns (Notice altered SSR strides)
    snrt_ssr_loop_4d(SNRT_SSR_DM0, 8,8,1,8, 0,8,0,64);
    snrt_ssr_read(SNRT_SSR_DM0, SNRT_SSR_4D, (void*)dct_m);
    snrt_ssr_loop_4d(SNRT_SSR_DM1, 8,8,1,8, 8,64,16,0);  // DM1: column stride
    snrt_ssr_read(SNRT_SSR_DM1, SNRT_SSR_4D, (void*)in);
    snrt_ssr_loop_1d(SNRT_SSR_DM2, 64, sizeof(double));
    snrt_ssr_write(SNRT_SSR_DM2, SNRT_SSR_1D, (void*)out);
    snrt_ssr_enable();
    asm volatile(
        /* Identical nested FREP accumulation logic omitted for brevity */
        ...
    );
    snrt_fpu_fence(); snrt_ssr_disable();
}
```

2. Stage 0: level shift with a one-instruction FREP loop (Eq. 4.5).

3. Stage 1 streams: row-wise 4D patterns for data (DM0) and DCT matrix (DM1); DM2 writes the result.

4. Nested FREP: 8 accumulators cleared, 7 MAC rounds, the 8th MAC writes each coefficient to DM2.

5. Stage 2 streams: only the DM1 strides change, giving column access (zero-cost transposition).

6. Same FREP body as Stage 1; wait and release.

4.4.2 Vectorized Implementation (Packed SIMD FP16)

To exploit the SIMD capabilities, a Packed FP16 implementation was developed. Here, the matrix operations are executed with vectorized multiplications (vfmul.h). However, calculating *dot products* requires reducing the vector lanes to single values, which introduces severe architectural roadblocks in both computational passes.

Listing 4.6 contains the two passes as separate functions. In simd_dct_pass1 the reduction relies on vfsum.h and the results leave the FPU through the integer core (fmv.x.w and sh,

blocks 4–5), whereas simd_dct_pass2 uses a vfadd.h reduction tree that can be written by the SSR but runs in a branch-controlled loop (addi, bnez; block 8).

Listing 4.6: *Packed FP16 DCT passes. Notice the ALU memory stores in Pass 1 and the fallback to standard branching in Pass 2.*

```
1  void simd_dct_pass1(float16_t *in, float16_t *out, float16_t *tmp,
2                      const float16_t *dct_m) {
3      /* Level shift logic omitted for brevity */
4
5      snrt_ssr_loop_3d(SNRT_SSR_DM0, 2, 8, 8, 8, 0, 16);  // DM0: data
6      snrt_ssr_read(SNRT_SSR_DM0, SNRT_SSR_3D, (void*)out);
7      snrt_ssr_loop_2d(SNRT_SSR_DM1, 16, 8, 8, 0);  // DM1: DCT matrix
8      snrt_ssr_read(SNRT_SSR_DM1, SNRT_SSR_2D, (void*)dct_m);
9      snrt_ssr_enable();
10
11     double zero_val = 0.0; double *p_zero = &zero_val;
12     uint32_t af0, af1, af2, af3, alu_idx = 16;
13     asm volatile(
14         "fld f31, 0(%[zero]) \n csrsi 0x801, 0x1 \n 1: \n"  // loop start
15         "vfmul.h f4, f0, f1 \n vfmul.h f5, f0, f1 \n"
16         "vfmul.h f8, f0, f1 \n vfmul.h f9, f0, f1 \n"
17         "vfmul.h f12, f0, f1 \n vfmul.h f13, f0, f1 \n"
18         "vfmul.h f16, f0, f1 \n vfmul.h f17, f0, f1 \n"
19         "vfadd.h f5, f5, f4 \n vfadd.h f9, f9, f8 \n"
20         "vfadd.h f13, f13, f12 \n vfadd.h f17, f17, f16 \n"
21         "fmv.d f4, f31 \n fmv.d f8, f31 \n"
22         "fmv.d f12, f31 \n fmv.d f16, f31 \n"
23         "vfsum.h f4, f5 \n vfsum.h f8, f9 \n"  // lane reduction
24         "vfsum.h f12, f13 \n vfsum.h f16, f17 \n"
25         "fmv.d f5, f31 \n fmv.d f9, f31 \n"
26         "fmv.d f13, f31 \n fmv.d f17, f31 \n"
27         "vfsum.h f5, f4 \n vfsum.h f9, f8 \n"
28         "vfsum.h f13, f12 \n vfsum.h f17, f16 \n"
29         "fmv.x.w %[af0], f5 \n fmv.x.w %[af1], f9 \n"  // FPU → ALU
30         "fmv.x.w %[af2], f13 \n fmv.x.w %[af3], f17 \n"
31         "sh %[af0], 0(%[tmp]) \n sh %[af1], 2(%[tmp]) \n"  // 16-bit stores
32         "sh %[af2], 4(%[tmp]) \n sh %[af3], 6(%[tmp]) \n"
33         "addi %[tmp], %[tmp], 8 \n addi %[alu_idx], %[alu_idx], -1 \n"
34         "bnez %[alu_idx], 1b \n csrci 0x801, 0x1 \n"  // branch
35         : [tmp] "+r"(tmp), [alu_idx] "+r"(alu_idx), [af0] "=&r"(af0),
36           [af1] "=&r"(af1), [af2] "=&r"(af2), [af3] "=&r"(af3)
37         : [zero] "r"(p_zero)
38         : "f4","f5","f8","f9","f12","f13","f16","f17","f31","memory"
39     );
40     snrt_ssr_disable();
41 }
42
43 void simd_dct_pass2(float16_t *in, float16_t *out, uint64_t *mat_bcast) {
44     snrt_ssr_loop_2d(SNRT_SSR_DM0, 2, 64, 0, 8);
45     snrt_ssr_read(SNRT_SSR_DM0, SNRT_SSR_2D, mat_bcast);
46     snrt_ssr_loop_2d(SNRT_SSR_DM1, 16, 8, 8, 0);
47     snrt_ssr_read(SNRT_SSR_DM1, SNRT_SSR_2D, in);
48     snrt_ssr_loop_1d(SNRT_SSR_DM2, 16, 8);  // DM2: output
49     snrt_ssr_write(SNRT_SSR_DM2, SNRT_SSR_1D, out);
50     snrt_ssr_enable();
51
52     uint32_t iters = 8;
53     asm volatile(
54         "1: \n"  // loop start (no FREP)
55         "vfmul.h f4, f0, f1 \n vfmul.h f5, f0, f1 \n"
56         "vfmul.h f6, f0, f1 \n vfmul.h f7, f0, f1 \n"
57         "vfmul.h f8, f0, f1 \n vfmul.h f9, f0, f1 \n"
58         "vfmul.h f10, f0, f1 \n vfmul.h f11, f0, f1 \n"
59         /* Additional 8 vfmul.h omitted */
60         "vfadd.h f4, f4, f6 \n vfadd.h f5, f5, f7 \n"
61         "vfadd.h f8, f8, f10 \n vfadd.h f9, f9, f11 \n"
62         /* Vector reduction tree logic */
63         "vfadd.h f4, f4, f8 \n vfadd.h f5, f5, f9 \n"
64         "vfadd.h f12, f12, f16 \n vfadd.h f13, f13, f17 \n"
65         "vfadd.h f4, f4, f12 \n vfadd.h f5, f5, f13 \n"
66         "fmv.d f2, f4 \n fmv.d f2, f5 \n"  // SSR write
67         "addi %[iters], %[iters], -1 \n bnez %[iters], 1b \n"
```

1. Pass 1: signature and streams (DM0 data, DM1 DCT matrix); no write stream.

2. Constants; start of a branch-controlled loop.

3. Vector products (vfmul.h) and pairwise additions.

4. Lane reduction with vfsum.h.

5. Results moved to the ALU (fmv.x.w) and stored as 16-bit values (sh); branch.

6. Operands and stream release.

7. Pass 2: streams, now including a DM2 write stream.

8. Branch-controlled loop: 16 vfmul.h and a vfadd.h reduction tree (too long for FREP); SSR write.

```
68          : [iters] "+r"(iters)
69          :: "f4","f5","f6","f7","f8","f9","f10","f11","f12",
70            "f13","f14","f15","f16","f17","f18","f19","memory"
71      );
72      snrt_ssr_disable();
73  }
```

9. Operands and stream release.

4.4.3 Comparative Analysis: Sub-Indexing and Instruction Buffer Overflows

The DCT execution exposed **two distinct hardware limitations** that severely compromised the FP16 implementation compared to the seamless FP64 pipeline.

A) ALU Interference and Sub-Indexing (Pass 1)

During the first pass, calculating the dot product required a *horizontal vector reduction* through the pairwise-sum instruction vfsum.h. As observed in the Chroma stage, vfsum.h packs the valid results into the lowest bits of the register. Because the SSRs cannot perform fine-grained (16-bit) writes without overwriting neighbouring elements, the FPU was unable to stream the results directly to memory. Consequently, the valid segments had to be extracted to the integer ALU (fmv.x.w), which then executed Store Halfword (sh) instructions manually. This memory-routing bottleneck forced the ALU to participate in the innermost loop and **broke the hardware streaming paradigm**.

B) FREP Buffer Overflow and Loop Control Overhead (Pass 2)

To circumvent the ALU memory bottleneck in the second pass, the algorithm was redesigned to use a purely FPU-based *reduction tree* with vfadd.h instead of vfsum.h, so that the final accumulated vectors could be written safely by the SSR. However, this reduction tree required extensive loop unrolling (16 vfmul.h and multiple vfadd.h instructions per iteration). The unrolling exposed a critical micro-architectural limit of the Snitch core: the hardware instruction buffer that supports the frep.o instruction is **strictly limited to a capacity of 32 instructions**.

Because the heavily unrolled FP16 loop body exceeded this 32-instruction threshold, **the assembler rejected the frep.o command**. As a result, the code had to fall back to a conventional loop structure with standard branch instructions (addi, bnez). Reintroducing loop-control instructions and branch-evaluation latency into a compute-bound mathematical kernel largely negated the throughput advantage of SIMD vectorization.

Structurally, the FP64 implementation therefore emerged as superior. By avoiding vector reduction operations entirely, it bypassed both the SSR write-alignment issue and the frep

buffer size limit, **maintaining a pure, unbroken stream of mathematical operations** executed at maximum hardware efficiency.

4.4.4 Cycle Count Analysis and Performance Evaluation

To quantify the effects of the structural issues discussed in Section 4.4.3, the execution cycles of the 2D-DCT stage (labelled SDCT in Table 4.3) were measured. The stage requires *the following cycle counts*:

- **Initial Design (Naive):** 40,865.75 cycles
- **Double FP64:** 9,669.25 cycles
- **Vectorized OPT FP16:** 6,814.25 cycles

The transition from the scalar C implementation to the Snitch cluster yields large improvements in both optimized precisions. According to Equation (4.10), the FP64 implementation achieves a **4.23× speedup over the Initial Design**, while the OPT FP16 implementation raises it to nearly 6× (5.997×).

The isolated performance of the FP16 implementation reveals an interesting hardware dynamic. Despite the architectural roadblocks, namely the forced ALU interference in Pass 1 and the frep.o instruction-buffer overflow in Pass 2, the *brute-force* throughput of the vectorized vfmul.h operations still outperforms the double-precision implementation by approximately 2,855 cycles (a 1.42× localized speedup). The ability of the Snitch core to execute multiple 16-bit MACs per cycle provides enough raw computational density to offset the latencies introduced by standard branching (bnez) and manual memory routing.

However, as concluded in Section 4.4.3, this localized cycle reduction comes at the cost of software elegance and pipeline scalability. The FP64 implementation uses the intended power of the hardware-software co-design: zero-cost transpositions via 4D SSR strides, zero loop-control overhead via nested frep.o execution, and fully automated memory streaming. By operating strictly within the limits of the instruction buffer and the SSR write capabilities, the FP64 approach remains the **superior structural paradigm**, guaranteeing stability and hardware efficiency when integrated into the complete JPEG pipeline.

4.5 Quantization and Zero-Latency Rounding

The quantization stage performs an element-wise scaling of the DCT coefficients to achieve lossy compression (Equation (4.7)). Because division is a notoriously slow operation in any

hardware architecture, the algorithm uses a pre-computed matrix of *reciprocal values*, so that the division is replaced by a much faster floating-point multiplication. After the multiplication, however, the values must be rounded to the nearest integer.

4.5.1 Micro-Architectural Optimization: "Magic Number" Fast Rounding

To eliminate the latency of the type-casting and rounding instructions, the implementation exploits a property of the *IEEE-754* double-precision format through the constant M_{magic} = 6755399441055744.0, which equals 1.5×2^{52}. The resulting rounding operation is given by Equation (4.12).

$$\text{round}(x) = \left(x + M_{\text{magic}}\right) - M_{\text{magic}}, \quad M_{\text{magic}} = 1.5 \cdot 2^{52} = 6755399441055744 \tag{4.12}$$

When a floating-point number is added to this very large "magic number", the hardware *exponent alignment* of the FPU pushes the fractional bits of the smaller number beyond the end of the 52-bit mantissa. The hardware automatically applies the active rounding mode (here rne, Round to Nearest, ties to Even) to the shifted bits. Subtracting the magic number immediately afterwards restores the original value, rounded to an integer but still stored in a floating-point register. The trick therefore acts as a **zero-latency round() function**, executed at the native speed of an addition and a subtraction.

4.5.2 Double Precision Implementation (FP64)

In the double-precision implementation, the three available SSR Data Movers **map exactly onto the requirements of the task**: DM0 streams the input DCT coefficients, DM1 streams the reciprocal matrix, and DM2 returns the quantized results to memory.

Because the data flow is strictly sequential and perfectly mapped to the hardware resources, the frep.o instruction controls the loop. As Listing 4.7 shows, the magic number is kept in register ft10 (block 1), and the six-instruction loop body (block 3) relies purely on fmul.d, fadd.d and fsub.d to process two elements per iteration (loop unrolling), implementing Equations (4.7) and (4.12).

Listing 4.7: *FP64 Quantization leveraging the "Magic Number" trick for zero-cost rounding within a continuous FREP stream.*

```
1  void opt_quant(double *in, double *out, const double *recip) {
2      register double magic asm("ft10") = 6755399441055744.0;
3
4      snrt_ssr_loop_1d(SNRT_SSR_DM0, 64, sizeof(double)); // ft0 ← F(u,v)
5      snrt_ssr_read(SNRT_SSR_DM0, SNRT_SSR_1D, (void*)in);
6
7      snrt_ssr_loop_1d(SNRT_SSR_DM1, 64, sizeof(double)); // ft1 ← R_rec(u,v)
8      snrt_ssr_read(SNRT_SSR_DM1, SNRT_SSR_1D, (void*)recip);
```

1. Signature; M_{magic} kept in register ft10.

2. Stream setup: DM0 reads the DCT coefficients, DM1 the reciprocals, DM2 writes the result.

```
9
10      snrt_ssr_loop_1d(SNRT_SSR_DM2, 64, sizeof(double));  // ft2 → F_Q(u,v)
11      snrt_ssr_write(SNRT_SSR_DM2, SNRT_SSR_1D, (void*)out);
12      snrt_ssr_enable();
13
14      asm volatile(
15          "frep.o %[cnt], 6, 0, 0    \n"  // 32 iterations
16          "fmul.d ft3, ft0, ft1      \n fmul.d ft4, ft0, ft1      \n"  // F · R_rec
17          "fadd.d ft5, ft3, ft10, rne\n fadd.d ft6, ft4, ft10, rne\n"  // + M_magic
18          "fsub.d ft2, ft5, ft10, rne\n fsub.d ft2, ft6, ft10, rne\n"  // − M_magic
19          :
20          : [cnt] "r"(31), "f"(magic)
21          : "ft0", "ft1", "ft2", "ft3", "ft4", "ft5", "ft6",
22            "ft10", "ft11", "memory"
23      );
24      snrt_fpu_fence();
25      snrt_ssr_disable();
26  }
```

3. 6-instruction FREP body, two coefficients per iteration: multiplication and magic-number rounding (Eq. 4.7, 4.12).

4. Operands and clobbered registers.

5. Wait for the FPU and release the streams.

4.5.3 Vectorized Implementation (Packed SIMD FP16)

Unlike the Chroma Subsampling and DCT stages, which suffered from register interleaving, vector-reduction bottlenecks and ALU memory interference, the Quantization stage presents a **fundamentally different data profile**. It is a strictly *point-to-point*, element-wise operation: there are no cross-lane data dependencies, no horizontal summations (vfsum.h) and no need for sub-indexing.

Because of this structural simplicity, the Packed FP16 implementation operates without obstacles. The magic-number logic is replicated with a vector constant magic_p loaded into f10. By packing four half-precision values into one 64-bit word, the 64-element block is reduced to just **16 fetch cycles**. Listing 4.8 shows the unrolled 12-instruction loop body (block 3), in which vfmul.h, vfadd.h and vfsub.h process four packed vectors per iteration.

Listing 4.8: *Packed FP16 Quantization. Freed from cross-lane dependencies, this vector implementation achieves true SIMD throughput without ALU interference.*

```
1   void opt_quant_fp16(float16_t *in, float16_t *out,
2                       const float16_t *recip, uint64_t *magic_p) {
3       snrt_ssr_loop_1d(SNRT_SSR_DM0, 16, 8);  // DM0: 16 × 4 FP16
4       snrt_ssr_read(SNRT_SSR_DM0, SNRT_SSR_1D, (void*)in);
5
6       snrt_ssr_loop_1d(SNRT_SSR_DM1, 16, 8);  // DM1: reciprocals
7       snrt_ssr_read(SNRT_SSR_DM1, SNRT_SSR_1D, (void*)recip);
8
9       snrt_ssr_loop_1d(SNRT_SSR_DM2, 16, 8);  // DM2: output
10      snrt_ssr_write(SNRT_SSR_DM2, SNRT_SSR_1D, (void*)out);
11      snrt_ssr_enable();
12
13      asm volatile(
14          "fld f10, 0(%[magic]) \n"  // vector M_magic
15          "frep.o %[cnt], 12, 0, 0 \n"  // 4 iterations
16          "vfmul.h f4, f0, f1 \n vfmul.h f5, f0, f1 \n"  // F · R_rec
17          "vfmul.h f6, f0, f1 \n vfmul.h f7, f0, f1 \n"
18          "vfadd.h f8, f4, f10 \n vfadd.h f9, f5, f10 \n"  // + M_magic
19          "vfadd.h f12, f6, f10 \n vfadd.h f13, f7, f10 \n"
20          "vfsub.h f2, f8, f10 \n vfsub.h f2, f9, f10 \n"  // − M_magic, store
21          "vfsub.h f2, f12, f10 \n vfsub.h f2, f13, f10 \n"
22          : : [cnt] "r"(3), [magic] "r"(magic_p)
23          : "f0", "f1", "f2", "f4", "f5", "f6", "f7", "f8",
```

1. Signature.

2. Stream setup: 16 packed 64-bit words per stream (four FP16 values each).

3. Vector M_{magic} loaded; 12-instruction FREP body processes four packed vectors per iteration.

4. Operands and clobbered

24 25	"f9", "f10", "f12", "f13", "memory");	registers.

26
27
28
	snrt_fpu_fence();
	snrt_ssr_disable();
	}
		5. Wait for the FPU and release the streams.

4.5.4 Comparative Analysis: SIMD Success in Element-Wise Operations

The comparison between the two precisions in the quantization stage yields a distinctly different conclusion from the previous sections. Because quantization involves no spatial data entanglement, the FP16 implementation fully uses the vector lanes of the hardware without reaching the 32-instruction frep limit and without requiring ALU-based memory stores.

The FP16 version establishes a **perfect 1:1 mapping between input and output vectors**. It executes with a significantly reduced instruction count (cnt = 3, i.e., only 4 iterations for all 64 elements), demonstrating that the packed SIMD capabilities of the Snitch architecture are highly effective provided that the target algorithm is strictly point-to-point and free of horizontal reduction requirements.

4.5.5 Cycle Count Analysis and Performance Evaluation

To quantify the success of the vectorized SIMD approach in element-wise operations, the execution cycles of the Quantization stage were measured. The stage requires *the following cycle counts*:

- **Initial Design (Naive):** 39,958.50 cycles
- **Double FP64:** 2,114.50 cycles
- **Vectorized OPT FP16:** 478.00 cycles

The cycle counts reflect the structural advantages outlined in Section 4.5.4. Unburdened by spatial data entanglement, cross-lane dependencies or SSR write bottlenecks, the OPT FP16 implementation reaches its theoretical potential. By packing four half-precision values per register and unrolling the loop without overflowing the 32-instruction frep.o buffer, the vectorized version processes the entire 8×8 block in just **478.00 cycles**.

According to Equation (4.10), this represents a 4.42× speedup over the already highly optimized Double FP64 implementation (2,114.50 cycles) and an **83.60× speedup over the Initial Design** (39,958.50 cycles).

This localized result confirms a critical architectural characteristic of the Snitch cluster: when the algorithm is strictly point-to-point and aligns cleanly with the uninterrupted streaming

paradigm of the hardware, **the Packed SIMD FP16 approach is clearly the best choice for that stage**, executing at maximum hardware efficiency.

4.6 Zig-Zag Reordering and Differential Encoding

The final stage before entropy coding is the Zig-Zag reordering of the 8×8 block and the Differential Pulse Code Modulation (DPCM) of the DC coefficient (Equation (4.8)). This stage is the **bridge between the floating-point FPU domain and the bit-level integer domain**: its output must consist of discrete 16-bit signed integers (int16_t) arranged in a 1D sequence optimized for Run-Length Encoding (RLE).

This stage revealed the **largest performance disparity** between the FP64 and FP16 implementations, in favour of FP64, whose hardware queuing mechanisms the FP16 variant, as implemented here, could not use. Section 4.8 shows that this gap is caused by the **unpacking method** rather than by the FP16 format itself.

4.6.1 Double Precision Implementation (FP64)

The FP64 implementation fully exploits the Snitch microarchitecture. It decouples memory fetching, floating-point conversion and integer logic by using *three distinct hardware features*:

- **Zero-Cost Reordering via ISSR:** Instead of looking up the indices manually (in[zigzag_map[i]]), the Indirect SSR is programmed to read the zigzag_map. The Address Generation Unit (AGU) automatically fetches the floating-point values in zig-zag order and streams them into the FPU with zero CPU overhead.
- **Hardware F2I Queues (copift):** Setting the copift CSR enables a hardware FIFO queue between the FPU and the integer ALU [30].
- **Pipelined Conversion:** A frep.o loop executes 64 fcvt.w.d (float-to-int) instructions. The FPU continuously pops the zig-zagged values from the SSR, converts them, and pushes the integers into the hardware queue (mapped to the t6 register).

Meanwhile, the main core simply pops the converted values from t6, applies the DPCM of Equation (4.8) to the first element and stores the remaining ones. The FPU and the CPU operate *asynchronously*, masking memory latency and achieving maximum throughput. Listing 4.9 shows this cooperation: the ISSR configuration (block 2), the single-instruction

FREP loop that fills the queue (block 3), and the two pop sequences for the DC and AC coefficients (blocks 4–5).

Listing 4.9: *FP64 Zig-Zag using ISSR for zero-cost routing and the F2I hardware queue (t6) to decouple FPU conversion from ALU execution.*

1
2
3
4
5
6
7
	void opt_zigzag_and_diff(**const double** *in_blocks, **int16_t** *out_blocks,
	int *prev_dc, **int** num_blocks,
	const uint8_t *zigzag_map) {
	for (**int** b = 0; b < num_blocks; b++) {
	const double *in = &in_blocks[b * 64];
	int16_t *out = &out_blocks[b * 64];
		1. Signature; pointers to the current block.

8
9
10
11
12
	// AGU automatically fetches in[zigzag_map[i]] to FPU
	snrt_issr_read(SNRT_SSR_DM0, (**void***)in, (**void***)zigzag_map,
	64, SNRT_SSR_IDXSIZE_U8); *// zig-zag gather*
	snrt_ssr_enable();
		2. ISSR gathers in[zigzag_map[i]] in zig-zag order.

13 14 15 16 17 18 19 20 21	**int32_t** val; **asm volatile**("mv t6, x0 \n csrsi copift, 0x1 \n" // Enable F2I queue "frep.o %[n], 1, 0, 0 \n" *// 64 conversions* "fcvt.w.d t6, ft0 \n" // FPU push to queue "mv %[val], t6 \n" // CPU pop from queue : [val] "=r"(val) : [n] "r"(63) : "t6", "ft0", "memory");	**3.** copift F2I queue enabled; FREP of 64 fcvt.w.d pushes the integers into t6.

22
23
24
25
26
	// Process DC component (ALU pops from queue)
	int current_dc = val;
	out[0] = (**int16_t**)(current_dc - *prev_dc); *// DIFF (Eq. 4.8)*
	*prev_dc = current_dc;
		4. DC: first value popped and differentially encoded (Eq. 4.8).

27
28
29
30
31
32
	// Process AC components
	for (**int** i = 1; i < 64; i++) {
	// Pop remaining 63 elements from F2I queue
	asm volatile("mv %0, t6" : "=r"(val) :: "t6");
	out[i] = (**int16_t**)val;
	}
		5. AC: the remaining 63 values popped and stored.

33
34
35
36
	asm volatile("csrci copift, 0x1 \n" ::: "memory");
	snrt_fpu_fence(); snrt_ssr_disable();
	}
	}
		6. Queue disabled; wait and release.

4.6.2 Scalar Fallback Implementation (FP16)

In the FP16 implementation described here, this stage becomes a **serious obstacle**. Because the data must be extracted into discrete integers for Huffman encoding, the vector (SIMD) capabilities of FP16 are of no use here: the elements must be unpacked and processed sequentially. Furthermore, because four values share each 64-bit word, the ISSR, which gathers whole 64-bit elements, cannot be used directly to fetch individual FP16 values in zig-zag order.

Listing 4.10 shows the resulting scalar loop. For every one of the 64 elements, the CPU computes the address, NaN-boxes the raw FP16 bits[4], moves them to the FPU and receives the converted integer back (blocks 2–3).

Listing 4.10: *FP16 unpacking loop. Stripped of hardware acceleration, the CPU must manually NaN-box bits and ping-pong data with the FPU.*

```
1  void unpack_and_zigzag_diff(const float16_t *in_blocks,
2                              int16_t *out_blocks, int *prev_dc,
3                              int num_blocks, const uint8_t *zz_map) {
4      for (int b = 0; b < num_blocks; b++) {
5          const float16_t *in = &in_blocks[b * 64];
6          int16_t *out        = &out_blocks[b * 64];
7
8          for (int i = 0; i < 64; i++) {
9              uint8_t z_idx = zz_map[i];  // manual lookup
10
11             // Manually NaN-box raw FP16 bits for RISC-V FPU
12             uint32_t raw_bits = 0xFFFF0000 | (uint32_t)in[z_idx];
13             int32_t val;
14
15             // Load into FPU, expand FP16->FP32, truncate FP32->INT
16             asm volatile(
17                 "fmv.w.x ft0, %[in_bits]    \n"  // ALU → FPU
18                 "fcvt.s.h ft1, ft0          \n"  // FP16 → FP32
19                 "fcvt.w.s %[out_val], ft1, rtz \n"  // FPU → ALU
20                 : [out_val] "=r"(val) : [in_bits] "r"(raw_bits)
21                 : "ft0", "ft1"
22             );
23
24             if (i == 0) {
25                 int current_dc = val;
26                 out[0] = (int16_t)(current_dc - *prev_dc);
27                 *prev_dc = current_dc;
28             } else {
29                 out[i] = (int16_t)val;
30             }
31         }
32     }
33 }
```

1. Signature; pointers to the current block.
2. Scalar loop: manual zig-zag index and NaN-boxing of the FP16 bits.
3. FPU round trip: move in, FP16 → FP32, FP32 → integer.
4. DC: differential encoding; AC: plain store.
5. End of loops.

4.6.3 Comparative Analysis: The Cost of Synchronization

The performance gap between the two implementations in this stage is very large and stems entirely from the way the code interacts with the hardware subsystems.

A) Loss of Hardware Memory Addressing

In the FP16 code, the CPU must calculate the address of every pixel manually (uint8_t z_idx = zz_map[i]; in[z_idx]). This turns a zero-latency memory stream into a series of **scalar memory loads that stall the pipeline**.

[4] NaN-boxing is the RISC-V convention for storing a narrower floating-point value in a wider floating-point register: all unused upper bits of the register must be set to 1. A value that is not correctly NaN-boxed is treated by the FPU as a canonical NaN (Not a Number) [16].

B) Synchronous FPU-ALU Ping-Pong vs. Decoupled Queues

The FP16 pipeline forces a lockstep, synchronous execution. For every single pixel, the CPU must load the data, execute bitwise operations to *NaN-box* the 16-bit value into a 32-bit register (0xFFFF0000 | ...), and transfer it manually to the FPU (fmv.w.x). The FPU then needs two separate conversion instructions (fcvt.s.h and fcvt.w.s) before passing the result back to the CPU. The FP64 implementation, conversely, uses the copift hardware queue: the FPU converts all 64 values independently and concurrently via frep.o and buffers them in a FIFO queue, from which the CPU simply pops the ready integers.

Ultimately, the FP16 implementation turns a highly parallel, decoupled architecture back into a **primitive scalar processor**, flooding the instruction stream with bitwise formatting and synchronization overhead. In this form, the stage cannot compete with the FP64 data path. The root of the problem, however, is the **per-value unpacking strategy** of Listing 4.10 (software NaN-boxing and an ALU–FPU round trip for every value), not the FP16 format as such; Section 4.8 shows that a different unpacking method removes most of this overhead.

4.6.4 Cycle Count Analysis and Performance Evaluation

To quantify the effects discussed in Section 4.6.3, the execution cycles of the Zig-Zag Reordering and Differential Encoding stage were measured. The stage requires *the following cycle counts*:

- **Initial Design (Naive):** 9,387.25 cycles
- **Double FP64:** 1,035.00 cycles
- **Vectorized OPT FP16:** 10,355.25 cycles

The cycle counts expose a striking performance divergence. The Double FP64 implementation, using the copift hardware queue and the zero-cost memory routing of the ISSR, achieves a **9.07× speedup over the Initial Design**. By decoupling the floating-point conversions of the FPU from the integer logic of the CPU, it processes the entire block with near-perfect hardware efficiency.

Conversely, the Vectorized OPT FP16 implementation is **much slower** and requires 10,355.25 cycles. This is approximately 10× (10.0×) slower than the FP64 implementation and even slower than the unoptimized Initial Design.

This regression quantifies the "*cost of synchronization*" discussed above. Stripped of its SIMD capabilities and hardware memory acceleration, the FP16 pipeline is forced to calculate

indices manually, NaN-box raw bits into 32-bit registers, and execute a synchronous ping-pong sequence between the CPU and the FPU for every individual pixel.

In this implementation, this stage is the **main bottleneck of the half-precision pipeline**. Although FP16 showed localized victories in earlier compute-bound stages (such as YCbCr and Quantization), the way the packed data is extracted here into discrete, sequential integers outweighs those gains. Section 4.8 revisits this unpacking step and shows that the penalty is **not inherent to FP16**.

4.7 Performance Evaluation and Baseline Selection

To determine the optimal foundation for the multi-core expansion of the JPEG pipeline, a comprehensive *cycle-accurate* performance evaluation was conducted. Three single-core implementations were benchmarked: the scalar C Initial Design (Naive), the hardware-accelerated Double Precision implementation (Double FP64), and the vectorized Packed SIMD FP16 implementation (OPT FP16).

Table 4.3 summarizes the profiling results, measured in clock cycles per Minimum Coded Unit (MCU), for every stage of the three implementations; the last row gives the total cycles of the complete pipeline.

Table 4.3: Performance evaluation between the different implementations

Stage	Initial Design / Naive (Cycles)	Double FP64 (Cycles)	OPT FP16 (Cycles)
YCBCR	14,503.00	4,039.50	2,103.00
CHROMA	2,303.75	1,456.25	725.00
SDCT	40,865.75	9,669.25	6,814.25
QUANT	39,958.50	2,114.50	478.00
ZIGZAG	9,387.25	1,035.00	10,355.25
HUFFMAN	14,943.50	14,814.50	14,886.75
Total Pipeline	**121,961.75**	**33,129.00**	**35,362.25**

Figure 4.1 presents the data of Table 4.3 graphically as grouped bars, one group per stage. The chart makes visible at a glance both the advantage of FP16 in the first four stages and its slowdown in the ZIGZAG stage, as well as the practically constant cost of HUFFMAN in all three implementations.

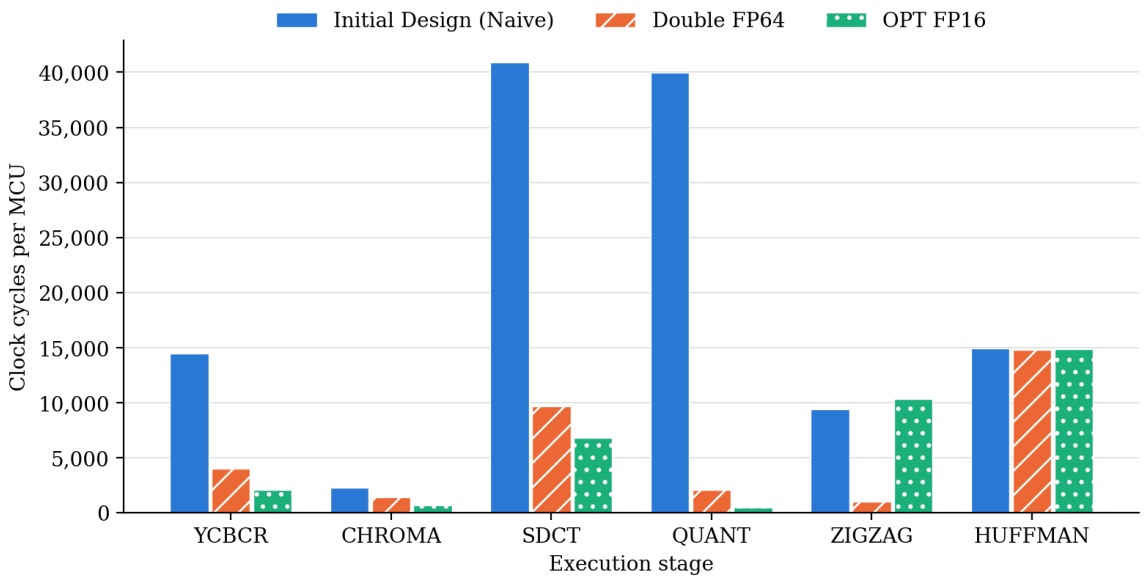


Figure 4.1: Performance comparison per execution stage between the different implementations

4.7.1 The Theoretical SIMD Advantage (Compute-Bound Stages)

An initial analysis of the compute-bound stages (YCbCr, Chroma, DCT and Quantization) reveals the **strong theoretical throughput advantage** of the Packed SIMD FP16 architecture.

By packing multiple half-precision values into a single register, the OPT implementation drastically reduces the cycle count of purely mathematical transformations. In the Quantization stage, for instance, the combination of SIMD vectorization and the "magic number" rounding trick allowed the OPT version to execute in only 478 cycles, an 83.6× speedup over the Initial Design (39,958.5 cycles) and a 4.4× speedup over the Double precision version (2,114.5 cycles).

However, despite processing two or four elements per instruction, the OPT implementation **rarely achieved its theoretical linear speedup** (2× or 4×) over the FP64 equivalent in the other stages. As explored in Sections 4.2 to 4.4, this efficiency loss was caused by significant packing overhead and sub-indexing complexity. Because the SSRs of the Snitch cluster stream data sequentially, the FP16 code frequently had to halt the FPU pipeline to rearrange data within the vector lanes, and the need to extract partial vector results to the ALU, due to the restrictive 64-bit store alignment, diluted the pure SIMD performance gains.

4.7.2 The Architectural Bottleneck: Data Serialization

The **most striking result** of the performance evaluation concerns the Zig-Zag and DPCM stage. This phase is the boundary between floating-point mathematics and integer bit-stream processing.

As seen in Table 4.3, the Double precision implementation executed this stage in **only 1,035 cycles**. By using the ISSR for hardware-level zig-zag fetching and the copift hardware F2I queue, the FP64 pipeline decoupled the FPU from the CPU and achieved near-zero overhead.

Conversely, the OPT (FP16) implementation slowed down sharply, requiring 10,355.25 cycles and thus becoming slower than the Initial Design. Because the packed SIMD vectors had to be serialized for integer Huffman encoding, the CPU was forced to extract each 16-bit element manually, perform bitwise NaN-boxing, and bounce the data between the FPU and the ALU for type conversion. Because this unpacking routine made no use of the hardware queue, its overhead **cancelled the gains of the earlier stages**. As Section 4.8 shows, this overhead comes from the unpacking method and can largely be removed.

4.7.3 The Serial Limit: Entropy Coding

Finally, the Huffman encoding stage remains virtually unchanged in all three implementations (approximately 14,800 to 14,900 cycles). As an inherently scalar, branch-heavy, bit-manipulation algorithm, it cannot be accelerated by the FPU or the SSRs. Consequently, it sets a **hard serial limit** on the single-core pipeline and becomes the primary candidate for multi-core distribution in the subsequent architecture.

4.7.4 Conclusion and Baseline Selection

The aggregated cycle counts are a classic manifestation of *Amdahl's Law* [19]. Although the OPT FP16 implementation outperformed the Double FP64 implementation in four of the six execution stages, the 9,320-cycle penalty of the vector-unpacking routine used in the Zig-Zag stage cancelled all of its computational gains. As a result, the total execution time of the Double precision pipeline (33,129 cycles) is lower than that of the OPT FP16 pipeline (35,362 cycles). Beyond raw speed, the FP64 code is significantly cleaner, free of register-interleaving complexity, ALU memory interference and instruction-buffer overflows.

Therefore, based on its superior overall execution speed, seamless hardware-queue integration and structural simplicity, **the Double Precision (FP64) implementation is selected as the**

Single-Core Baseline. This highly optimized pipeline serves as the foundational building block of the multi-core parallelization strategy discussed in Chapter 5. This selection reflects the implementations available at this stage of the work. The further analysis of Section 4.8 shows that a **hybrid FP16/FP64 pipeline is faster still** on a single core; the reasons for keeping FP64 as the basis of Chapter 5 are discussed at the end of that section.

4.8 Revisiting the Unpack: A Hybrid FP16/FP64 Pipeline

In Sections 4.6 and 4.7, the FP16 pipeline lost to FP64 because of a **single stage**: the Zig-Zag stage, which needed 10,355.25 cycles per MCU instead of 1,035.00. This section looks at why this stage was so slow. It first identifies the cause, then tests other ways of unpacking the FP16 values, and finally presents a **hybrid pipeline** that uses FP16 for the first four stages and FP64 only for the Zig-Zag stage.

4.8.1 Root Cause: NaN-Boxing of Packed Registers

The RISC-V floating-point extensions require a value narrower than the register that holds it to be *NaN-boxed*: all unused upper bits of the register must be set to 1 [16]. A packed SIMD register breaks this rule by design, because its upper bits hold three further FP16 values instead of ones. Any scalar half-precision instruction applied to such a register therefore **sees a canonical NaN** rather than the value stored in its lowest lane.

A micro-benchmark confirms this behaviour. The packed word 0xC4004200C0003C00, which holds the four lanes {1.0, −2.0, 3.0, −4.0}, was converted with fcvt.d.h. Applied directly to the packed register, the instruction returns 0x7FF8000000000000, the canonical NaN. Applied to a correctly NaN-boxed copy of the first lane, it returns 0x3FF0000000000000, i.e. 1.0. This explains the structure of Listing 4.10: the routine had to rebuild the NaN-box in software (0xFFFF0000 | bits) for every coefficient, move it from the integer core to the FPU, convert it twice and move the result back, at a cost of **about 27 cycles per coefficient**. The overhead was therefore caused by **this unpacking method, not by the FP16 arithmetic** of the earlier stages.

4.8.2 Evaluating Unpacking Alternatives

Several ways of getting individual FP16 lanes out of packed storage were tested on the Snitch simulator. Table 4.4 summarises the results; cycle counts refer to one 8×8 block (64 values), and all correct variants were checked *value by value* against a reference.

Table 4.4: Micro-benchmark results of the FP16 unpacking alternatives

Method	Result	Assessment
fcvt.d.h on the packed register	0x7FF8000000000000 (NaN)	Unusable
fcvt.d.h on a NaN-boxed half	1.0 (correct)	Correct, needs NaN-boxing
vfcvt.s.h / vfcvtu.s.h (SIMD widen to FP32)	Correct lanes	Output still packed
vfcvt.x.h (packed FP16 → int16)	Integer rd only; result 0	Unusable
flh per lane + fcvt.d.h	1, −2, 3, −4 (correct)	Clean per-lane unpack
flh widen in zig-zag order, simple loop	852 cycles/block (≈13/value)	Correct
flh widen in zig-zag order, unrolled ×4	544 cycles/block (≈8.5/value)	Correct
Integer-only bit decode (no FPU)	1,601 cycles/block	Correct, slower
Integer-only bit decode, fully unrolled	963 cycles/block	Correct, slower
Original routine (Listing 4.10)	≈1,726 cycles/block (≈27/value)	Reference

The measurements lead to three conclusions:

- **Hardware NaN-boxing through flh:** The flh instruction loads a single 16-bit value from memory into an FP register and NaN-boxes it in hardware. The value is then valid for any scalar conversion, and no integer-core round trip is needed.
- **No direct packed conversion:** The packed FP16-to-integer conversion vfcvt.x.h, which would convert all four lanes at once, is not usable on this hardware and toolchain: the assembler accepts it only with an integer destination register, and the result was 0.
- **FPU route faster than integer decoding:** A pure integer decoding of the FP16 bit fields works correctly but needs 963 cycles per block even when fully unrolled, clearly more than the flh-based route.

4.8.3 Hybrid Pipeline Design

The micro-benchmarks suggest that the cheapest place for the reordering is **not the unpacking loop but the hardware**: the FP64 kernel of Listing 4.9 already performs the zig-zag gather with the ISSR and the conversion to integers through the copift queue. The hybrid pipeline therefore keeps stages 1 to 4 (YCbCr, Chroma, 2D-DCT and Quantization) and the Huffman encoder exactly as in the FP16 implementation, and replaces the Zig-Zag stage by two steps:

- **Stage 5a, widening (widen_fp16_to_f64):** The quantized FP16 blocks are read contiguously, in *natural order*, with flh; each value is widened with fcvt.d.h and stored

with fsd into one of three FP64 buffers in the TCDM (q_Y_d, q_Cb_d and q_Cr_d, 384 values or 3 KB per MCU). Four values are processed per iteration so that independent loads and conversions overlap (Listing 4.11).

- **Stage 5b, FP64 zig-zag (opt_zigzag_and_diff):** The unchanged FP64 kernel of Listing 4.9 then reads the FP64 buffers through the ISSR in zig-zag order, converts them under frep.o, and delivers the integers through the copift queue, including the DPCM of the DC coefficient.

Because the widening step reads memory sequentially, it needs no index computation, and the **reordering stays in hardware**. A *compile-time switch* (USE_HYBRID_ZIGZAG) selects either the hybrid stage 5 or the original FP16 stage 5 in the same source file, so that both versions can be measured on the same build.

Listing 4.11: *Hybrid stage 5a: widening of the quantized FP16 coefficients to FP64 with flh, which NaN-boxes each lane in hardware.*

```
1  void widen_fp16_to_f64(const float16_t *in, double *out, int n) {
2      for (int i = 0; i < n; i += 4) {
3          asm volatile(
4              "flh     f4,   0(%[src]) \n"
5              "flh     f5,   2(%[src]) \n"
6              "flh     f6,   4(%[src]) \n"
7              "flh     f7,   6(%[src]) \n"
8              "fcvt.d.h f28,  f4       \n"
9              "fcvt.d.h f29,  f5       \n"
10             "fcvt.d.h f30,  f6       \n"
11             "fcvt.d.h f31,  f7       \n"
12             "fsd     f28,  0(%[dst]) \n"
13             "fsd     f29,  8(%[dst]) \n"
14             "fsd     f30, 16(%[dst]) \n"
15             "fsd     f31, 24(%[dst]) \n"
16             :: [src] "r"(&in[i]), [dst] "r"(&out[i])
17             : "f4", "f5", "f6", "f7",
18               "f28", "f29", "f30", "f31", "memory"
19         );
20     }
21 }
```

1. Signature; n is a multiple of 4.

2. Four values per iteration.

3. flh loads one 16-bit lane and NaN-boxes it in hardware.

4. Exact widening FP16 → FP64.

5. Contiguous stores into the FP64 buffer read later by the ISSR.

6. Operands and clobbers.

7. End of loop.

4.8.4 Results and Verification

Table 4.5 compares the hybrid pipeline with the FP16 pipeline, both measured on the *same build* and the same test image by toggling USE_HYBRID_ZIGZAG, and with the FP64 baseline of Table 4.3. All values are cycles per MCU.

Table 4.5: Performance of the hybrid FP16/FP64 pipeline compared with the FP64 and FP16 implementations

Stage	Double FP64 (Table 4.3)	OPT FP16 (same run)	Hybrid FP16/FP64
YCBCR	4,039.50	2,103.00	2,100.00
CHROMA	1,456.25	731.00	725.00
SDCT	9,669.25	6,819.00	6,801.00
QUANT	2,114.50	478.00	462.50
ZIGZAG	1,035.00	10,363.00	2,652.00 (1,591.00 widen + 1,061.00 zig-zag)
HUFFMAN	14,814.50	14,962.00	14,961.75
Total Pipeline	**33,129.00**	**35,456.00**	**27,702.25**

The Zig-Zag stage drops from 10,363 to 2,652 cycles, a **3.9× speedup**. The widening step costs about 265 cycles per block (roughly 4 cycles per value), well below the 544 cycles of the best zig-zag-order variant in Table 4.4, because it reads memory sequentially. The FP64 zig-zag inside the hybrid (1,061 cycles) matches the original FP64 kernel (1,035 cycles), which confirms that the kernel runs unchanged. All other stages remain at their FP16 values.

Over the complete pipeline, the hybrid needs **27,702.25 cycles per MCU**. This is 21.9% fewer cycles than the FP16 pipeline of the same run (35,456.00) and 16.4% fewer than the FP64 baseline (33,129.00), a 1.20× speedup, and 4.40× faster than the Initial Design. The hybrid is therefore the **fastest single-core implementation** developed in this thesis.

Correctness was checked on the final output: with the switch set to 0 and to 1, the program produces an **identical 822-byte JPEG bitstream** (identical hex dump). This is expected, since every FP16 value is *exactly representable* in FP64, so the widening step does not change any coefficient.

4.8.5 Implications for the Baseline Selection

The hybrid result corrects the interpretation of Sections 4.6 and 4.7. The slowdown of the FP16 pipeline in the Zig-Zag stage was caused by the unpacking method of Listing 4.10 (software NaN-boxing and a per-value ALU–FPU round trip), not by the FP16 format. Once the unpacking uses flh and the reordering is left to the ISSR, the **SIMD gains of the compute-bound stages are preserved**. The per-stage measurements of Table 4.3 remain valid for the implementations they describe.

The result is subject to the following limitations:

- The hybrid has been measured on a **single core only**; it has not been ported to the 8-core cluster.

- It needs about 3 KB of additional TCDM per MCU for the three FP64 buffers.
- The FP64 column of Table 4.5 comes from an earlier run (Table 4.3); the comparison between FP16 and the hybrid, taken from the same build and run, is the most direct one.
- The test image contains 4 MCUs.

The multi-core design of Chapter 5 was developed on the FP64 baseline selected in Section 4.7, before this finding. Its contributions (the decoupling of DMA transfers through the DM core, the partitioning of the TCDM and the *Dual-Level Bank Staggering*) concern the organisation of memory and synchronisation rather than the number format, and they **remain valid**. A multi-core port of the hybrid would, however, change the buffer sizes and the mix of FP16 and FP64 buffers, and therefore the TCDM layout and bank-staggering analysis of Sections 5.2 and 5.3. The FP64 pipeline is therefore kept as the basis of Chapter 5, and the multi-core port of the hybrid is **left as future work**.

5 Multi-Core Architecture & Memory Optimization

With the single-core baseline definitively established around the Double Precision (FP64) implementation, the focus of the optimization effort shifts from purely compute-bound instruction-level acceleration to *cluster-level parallelization*. The Snitch cluster provides a highly specialized asymmetric architecture, featuring eight dedicated compute cores heavily augmented with Stream Semantic Registers (SSRs) and Floating-Point Units (FPUs), alongside a single Data Mover (DM) core responsible for all Direct Memory Access (DMA) operations.

Transitioning the JPEG compression pipeline to this multi-core environment introduces a new paradigm of challenges. **The primary bottleneck is no longer mathematical throughput**, but rather memory bandwidth, spatial hardware contention, and precise inter-core synchronization. This chapter details the architectural strategies employed to distribute the workload across the cluster, ensuring that the compute cores achieve maximum utilization while operating seamlessly within the strict physical limitations of the Tightly Coupled Data Memory (TCDM).

5.1 Multi-Core Parallelization (Tile-by-Tile, DM Core/DMA)

The multi-core parallelization strategy revolves around a strict architectural *separation of concerns*, fundamentally dividing the execution logic between memory management and computational processing. This separation is enforced at the hardware level by interrogating the core topology using the snrt_is_compute_core() and snrt_is_dm_core() hardware identification functions.

5.1.1 The Role of the Data Mover (DM) Core

In this architecture, the eight compute cores are **completely isolated from the high-latency main memory** (L3 DRAM). The Data Mover (DM) core acts as the exclusive orchestrator of the cluster's memory traffic. The execution follows a highly synchronized "Tile-by-Tile" *Producer-Consumer* model.

For every iteration of the main Minimum Coded Unit (MCU) loop, the DM core utilizes two-dimensional DMA transfers (snrt_dma_start_2d) to fetch the corresponding 16x16 RGB image tiles directly from the main memory into the private L1 TCDM workspaces of all active compute cores. To maximize routing efficiency during this scanning process, the DM core calculates the next memory block using *branchless pointer arithmetic*. By utilizing bitwise masks (is_wrap, mask) to handle row-wrapping boundaries (extra_jump), the DM core entirely avoids branch prediction penalties.

5.1.2 Tile-by-Tile Synchronization and Execution

The parallel execution is governed by a rigid synchronization framework using *hardware barriers* (snrt_cluster_hw_barrier()). The lifecycle of a single MCU tile proceeds *as follows*:

- **Pre-fetch**: The DM core initiates the 2D DMA transfers to populate the compute cores' L1 input buffers.
- **Execution**: Once the hardware barrier signals that the data is resident in the TCDM, the eight compute cores simultaneously wake up and execute the heavily optimized FP64 JPEG pipeline (YCbCr to Huffman) independently on their assigned tiles.
- **Write-back**: Upon completion, the compute cores register their final compressed byte sizes in a shared directory. The DM core then utilizes one-dimensional DMA transfers (snrt_dma_start_1d) to flush the generated bitstreams sequentially into the final L3 DRAM output payload.

This decoupled architecture ensures that **the compute cores never stall waiting for memory transactions**, as all latency is hidden behind the DM core's asynchronous DMA engine.

5.1.3 Execution Flow Diagram

The sequence diagram in Figure 5.1 illustrates the precise synchronization and separation of roles between the L3 Memory, the DM Core, and the Compute Cores during the Tile-by-Tile execution loop.

Besides the tile-by-tile loop, the diagram shows the phases that surround it. In the *initialization phase*, the DM core reads the constant tables (DCT matrix and quantization reciprocals) from L3 memory and copies them with one-dimensional DMA transfers into the workspace of every core; Core 0 writes the JPEG file headers, which the DM core flushes to the output payload before a global barrier releases the loop. In every loop iteration, the DM core fetches **eight different 16×16 tiles, one per compute core**, into shared_input_ptrs[0..7];

after the compute phase, it reads the sizes reported in shared_size_array[0..7] and appends the eight bitstreams to the output payload in core order. After the last iteration, in the *post-loop phase*, the compute cores write the End-of-Image marker (0xFF 0xD9) and the DM core transfers the remaining tail bytes with a final DMA flush. Hardware barriers separate every step in which one side must wait for the other.

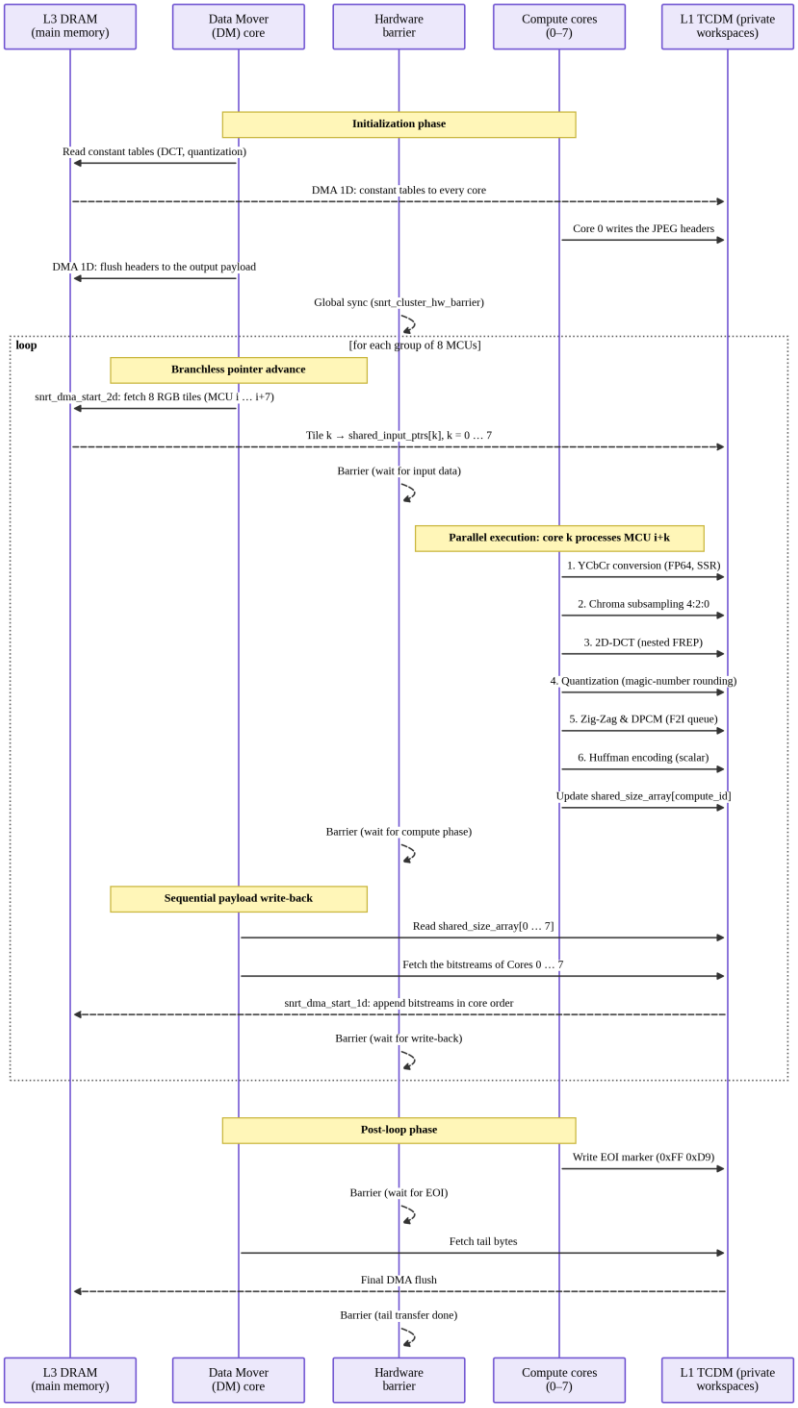


Figure 5.1: Graph showing the complete multicore movement of data through JPEG execution

5.2 TCDM Memory Layout and Workspace Partitioning

The Snitch cluster architecture lacks hardware-managed data caches, relying entirely on a Tightly Coupled Data Memory (TCDM) shared among all cores. This architectural choice removes cache coherence overheads and provides deterministic, single-cycle memory access latencies, provided that spatial contention is avoided. Consequently, **the memory layout must be explicitly managed by software**. To support the multi-core parallelization strategy outlined in Section 5.1, the L1 TCDM was heavily structured into isolated partitions, ensuring conflict-free operation while maximizing the utilization of the available capacity.

5.2.1 The TCDM Root Control Directory

To facilitate the asynchronous "Producer-Consumer" communication between the Data Mover (DM) core and the compute cores, a **centralized control directory** is established at the absolute base address of the TCDM. This directory acts as the hardware-software interface for DMA transfers.

The root directory consists of multiple shared pointer arrays, including shared_input_ptrs, shared_payload_ptrs, and constant table pointers (shared_matrix_ptrs, shared_recip_lum_ptrs, shared_recip_chr_ptrs). Additionally, it contains a shared_size_array which the compute cores use to report the final byte count of their compressed JPEG payloads to the DM core. To ensure strict hardware alignment with the TCDM banking structure, the entire directory size (table_bytes) is aligned to the memory's row boundary using the ALIGN_UP_TCDM macro. In this architecture, a row is defined as the number of memory banks multiplied by the bank size (32 banks × 8 bytes = 256 bytes).

5.2.2 Strict Core Isolation and Partitioning

Immediately following the root directory, the remaining TCDM capacity is strictly partitioned into private, non-overlapping workspaces for each compute core. By assigning a dedicated memory block to each core, the architecture **completely eliminates inter-core data dependencies** and the need for software locks during the computational phases.

Each compute core calculates its absolute base address (my_tcdm_base) by offsetting the end of the root directory (table_bytes) by its unique hardware ID (compute_id) multiplied by the predefined private block size. Based on the maximum memory footprint of the JPEG pipeline for a 4:2:0 Minimum Coded Unit (MCU), the private block size (memory_of_each_core) was

fixed at **14,080 bytes** (equivalent to 55 TCDM rows). This static allocation guarantees that a memory write executed by Core 0 will never corrupt the execution state of Core 1.

5.2.3 Intra-Core Memory Aliasing and Buffer Reusability

While 14,080 bytes per core provides sufficient capacity, the full 4:2:0 JPEG pipeline generates numerous intermediate data arrays across its six processing stages (RGB, YCbCr, 2D-DCT, Quantization, Zig-Zag, and Huffman). If every intermediate matrix were allocated to a unique memory address, the required capacity would severely exceed the L1 limits.

To bypass this physical constraint without sacrificing performance, the implementation employs aggressive **Memory Aliasing**. Instead of declaring separate variables for every stage, two large generic memory buffers are allocated within the core's private workspace: pipeline_workspace_a and pipeline_workspace_b.

As the pipeline progresses, sequential execution stages intentionally overwrite the memory addresses of *previous, now-redundant stages*:

- **Workspace A**: Initially utilized to store the double-precision Luminance (naive_Y) and 4:2:0 Chroma channels (naive_Cb_420, naive_Cr_420) after the color space conversion. During Stage 4, this exact memory region is aliased and overwritten to store the quantized coefficients (naive_quant_Y, naive_quant_Cb, naive_quant_Cr).
- **Workspace B**: Originally allocated for the interleaved Chroma output (naive_CbCr), it is subsequently aliased by the DCT outputs (naive_dct_Y, naive_dct_Cb, naive_dct_Cr) in Stage 3. In the final hardware stages, this block is cast from a double-precision floating-point pointer to a 16-bit signed integer pointer (int16_t *zz_diff_Y), serving as the final bit-stream output buffer before Huffman encoding.

Through this rigorous life-cycle analysis of the data structures, the dynamic memory footprint of the JPEG pipeline is heavily compressed, ensuring that the entire multi-core execution **remains comfortably within the ultra-fast boundaries of the L1 TCDM**.

Figure 5.2 summarizes this organization. The left side shows the global L1 TCDM map: the root directory, aligned to a 256-byte TCDM row, is followed by the private 14,080-byte workspaces of Cores 0 to 7. Of the shared arrays of Section 5.2.1, the figure shows the three that are accessed in every iteration (shared_size_array, shared_input_ptrs and shared_matrix_ptrs). The right side expands the workspace of one core into **five regions**, listed from the lowest address:

- **Constants & LUTs**: the type-conversion look-up table (lut), the DCT matrix (l1_matrix) and the reciprocal quantization tables (l1_recip_lum, l1_recip_chr).
- **Input RGB Tile**: the 16×16×3-byte buffer (input_idx) into which the DM core copies the MCU of the current iteration.
- **Pipeline Workspace A**: the outputs of color conversion and chroma subsampling (naive_Y, naive_Cb_420, naive_Cr_420), later overwritten by the quantized coefficients of Stage 4.
- **Pipeline Workspace B**: the interleaved chroma of Stage 1 (naive_CbCr), overwritten by the DCT outputs of Stage 3 and finally reinterpreted as int16_t for the Zig-Zag and differential outputs of Stage 5 (zz_diff_Y, zz_diff_Cb, zz_diff_Cr).
- **ZigZag Map & Output Payload**: the zig-zag index table (l1_zigzag) and the compressed bitstream of the core (jpeg_output_payload), which stays in the TCDM until the DM core writes it back.

The arrows inside Workspaces A and B mark the order in which the stages overwrite each other, which is the *memory aliasing* described above. The small bank offsets of Section 5.3.4 (+16, +0 and +8 bytes) are not drawn, because they are negligible at the scale of the figure.

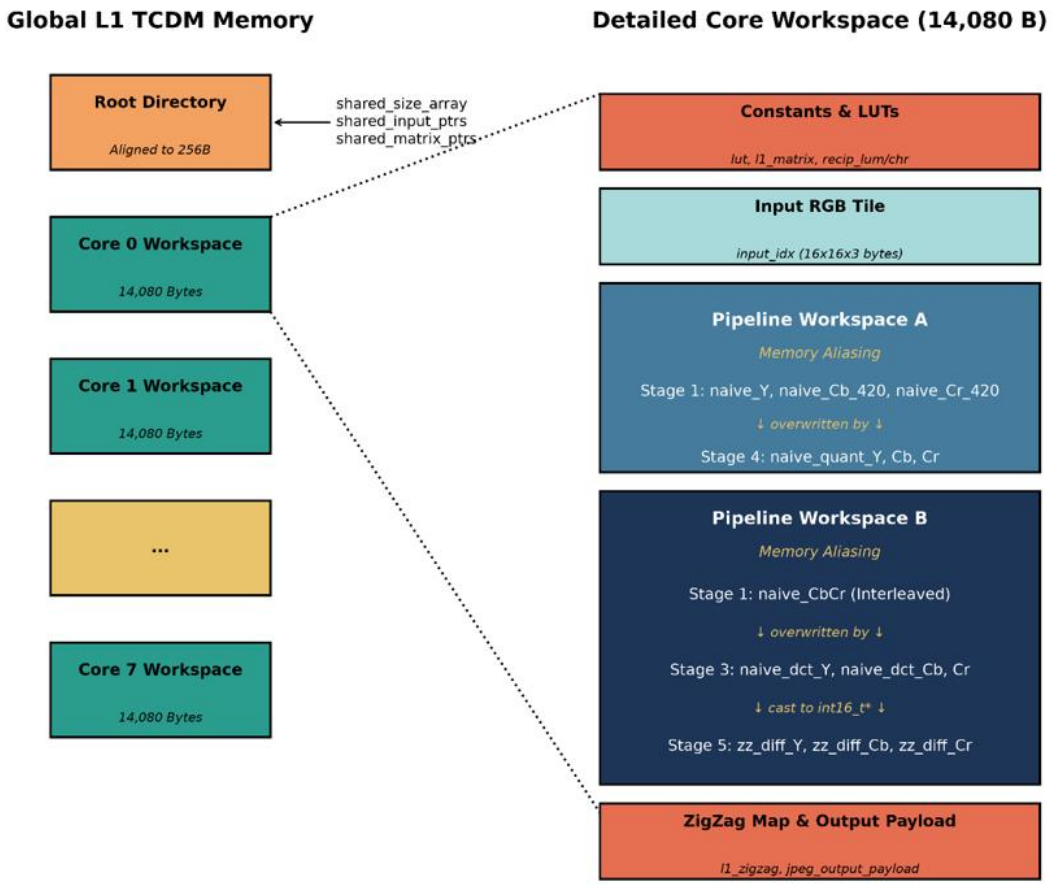


Figure 5.2: Detailed Memory of TCDM belonging to each specific Core

5.3 Resolving TCDM Bank Contention

The Snitch cluster's Tightly Coupled Data Memory (TCDM) is engineered to provide single-cycle access latency to all eight compute cores. To achieve this high bandwidth, the L1 memory is physically divided into 32 independent SRAM banks, with consecutive 64-bit (8-byte) words interleaved across them in a *round-robin* fashion.

While this architecture supports highly parallel accesses, its single-cycle latency guarantee strictly relies on a spatial distribution condition: **no two memory requests in the same clock cycle can target the same physical memory bank**. If multiple hardware units (e.g., SSR Data Movers or ALUs) attempt to access the same bank simultaneously, a hardware conflict known as Bank Contention occurs. The memory interconnect is forced to serialize the requests, introducing severe pipeline stalls and drastically degrading the execution throughput.

5.3.1 Lockstep Execution and the "Bank 13" Collision Scenario

The parallelization strategy of the JPEG pipeline relies on Single Program, Multiple Data (SPMD) execution. All eight compute cores are released from a hardware barrier simultaneously, executing the exact same assembly instructions on their respective image tiles. Because the instruction streams are identical and branchless, the cores execute in a near-perfect *temporal lockstep*.

If the private workspaces of all cores were identically aligned to the start of a TCDM row (e.g., $N_{rows} \times 256$ bytes), this lockstep execution would trigger catastrophic **Lockstep Bank Aliasing**.

For example, consider the Quantization stage, where all cores read from their local l1_recip_lum array. If Core 0 attempts to read the 13th element of this array, it will access Bank 13. Because all cores are executing in lockstep, Cores 1 through 7 will attempt to read the 13th element of their respective arrays at the exact same clock cycle. Due to identical memory alignment, all 8 requests will route directly to Bank 13. The interconnect arbitration will serialize these 8 requests, resulting in an immediate **8-cycle stall for a single memory fetch**. Across the thousands of memory accesses required per MCU, this aliasing effectively reduces the multi-core cluster's performance back to that of a single core.

5.3.2 Prime-Stride Alignment vs. Algorithmic Staggering

A classical theoretical approach to resolving bank aliasing in interleaved memories is *Prime-Stride Alignment*. In this technique, data structures are padded with dummy bytes so that the stride (the distance between corresponding elements across different cores) is relatively prime to the number of memory banks (32). By ensuring a prime stride (e.g., padding structures to occupy exactly 33 banks instead of 32), simultaneous accesses automatically wrap around the interconnect and land on distinct physical banks.

However, Prime-Stride Alignment **inherently wastes precious L1 memory capacity** due to the requisite padding. Given that the entire JPEG pipeline was aggressively memory-aliased to fit within the strict 14,080-byte per-core limit, injecting padding bytes was architecturally prohibitive.

Instead of theoretical padding, this implementation achieves contention-free routing through a highly deterministic, zero-waste technique: **Dual-Level Bank Staggering**.

5.3.3 Level 1: Inter-Core Bank Staggering

To break the inter-core lockstep aliasing, a calculated spatial offset was introduced to the base address of each core's private workspace. This is implemented via the core_bank_offset variable, as shown in Listing 5.1.

Listing 5.1: *Inter-core bank offset applied to the base address of each core's private workspace (C).*

```
1 uintptr_t core_bank_offset = compute_id * 32;
```

This formula shifts the memory map of each successive core by exactly 32 bytes (which corresponds to exactly 4 physical banks of 8 bytes). Consequently, when all eight cores execute the same load instruction *at Cycle T*:

- **Core 0**: accesses Bank X
- **Core 1**: accesses Bank (X+4) mod 32
- **Core 2**: accesses Bank (X+8) mod 32
- **Cores 3–6**: continue the same pattern in steps of 4 banks
- **Core 7**: accesses Bank (X+28) mod 32

Because there are 8 cores and they are spaced by 4 banks each (8×4=32), the simultaneous memory requests are perfectly distributed across the entire width of the TCDM interconnect, entirely eliminating inter-core bank collisions **without wasting a single byte of memory padding**.

Figure 5.3 visualizes this inter-core staggering during a lockstep access: each row corresponds to a compute core and each column to one of the 32 TCDM banks, and Core i accesses Bank 4i, so no two cores request the same bank in the same cycle.

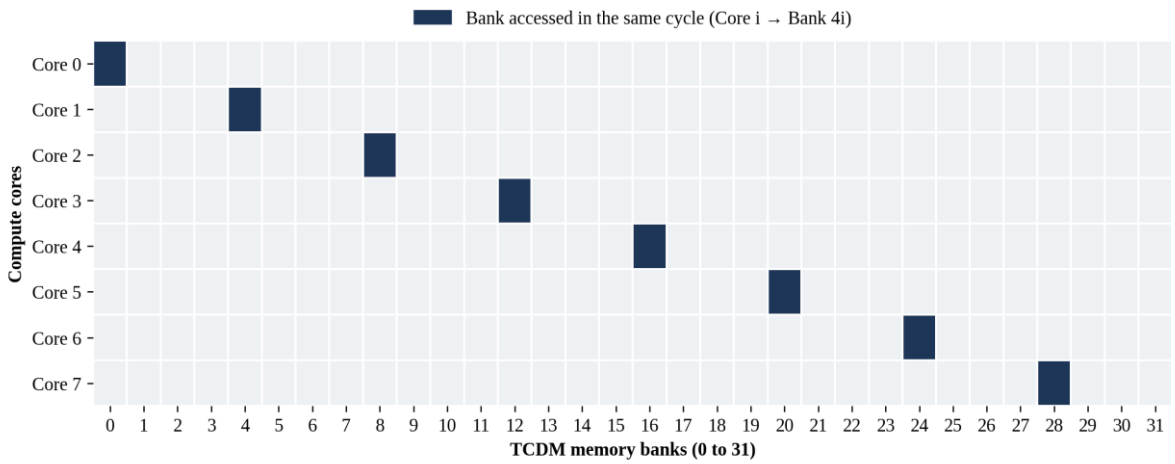


Figure 5.3: Inter-core bank staggering during lockstep execution: Core i accesses Bank 4i, so no two cores collide

5.3.4 Level 2: Intra-Core Workspace Alignment

While the inter-core offset prevents Cores 0 and 1 from colliding with each other, **it does not prevent a core from colliding with itself**. During the heavily optimized 2D-DCT and Quantization stages, each core operates three independent SSR Data Movers (DM0, DM1, DM2) concurrently. If a core's internal input matrix, constant table, and output matrix were identically aligned relative to its own base offset, the three SSRs would attempt to access the same bank simultaneously, causing an internal stall.

To resolve this intra-core contention, deliberate manual offsets were integrated into the *pointer allocation arithmetic*:

- **Constant Tables (+16 bytes)**: The look-up tables and matrices (lut, l1_matrix, l1_recip_lum) are shifted by +16 bytes (2 banks) relative to the workspace base.
- **Pipeline Workspace A (+0 bytes)**: The primary input/output workspace is naturally aligned to the core's staggered base address.
- **Pipeline Workspace B (+8 bytes)**: The secondary aliased workspace is shifted by +8 bytes (1 bank).

Through this microscopic spatial manipulation, the implementation ensures that, for unit-stride access patterns, a standard three-operand SSR instruction (e.g., fmadd.d reading from DM0, DM1 and writing to DM2) will intrinsically route its physical memory requests to **three distinct, non-overlapping SRAM banks**. This dual-level staggering perfectly aligns the software's data layout with the hardware's interconnect topology, unlocking the cluster's peak

theoretical MAC throughput. The column-wise pass of the 2D-DCT, discussed next, is the exception.

5.3.5 The "Jump of 8" (Power-of-Two Stride) Matrix Collision

A specific and severe memory access anomaly arises during Pass 2 of the 2D-DCT algorithm. Because the 2D-DCT is a separable transform, the second pass must process the intermediate matrix column-by-column. In an 8×8 matrix composed of 64-bit double-precision floats, moving vertically to the next element in a column requires **jumping exactly 8 elements (or 64 bytes)**.

Because each TCDM memory bank is exactly 8 bytes wide, a 64-byte memory jump corresponds to a stride of exactly 8 banks. This creates a destructive mathematical phenomenon within a 32-bank memory interconnect system. The number of unique physical banks visited by a Stream Semantic Register (SSR) is determined by the Greatest Common Divisor (GCD) of the stride and the total number of banks, as expressed by Equation (5.1):

$$\text{Unique Banks} = \frac{\text{Total Banks}}{\gcd(\text{Stride},\text{Total Banks})} = \frac{32}{\gcd(8,32)} = \frac{32}{8} = 4 \tag{5.1}$$

Instead of sweeping gracefully across the entire 32-bank memory space, the DM1 SSR (responsible for reading the matrix columns) becomes mathematically trapped in a **repeating 4-bank cycle** (e.g., Banks 0, 8, 16, and 24). This "Jump of 8" stride creates two distinct forms of hardware contention, discussed below.

A) Intra-Core Interception (The "Eclipse" Effect)

While DM1 is trapped oscillating between just 4 banks, DM0 (reading the constant DCT matrix) and DM2 (writing the output) are traversing matrix rows with a stride of 1, meaning they sweep sequentially across all 32 banks. Because DM1 keeps repeatedly striking banks 0, 8, 16, and 24, the sequential SSRs (DM0 and DM2) are mathematically guaranteed to eventually cross its path and attempt to access one of those exact banks at the exact same clock cycle. Much like planetary bodies intersecting orbits, this "*eclipse*" effect causes inevitable, unavoidable intra-core stall cycles during the nested frep loop execution.

Figure 5.4 illustrates this effect for a single core over 32 cycles of the nested FREP loop: DM1 (stride 8) repeatedly hits Banks 0, 8, 16 and 24, while DM0 and DM2 (stride 1) sweep across all 32 banks and eventually collide with it (red cells). In the 32 cycles shown, two such collisions occur, at Cycles 14 and 23.

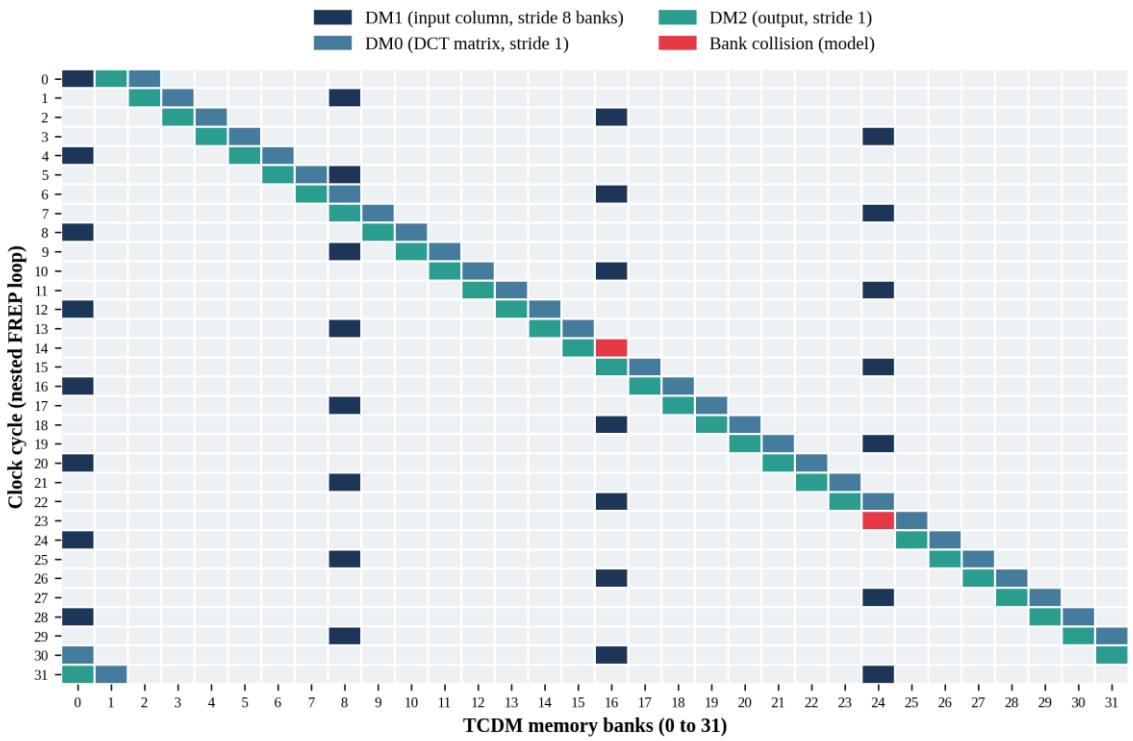


Figure 5.4: Theoretical model of the SSR bank accesses of one core during Pass 2 of the 2D-DCT

B) Inter-Core Overlap Constraint

Furthermore, the inter-core bank staggering (discussed in Section 5.3.3) offsets each core's base address by 4 banks (32 bytes). This results in the following bank access patterns *for column fetches*:

- **Core 0 (starts at Bank 0)**: hits Banks {0, 8, 16, 24}
- **Core 1 (starts at Bank 4)**: hits Banks {4, 12, 20, 28}
- **Core 2 (starts at Bank 8)**: hits Banks {8, 16, 24, 0}

Because the subset of accessed banks repeats every 8 banks, Core 0 and Core 2 utilize the exact same physical banks for their column fetches. If the cores are in perfect hardware lockstep, their DM1 streams avoid each other. However, if Core 0 experiences even a single cycle of execution latency (*jitter*) and falls out of lockstep, its DM1 will instantaneously collide with Core 2's DM1, triggering a **cascading stall across the entire cluster**.

Figure 5.5 extends this view to all eight cores at the same time (24 active SSRs), showing the banks accessed by DM0, DM1 and DM2 in each clock cycle of the nested FREP loop and the resulting bank collisions.

Figures 5.4 and 5.5 are **theoretical models** of the address patterns: they assume that every stream issues one request per cycle in perfect lockstep and mark each cycle in which two requests target the same bank. Figure 5.5 also shows that, although the DM1 streams of different cores avoid each other, the unit-stride DM0 and DM2 streams of every core repeatedly cross the banks used by the DM1 streams of the other cores; this causes most of the collisions of the model. In the real hardware, however, a collision only stalls the losing port for one cycle. This delay shifts the affected stream relative to the others, so the streams leave the colliding phase and their following requests no longer target the same banks. The collisions of the model therefore turn into **isolated one-cycle delays** rather than continuous serialization, which is consistent with the small number of TCDM bank stall cycles measured in Section 5.4.1 (about 78 cycles). Bank contention in this pass is real, but it is **not the bottleneck** of the pipeline.

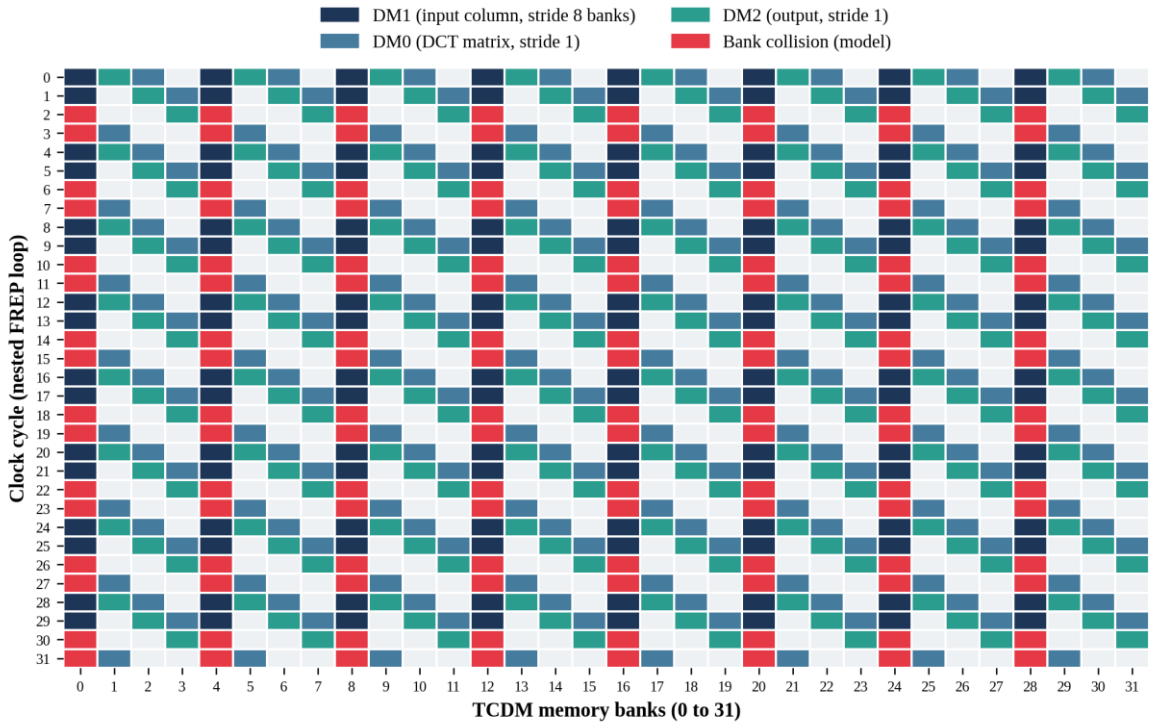


Figure 5.5: Theoretical model of the SSR bank accesses and collisions of all 8 cores during Pass 2 of the 2D-DCT

This anomaly reveals a critical architectural limitation: while Dual-Level Staggering effectively resolves baseline lockstep contention, algorithms exhibiting large, power-of-two memory strides **remain intrinsically hostile to interleaved TCDM architectures**.

5.4 Performance Analysis: Theoretical vs. Empirical Optimizations

5.4.1 Empirical Micro-Benchmarking and Stall Analysis

A) Measurement Methodology

To accurately profile the parallel execution phase, metrics were gathered from a *worker core* (e.g., Core 1) rather than the *master core* (Core 0). Core 0 is responsible for cluster initialization and executing computationally heavy serial tasks, such as printf statements. Including Core 0 in the performance average would skew the data with one-time execution overheads, whereas the worker cores provide a clean profile of steady-state execution stalls.

B) The Bank Staggering Paradox

Initial profiling yielded the following stall summary[5] *without bank staggering*:

- **TCDM_MEM_BANK_STALL**: 78 cycles
- **RAW_ACCEL_HAZARD**: 267 cycles
- **RAW_LOAD_USE_HAZARD**: 2529 cycles

After introducing the Dual-Level Bank Staggering offsets of Section 5.3 (that is, with all optimizations except the Huffman prefetching), the stall profile remained *virtually identical*:

- **TCDM_MEM_BANK_STALL**: 78 cycles
- **RAW_ACCEL_HAZARD**: 268 cycles
- **RAW_LOAD_USE_HAZARD**: 2524 cycles

Because the memory allocated per core is an exact multiple of the local memory's TCDM bank line width, the base address for each core systematically maps to the same memory bank. With the eight cores executing identical code in strict synchronization, theoretical models predicted severe bank conflicts as all cores attempt to access the same relative address simultaneously.

Despite this theoretical expectation, empirical measurements showed **no significant difference in bank stall cycles**. Detailed trace analysis revealed that the recorded bank

[5] TCDM_MEM_BANK_STALL counts the cycles lost to TCDM bank conflicts. RAW_ACCEL_HAZARD counts the cycles in which an instruction waits for a result produced by an instruction offloaded to the accelerator interface (e.g., the FPU). RAW_LOAD_USE_HAZARD counts the cycles in which an instruction waits for the data of a preceding load (a read-after-write load-use dependency).

conflicts did not originate from the data arrays targeted by the staggering offsets. Instead, the collisions stemmed from shared data structures that the staggering offsets do not cover—such as the array responsible for assembling the final Huffman bitstream—which remained unaffected by the alignment adjustments.

Figure 5.6 compares the three stall categories across the successive optimization steps (baseline, with bank staggering, and with bank staggering plus the Huffman prefetching described below), showing that staggering leaves all three categories practically unchanged.

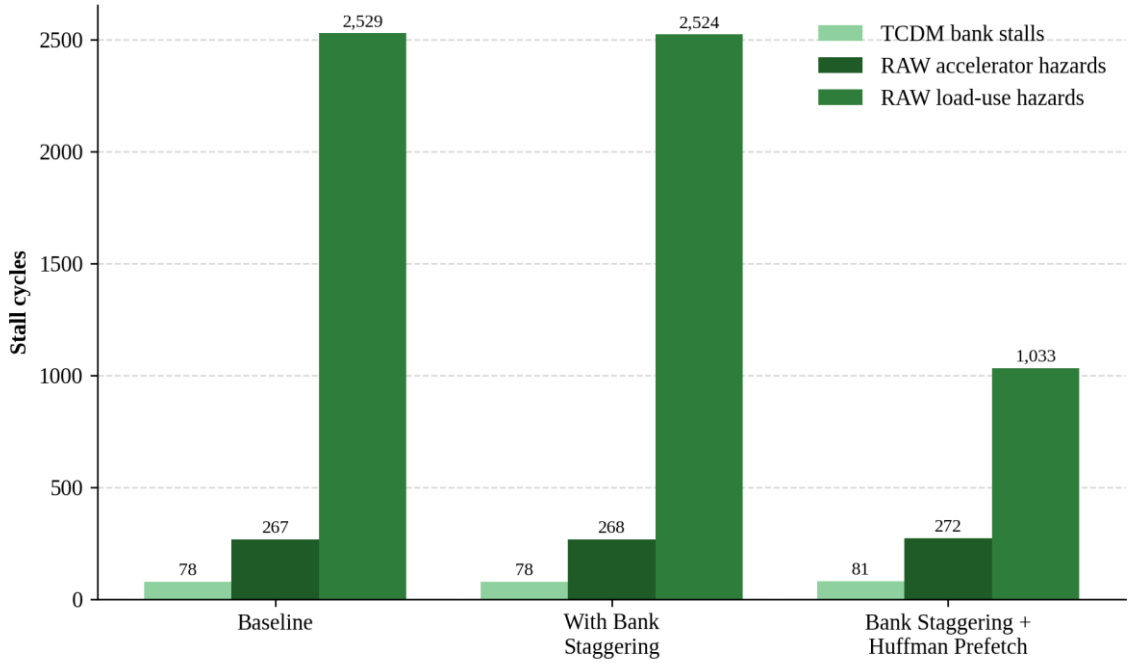


Figure 5.6: Stall cycles per category across the three optimization steps

C) Identifying and Resolving the True Bottleneck

The data revealed that structural contention among multiple cores (TCDM bank stalls) **was not the primary bottleneck**. The overwhelming majority of pipeline delays originated from RAW_LOAD_USE_HAZARDs. This indicates high pipeline latency: the delay between requesting data from memory and that data being available in the register file.

This issue specifically plagued the Huffman encoding phase. Unlike the floating-point operations, which leverage Stream Semantic Registers (SSR) to stream data directly to the FPU and mask memory latency, the scalar Huffman encoder relied on standard load instructions.

By applying instruction scheduling (*software pipelining*) to the Huffman encoder—specifically prefetching data one cycle before it was required—the pipeline could perform

useful work while waiting for the memory response. This optimization yielded a dramatic reduction *in stall cycles*:

- **TCDM_MEM_BANK_STALL**: 81 cycles
- **RAW_ACCEL_HAZARD**: 272 cycles
- **RAW_LOAD_USE_HAZARD**: 1033 cycles (a ~59% reduction)

Figure 5.7 isolates the RAW load-use hazard cycles of the three configurations, highlighting the drop **from 2,524 to 1,033 cycles** once Huffman prefetching is applied.

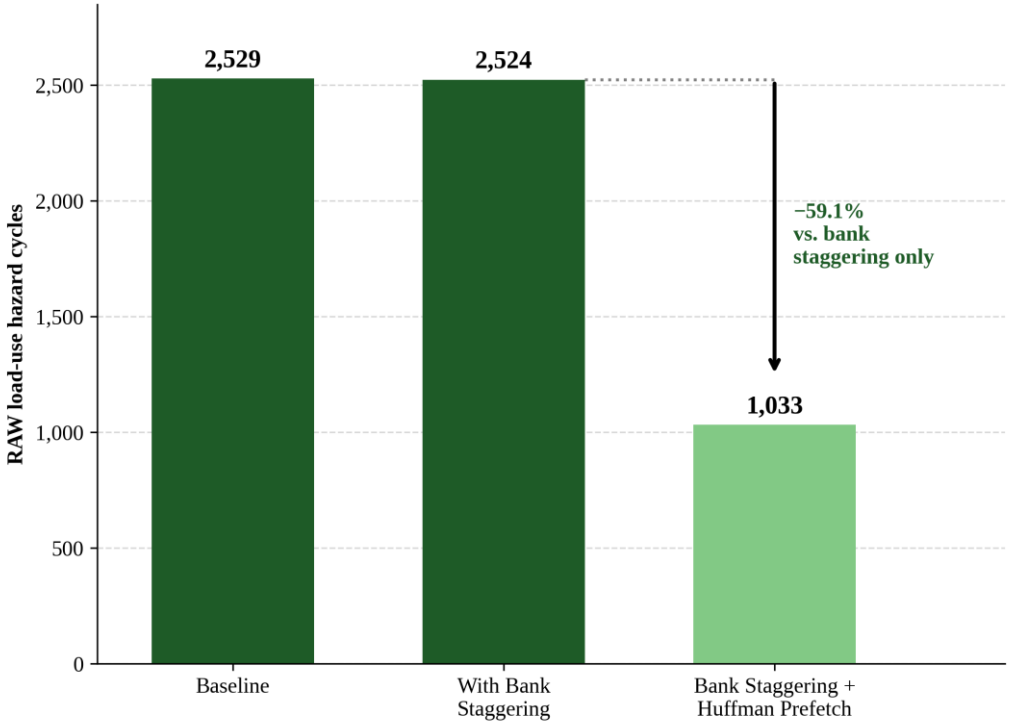


Figure 5.7: Huffman Memory Optimization

D) Architectural Shift in Stall Distribution

By virtually eliminating the Huffman pipeline latency, the overall architectural profile of the core execution shifts. While RAW_LOAD_USE_HAZARDs still represent the largest absolute delay, the processor spends a significantly larger proportion of its remaining idle cycles waiting on true hardware limitations, such as Accelerator Hazards. This shift proves that the software execution is now highly optimized and operating **much closer to the physical limits of the RISC-V Snitch architecture**.

5.4.2 Cluster-Level Speedup and Combined Scaling Efficiency

To quantify the ultimate success of the optimization effort, the fully optimized 8-core cluster execution was benchmarked against the original single-core baseline.

Table 5.1 illustrates the cycle counts required to process a single Minimum Coded Unit (MCU). The single-core baseline represents an isolated core executing the pipeline prior to the Huffman software pipelining optimizations discussed earlier in Section 5.4. The multi-core cluster metrics reflect the performance of a single compute core operating concurrently alongside the other seven cores within the synchronized environment, utilizing all final algorithmic improvements.

Table 5.1: Total Compute comparison per one MCU per Core for a single Core Baseline vs an 8 core system using the memory optimizations.

Pipeline Stage	Single-Core Baseline (Cycles per MCU)	Multi-Core Cluster (Cycles per MCU)
YCbCr Conversion	4,039.50	4,039.25
Subsampling 4:2:0	1,456.25	1,456.25
Forward 2D-DCT	9,669.25	9,642.00
Quantization	2,114.50	2,112.75
ZigZag & Diff	1,035.00	1,028.50
Huffman Encoding	14,814.50	14,229.00
Total Compute	**33,129.00**	**32,507.75**

To process a batch of 8 MCUs sequentially, the baseline standalone core would require 265,032 cycles (8 × 33,129). In contrast, the Snitch cluster processes 8 MCUs simultaneously across its 8 compute cores in just **32,508 cycles**, yielding a final combined speedup of **8.15x**.

This *super-linear scaling* (exceeding the theoretical hardware maximum of 8.0x for 8 cores) is the direct result of compounding architectural and algorithmic optimizations. As the data shows, the effective cycle cost per MCU drops by roughly 621 cycles in the cluster environment. **Approximately 94% of this per-core improvement (585 cycles)** originates from the Huffman encoding stage. Because the single-core baseline was captured before resolving the RAW_LOAD_USE_HAZARD bottlenecks, the 8.15x metric represents the holistic acceleration of the cluster: the simultaneous execution of 8 parallel cores, the complete offloading of memory latency to the asynchronous Data Mover (DM) core, and the localized instruction-level prefetching applied to the scalar Huffman encoder.

6 Conclusion

This chapter summarizes the thesis. Section 6.1 reviews the research, Section 6.2 presents its key architectural insights and contributions, and Section 6.3 outlines directions for future work.

6.1 Summary of the Research

The transition toward ultra-low-power computing demands a fundamental rethinking of how algorithms interact with hardware. This thesis set out to optimize and parallelize the ubiquitous JPEG compression algorithm for the RISC-V Snitch cluster, a highly specialized, cache-less architecture that derives its energy efficiency from explicit software-managed memory and specialized hardware accelerators. By systematically adapting the JPEG pipeline from a naive scalar C implementation to a fully synchronized 8-core execution model, this research demonstrated that peak computational throughput can only be achieved through **rigorous hardware–software co-design**.

6.2 Key Architectural Insights and Contributions

Throughout the optimization process, several critical insights regarding heterogeneous processor architectures and memory management were established:

A) The Illusion of SIMD Dominance and the Cost of Serialization

A central finding of this research was the identification of **data serialization as a catastrophic pipeline bottleneck**. While the Packed SIMD FP16 implementation achieved localized theoretical dominance, such as an 83.6× speedup in the element-wise Quantization stage, it structurally collapsed during the Zig-Zag and DPCM phase. The requirement to manually unpack 16-bit vectors into discrete integers for entropy encoding forced synchronous FPU–ALU interference, inflating the cycle count to over 10,300 cycles. By contrast, the Double Precision (FP64) pipeline utilized hardware queues (copift) and Indirect SSRs to completely decouple floating-point arithmetic from integer logic, proving that **sustained pipeline stability is far more valuable than isolated mathematical throughput**.

B) Zero-Cost Data Routing via Hardware–Software Co-Design

The FP64 baseline proved that **explicitly programming the memory subsystem yields the highest performance returns**. By offloading address generation to the hardware (4D SSR strides for zero-cost matrix transpositions in the 2D-DCT, ISSRs for automated zig-zag fetching, and IEEE-754 mantissa shifts for zero-latency rounding), the compute cores were entirely freed from pointer arithmetic and type-casting overheads.

C) Mastering the TCDM and Super-Linear Cluster Scaling

During the multi-core parallelization phase, the primary bottleneck predictably shifted from FPU utilization to **spatial memory contention**. By strictly enforcing a Producer–Consumer execution model, all Direct Memory Access (DMA) latency was successfully hidden behind the asynchronous Data Mover (DM) core. Furthermore, the implementation of Dual-Level Bank Staggering successfully broke the lockstep execution collision pattern, distributing simultaneous memory requests perfectly across the 32-bank Tightly Coupled Data Memory (TCDM) without wasting capacity on structural padding.

Ultimately, resolving the TCDM contention and applying aggressive instruction scheduling to the scalar Huffman encoder yielded a final compute time of 32,508 cycles for 8 MCUs. This translates to an exceptional, **super-linear architectural speedup of 8.15×** across the 8-core cluster, proving that the compute cores were operating continuously, entirely unhindered by the cluster's complex memory topology.

6.3 Future Work

While this thesis successfully maximizes the mathematical and spatial efficiency of the software execution pipeline, it also highlights the remaining physical boundaries of the Snitch architecture. The findings point to several promising avenues for future research:

- **Hardware-Accelerated Entropy Coding:** As demonstrated in Chapter 4, the Huffman encoding stage represents the hard serial limit of the pipeline. Integrating a dedicated, specialized hardware accelerator specifically designed for bit-stream generation and variable-length integer coding would eliminate this final sequential bottleneck, potentially increasing cluster throughput by an *order of magnitude*.
- **Dynamic Power and Energy Profiling:** With the execution cycles definitively minimized, future work should transition from cycle-accurate simulation to physical

silicon power profiling. Quantifying the exact energy-per-bit cost (pJ/bit) of the Dual-Level Bank Staggering and DM core synchronization will solidify the algorithm's standing in extreme edge and IoT deployments.

- **Advanced Codec Adaptation:** The zero-cost memory routing techniques and SSR stride manipulations developed for the 2D-DCT in this thesis present a foundational framework. These methodologies can be expanded and applied to more modern, computationally demanding video and image codecs, such as HEVC (H.265) or AV1, which rely on larger, more complex matrix transforms and spatial prediction models.

References

[1] F. Zaruba, F. Schuiki, T. Hoefler, and L. Benini, “Snitch: A tiny pseudo dual-issue processor for area and energy efficient execution of floating-point intensive workloads,” *IEEE Trans. Comput.*, vol. 70, no. 11, pp. 1845–1860, Nov. 2021.

[2] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson, 2018.

[3] J. L. Mannos and D. J. Sakrison, “The effects of a visual fidelity criterion on the encoding of images,” *IEEE Trans. Inf. Theory*, vol. IT-20, no. 4, pp. 525–536, Jul. 1974.

[4] *Information Technology—Digital Compression and Coding of Continuous-Tone Still Images—Requirements and Guidelines*, ITU-T Rec. T.81 | ISO/IEC 10918-1, Sep. 1992.

[5] S. van der Walt *et al.*, “scikit-image: Image processing in Python,” *PeerJ*, vol. 2, Art. no. e453, Jun. 2014.

[6] *Commission Internationale de l’Éclairage Proceedings, 1924*. Cambridge, U.K.: Cambridge Univ. Press, 1926.

[7] *Studio Encoding Parameters of Digital Television for Standard 4:3 and Wide-Screen 16:9 Aspect Ratios*, Rec. ITU-R BT.601-7, Mar. 2011.

[8] N. Ahmed, T. Natarajan, and K. R. Rao, “Discrete cosine transform,” *IEEE Trans. Comput.*, vol. C-23, no. 1, pp. 90–93, Jan. 1974.

[9] W.-H. Chen, C. H. Smith, and S. C. Fralick, “A fast computational algorithm for the discrete cosine transform,” *IEEE Trans. Commun.*, vol. COM-25, no. 9, pp. 1004–1011, Sep. 1977.

[10] C. Loeffler, A. Ligtenberg, and G. S. Moschytz, “Practical fast 1-D DCT algorithms with 11 multiplications,” in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process. (ICASSP)*, Glasgow, U.K., 1989, pp. 988–991.

[11] Independent JPEG Group, “libjpeg.” [Online]. Available: https://www.ijg.org/

[12] *Information Technology—JPEG 2000 Image Coding System: Core Coding System*, ITU-T Rec. T.800 | ISO/IEC 15444-1, 2002.

[13] C. E. Shannon, “A mathematical theory of communication,” *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, Jul. 1948.

[14] W. B. Pennebaker and J. L. Mitchell, *JPEG Still Image Data Compression Standard*. New York, NY, USA: Van Nostrand Reinhold, 1993.

[15] D. A. Huffman, “A method for the construction of minimum-redundancy codes,” *Proc. IRE*, vol. 40, no. 9, pp. 1098–1101, Sep. 1952.

[16] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, Document Version 20240411, Apr. 2024. [Online]. Available: https://github.com/riscv/riscv-isa-manual

[17] D. A. Patterson and A. Waterman, *The RISC-V Reader: An Open Architecture Atlas*. Berkeley, CA, USA: Strawberry Canyon, 2017.

[18] A. P. Chandrakasan, S. Sheng, and R. W. Brodersen, "Low-power CMOS digital design," *IEEE J. Solid-State Circuits*, vol. 27, no. 4, pp. 473–484, Apr. 1992.

[19] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Comput. Conf.*, Atlantic City, NJ, USA, 1967, pp. 483–485.

[20] PULP Platform, "Snitch cluster documentation." [Online]. Available: https://pulp-platform.github.io/snitch/

[21] PULP Platform, "snitch_cluster," GitHub repository. [Online]. Available: https://github.com/pulp-platform/snitch_cluster

[22] M. Horowitz, "1.1 Computing's energy problem (and what we can do about it)," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, 2014, pp. 10–14.

[23] G. Paulin *et al.*, "Soft tiles: Capturing physical implementation flexibility for tightly-coupled parallel processing clusters," in *Proc. IEEE Comput. Soc. Annu. Symp. VLSI (ISVLSI)*, 2022, pp. 44–49.

[24] L. Bertaccini, G. Paulin, T. Fischer, S. Mach, and L. Benini, "MiniFloat-NN and ExSdotp: An ISA extension and a modular open hardware unit for low-precision training on RISC-V cores," in *Proc. IEEE 29th Symp. Comput. Arithmetic (ARITH)*, 2022, pp. 1–8.

[25] T. Benz *et al.*, "A high-performance, energy-efficient modular DMA engine architecture," *IEEE Trans. Comput.*, vol. 73, no. 1, pp. 263–277, Jan. 2024.

[26] P. Scheffler *et al.*, "Occamy: A 432-core dual-chiplet dual-HBM2E 768-DP-GFLOP/s RISC-V system for 8-to-64-bit dense and sparse computing in 12nm FinFET," 2025, arXiv:2501.07330.

[27] F. Schuiki, F. Zaruba, T. Hoefler, and L. Benini, "Stream semantic registers: A lightweight RISC-V ISA extension achieving full compute utilization in single-issue cores," *IEEE Trans. Comput.*, vol. 70, no. 2, pp. 212–227, Feb. 2021.

[28] P. Scheffler, F. Zaruba, F. Schuiki, T. Hoefler, and L. Benini, "Sparse stream semantic registers: A lightweight ISA extension accelerating general sparse linear algebra," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 12, pp. 3147–3161, Dec. 2023.

[29] P. Scheffler, L. Colagrande, and L. Benini, "SARIS: Accelerating stencil computations on energy-efficient RISC-V compute clusters with indirect stream registers," in *Proc. 61st ACM/IEEE Design Autom. Conf. (DAC)*, San Francisco, CA, USA, 2024, pp. 1–6.

[30] L. Colagrande and L. Benini, “Dual-issue execution of mixed integer and floating-point workloads on energy-efficient in-order RISC-V cores,” in *Proc. 62nd ACM/IEEE Design Autom. Conf. (DAC)*, San Francisco, CA, USA, 2025.

[31] chendilinn, “simple-jpeg-encoder,” GitHub repository, 2025. [Online]. Available: https://github.com/chendilinn/simple-jpeg-encoder